第九章 ISP 设计实验

第一节 ISP 设计实验步骤

第一步：在 ispLEVER 软件中对 ISP 实验板上的 LC4256V 芯片进行设计。

首先使用电原理图输入方式、或者纯 HDL 语言输入方式、或者二者混合输入方式来输入为 LC4256V 所设计的逻辑；在成功通过波形仿真验证、并且恰当锁定引脚之后，接着执行 Fit Design（适配设计）流程，为 LC4256V 生成可下载的 Fusemap File（熔丝图文件），即在工程文件同一目录下、与工程文件同名的后缀为.jed 的 JEDEC 文件；保存好该文件，准备下载（编程）。

请参考本实验教程第二部分第三章、第四章、第五章和第六章的相关内容。

第二步：在 ispVM System 或 Diamond Programmer 软件中将设计下载到 ISP 实验板中，完成设计调试。

利用 ispVM System 软件，将第一步生成的 LC4256V 的 JEDEC 文件，下载（即在系统编程）到 ISP 实验板上的 LC4256V 芯片中去，调试验证设计结果，具体做法是：利用 ispVM System 软件进行 Scan Chain（点“Scan”图标按钮扫描检测 ISP 实验板上的 ISP 器件）及 Download（点“GO”图标按钮开始下载编程）操作。

或者利用 Diamond Programmer 软件，将第一步生成的 LC4256V 的 JEDEC 文件，下载（即在系统编程）到 ISP 实验板上的 LC4256V 芯片中去，调试验证设计结果，具体做法是：利用 Diamond Programmer 进行 Scan（点“Scan”图标按钮或菜单 Design→Scan 扫描检测 ISP 实验板上的 ISP 器件）及 Program（点“Program”图标按钮或菜单 Design→Program 开始下载编程）操作。

下载成功后，请先按动实验板电源开关断电重启，再观察实际运行效果。

请参考本实验教程第二部分第七章的相关内容。

第二节 ISP 设计实验注意事项

- ispLEVER 软件中的原理图编辑界面，操作起来跟其它软件不太一样：
  - 其它软件中一般是先选中要进行操作的对象，然后再对其进行操作；
  - **ispLEVER 的原理图编辑器中是先指定要进行什么操作，再选定要操作的对象，然后进行相应的操作。**
- 熟练应用常用的快捷键及相应的工具箱图标按钮和菜单命令，可加快电原理图的绘制进度：
  - 快捷键（**粗体为常用的**）：**F2**－**添加元件 Add Symbol**；**F3**－**连线（带电的线）Add Wire**；**F4**－**添加信号线名 Add Net Name**；**F5**－**删除 Delete**；F6－重复画元件 Duplicate；**F7**－**移动（原有连接关系丢失）Move**；**F8**－**拖动（原有连接关系保持不变）Drag**；**Alt+M**－**添加 I/O Marker**；Ctrl+L－画线（不带电的装饰性的线）Add Line；Ctrl+T－添加文本 Add Text；Ctrl+Q－查询信号线、元件等对象信息。
  - 工具箱图标按钮，如图 9-1 所示。

添加元件 Add Symbol	连线（带电）Add Wire	添加总线抽头 Add Bus Tap
添加实例名称 Add Instance Name	添加信号线名 Add Net Name	添加 I/O Marker
编辑引脚属性 Edit Pin Attribute	编辑元件属性 Edit Symbol Attribute	编辑信号线属性 Edit Net Attribute
重复画元件 Duplicate	移动 Move	拖动 Drag
旋转 Rotate	镜像翻转 Mirror	删除 Delete
添加文件 Draw Text	画线（不带电）Draw Line	画矩形 Draw Rectangle
画弧线 Draw Arc	画圆 Draw Circle	高亮显示 Highlight

图 9-1 电原理图编辑界面的工具箱（功能解释与工具箱位置一一对应）

- 菜单命令：Add→Symbol…－添加元件；Add→Wire－连线；Add→Net Name－添加信号线名；Edit→Delete－删除；Edit→Duplicate－重复画元件；Edit→Move－移动；Edit→Drag－拖动；Add→I/O Marker－添加 I/O Marker；Add→Line－画线；Add→Text－添加文本；DRC（Design Rule Check）→Highlight－高亮显示；DRC（Design Rule Check）→Query－查询对象信息；DRC（Design Rule Check）→Consistency Check－检查电路

错误。

- 关于宏库元件问题
  - Lattice 提供的基本元件库有 5 个（库文件名形如“Generic\XXX.LIB”），分别为算术运算库(ARITHS.LIB)、逻辑门库(GATES.LIB)、输入输出引脚库(IOPADS.LIB)、多路选择库(MUXES.LIB)和寄存器库(REGS.LIB)，内含近 80 种元件。基本元件库中元件的功能一般都一望而知，详细使用说明参阅《Lattice 基本元件库参考手册》（PDF 文档）。
  - Lattice 还提供了若干个扩展元件库(Macro Library，即 Lattice 系统宏库，库文件名形如“plsi\xxx.lib”及“symbols\xxx.lib”)，包括 ispLSI 的算术运算宏库(arith.lib)、编码译码器宏库(coder.lib)、计数器宏库(counter.lib)、输入输出引脚宏库(iopin.lib)、逻辑门宏库(gate.lib)、多路选择宏库(mux.lib)、寄存器宏库(register.lib)、一些功能模块宏库(control.lib)以及 MACH 的 TTL 功能宏库(vanttl.lib)、基本功能宏库(vanfunc.lib)、原语元件宏库(vanprim.lib)。宏库简介见本实验教程第二部分第六章（可作为查找宏库元件的索引），详细使用说明参阅《Lattice 系统宏库参考手册》（PDF 文档）。
  - 注意：虽然其中有些元件（包括扩展元件库和基本元件库）不是 74 等标准系列元件，但各种具有标准系列功能的数字电路元件都是齐全的，只要通过先找第六章索引、再查宏库参考手册 PDF 文档的方法一般都能找到所需的元件。另外，TTL 功能宏库(vanttl.lib)中包含部分的 74 系列元件。
  - 提示：在采用 Verilog HDL 或 VHDL 语言与电原理图的混合输入方式进行设计的工程中，ispLEVER 软件要求只能采用扩展元件库中的元件，不能采用基本元件库中的元件！而在利用 ABEL-HDL 语言与电原理图的混合输入方式进行设计时，则可扩展库元件和基本库元件混用！
- 关于物理连接和逻辑连接问题
  - 物理连接是实验板上固定的连接，比如基本 ISP 实验板上的 ISP 器件跟按键、时钟和数码管等外围资源相连，又如扩展实验板上的 ADC0809 跟某些扩展插针引脚相连。
  - 逻辑连接是你自己在软件开发环境中从逻辑上为 ISP 器件所设计出来的电路，来指示分别给 ISP 器件的相应管脚送入或送出什么样的信号，指挥其按照你的设计来工作，完成相应的功能。
  - 逻辑连接必须要与物理连接紧密配合，才能在实验板上实现你所设计的系统。
- 关于仿真测试问题
  - 要重视仿真激励输入波形(.wdl 文件)的绘制以及仿真输出波形的检查，因为它们的作用就是在下载到硬件板子上之前先进行软件仿真模拟得到时序波形图，然后看这些仿真波形与设计要求是否相符。如果不相符就得返回去修改设计；如果相符则就可以下载到硬件板子上去看实际结果了。实验中常用的仿真激励输入波形(.wdl 文件)方法可以参考本实验教程第二部分第三章第四节中“波形编辑功能”这一部分的内容！
  - 各实验题一般只要功能仿真正确即可！
  - 一般都要先通过了仿真测试，然后再去下载！
- 关于锁定引脚问题
  - 最后设计出来的下载到基本 ISP 实验板上去的电路要正常工作，就必须要在设计时为 ISP 器件的各个管脚锁定引脚，目的是让设计的信号知道从 ISP 器件的哪个实际管脚流入或流出，来完成一定的功能。所以设计者一定要知道如何锁定引脚！（参考本实验教程第二部分第三章第七节的相关内容）
- 关于 Verilog 语言的使用
  - ✓ 本部分 CPLD 实验鼓励对 Verilog 语言熟悉的同学利用 Verilog 语言输入设计。
  - ✓ 这里我们用 Lattice 自己的仿真工具 Lattice Logic Simulator 进行仿真，而用 Synopsys Synplicity 的 Synplify 或 Lattice 的 LSE 进行综合。具体参见本实验教程第二部分第三章第六节。
- 关于 VHDL 语言的使用
  - ✓ 本部分 CPLD 实验鼓励对 VHDL 语言熟悉的同学利用 VHDL 语言输入设计。
  - ✓ 这里我们用 Lattice 自己的仿真工具 Lattice Logic Simulator 进行仿真，而用 Synopsys Synplicity 的 Synplify 或 Lattice 的 LSE 进行综合。具体参见本实验教程第二部分第三章第六节。

第三节 ISP 设计实验及相关原理

实验一 ispLEVER 软件开发环境的熟悉实验

要求：按照本实验教程第二部分第四章“ispLEVER 软件开发实例”的步骤并参考本实验教程第二部分第三章相关内容设计电路。以纯电原理图 Schematic 方式或者以电原理图 Schematic 和 HDL 语言混合方式输入设计，创建 Symbol，编辑模拟测试输入激励波形文件（.wdl）或模拟测试向量文件（.abv），并进行模拟波形输出仿真。按照本实验教程第二部分第五章“CPLD 实验板介绍”的内容，结合观察基本 ISP 实验板（即 MAGIC3100 CPLD 实验板）实物，熟悉该实验板的结构、组成，为应用其进行开发设计实验做好准备。

实验二 随机数发生器的设计

要求：设计一个随机数发生器：

1. 利用基本 ISP 实验板（即 MAGIC3100 CPLD 实验板），通过按键触发，在一个数码管上显示该随机数（0~9 的一位随机数即可），并检测其奇偶性。若为偶数，则一个发光二极管亮；若为奇数，则另一个发光二极管亮。
2. 有清零功能。
3. 设计消抖电路，消除在按键过程中产生的抖动现象。
4. 以纯电原理图 Schematic 方式或者以电原理图 Schematic 和 HDL 语言混合方式输入设计，也可以纯 Verilog 语言或纯 VHDL 语言方式输入设计，并编辑模拟测试输入激励波形文件（.wdl）进行波形仿真，验证仿真结果是否符合设计要求。
5. 整个设计完成后，将所产生的熔丝图文件（.jed）下载到基本 ISP 实验板上，进行调试验证和操作演示。

重点：本实验主要学习数码管动态扫描显示和按键消抖技术。

实验原理一：数码管动态扫描显示

- **功能现象：**

四位数码管显示“4321”。

注意：以下介绍的是一个比较通用的四位数码管动态扫描显示的设计举例，但是在实验二中只需固定一个数码管显示而无需扫描显示，不过由于数码管扫描显示技术在后面的多个实验中均要用到，因此最好在实验二中就把它的原理先搞懂学会。

- **实现原理：**

数码管扫描同键盘扫描一样，在单片机和 CPLD/FPGA 应用中也经常遇到，它可以大大地节约 I/O 口的使用，比如以四位数码管的显示为例，本来需占用 4×7 位信号（或 4×7 个 I/O 口），而使用动态扫描显示技术后只需 1×7+4 位信号（或 1×7+4 个 I/O 口）。它的原理和显示器相似，在一个时间点只有一个数码管是点亮的，通过快速地定时对四个数码管进行刷新，由于人眼的视觉暂留效应，只要刷新频率不低于 50 次/秒，则在人眼中就能看到一个完整的“同时显示”的四位数字。

- **部分参考源代码（Verilog 语言）：**

```
//时钟进程，产生各种时钟信号
always @(posedge clk) //clk 为系统时钟信号 6MHz
begin //整体刷新频率为 1000Hz/4=250Hz，即 1 秒钟内对四位数码管都刷新 250 次，大于 50 次/秒的要求
    if(reset==1'b0)count=0; //count 为分频器计数值，用于产生毫秒时钟基准
    else if(count==5999)
        begin
            count=0;
            if(scancnt==3)scancnt=0; //scancnt 为 LED 扫描轮转信号
            else scancnt=scancnt+1;
        end
    else count=count+1;
end
```

```
//数码管扫描显示
always @(posedge clk)
begin
    if(reset==1'b0)
        begin //四位数码管全暗
            LED_A=1;LED_B=1;LED_C=1;LED_D=1;LED_E=1;LED_F=1;LED_G=1;
            LED_VCC1=0;LED_VCC2=0;LED_VCC3=0;LED_VCC4=0;
            //LED_A、LED_B 等为数码管的 7 段码信号
            //LED_VCC1、LED_VCC2、LED_VCC3、LED_VCC4 分别为千位、百位、十位、个位数码管片选信号
        end
    else if(scancnt==0) //注意：单就实验二而言，并不需要 scancnt 变量及其值为 1、2、3 的以下几段代码
        begin //个位数码管显示数字 1，注意 LED_VCC4=1
            case(Data0) //Data0 为个位数码管待显示的数字
                0:begin
                    LED_A=0;LED_B=0;LED_C=0;LED_D=0;LED_E=0;LED_F=0;LED_G=1;
                    LED_VCC1=0;LED_VCC2=0;LED_VCC3=0;LED_VCC4=1;end
                1:begin
                    LED_A=1;LED_B=0;LED_C=0;LED_D=1;LED_E=1;LED_F=1;LED_G=1;
                    LED_VCC1=0;LED_VCC2=0;LED_VCC3=0;LED_VCC4=1;end
                2:begin
                    LED_A=0;LED_B=0;LED_C=1;LED_D=0;LED_E=0;LED_F=1;LED_G=0;
                    LED_VCC1=0;LED_VCC2=0;LED_VCC3=0;LED_VCC4=1;end
                3:begin
                    LED_A=0;LED_B=0;LED_C=0;LED_D=0;LED_E=1;LED_F=1;LED_G=0;
                    LED_VCC1=0;LED_VCC2=0;LED_VCC3=0;LED_VCC4=1;end
                4:begin
                    LED_A=1;LED_B=0;LED_C=0;LED_D=1;LED_E=1;LED_F=0;LED_G=0;
                    LED_VCC1=0;LED_VCC2=0;LED_VCC3=0;LED_VCC4=1;end
                5:begin
                    LED_A=0;LED_B=1;LED_C=0;LED_D=0;LED_E=1;LED_F=0;LED_G=0;
                    LED_VCC1=0;LED_VCC2=0;LED_VCC3=0;LED_VCC4=1;end
                6:begin
                    LED_A=0;LED_B=1;LED_C=0;LED_D=0;LED_E=0;LED_F=0;LED_G=0;
                    LED_VCC1=0;LED_VCC2=0;LED_VCC3=0;LED_VCC4=1;end
                7:begin
                    LED_A=0;LED_B=0;LED_C=0;LED_D=1;LED_E=1;LED_F=1;LED_G=1;
                    LED_VCC1=0;LED_VCC2=0;LED_VCC3=0;LED_VCC4=1;end
                8:begin
                    LED_A=0;LED_B=0;LED_C=0;LED_D=0;LED_E=0;LED_F=0;LED_G=0;
                    LED_VCC1=0;LED_VCC2=0;LED_VCC3=0;LED_VCC4=1;end
                9:begin
                    LED_A=0;LED_B=0;LED_C=0;LED_D=0;LED_E=1;LED_F=0;LED_G=0;
                    LED_VCC1=0;LED_VCC2=0;LED_VCC3=0;LED_VCC4=1;end
                default:begin //数码管全暗
                    LED_A=1;LED_B=1;LED_C=1;LED_D=1;LED_E=1;LED_F=1;LED_G=1;
                    LED_VCC1=0;LED_VCC2=0;LED_VCC3=0;LED_VCC4=1;end
            endcase
        end
    else if(scancnt==1)
```

```
        begin //十位数码管显示数字 2，注意 LED_VCC3=1
            case(Data1) //Data1 为十位数码管待显示的数字
                //略
            endcase
        end
    else if(scancnt==2)
        begin //百位数码管显示数字 3，注意 LED_VCC2=1
            case(Data2) //Data2 为百位数码管待显示的数字
                //略
            endcase
        end
    else if(scancnt==3)
        begin //千位数码管显示数字 4，注意 LED_VCC1=1
            case(Data3) //Data3 为千位数码管待显示的数字
                //略
            endcase
        end
end
```

在上述参考源代码中，通过 scancnt 随时钟的变化分别对 DATA0、DATA1、DATA2、DATA3 进行刷新，它们分别代表的就是数码管的个位、十位、百位、千位。而 case 语句其实是在译码，将 DataX（X=0、1、2、3）所代表的数字翻译成数码管 7 段的段码值。

此外，四位数码管扫描显示的功能也可用电原理图方式实现，如图 9-2 所示，其原理与上述 Verilog 源程序相似。

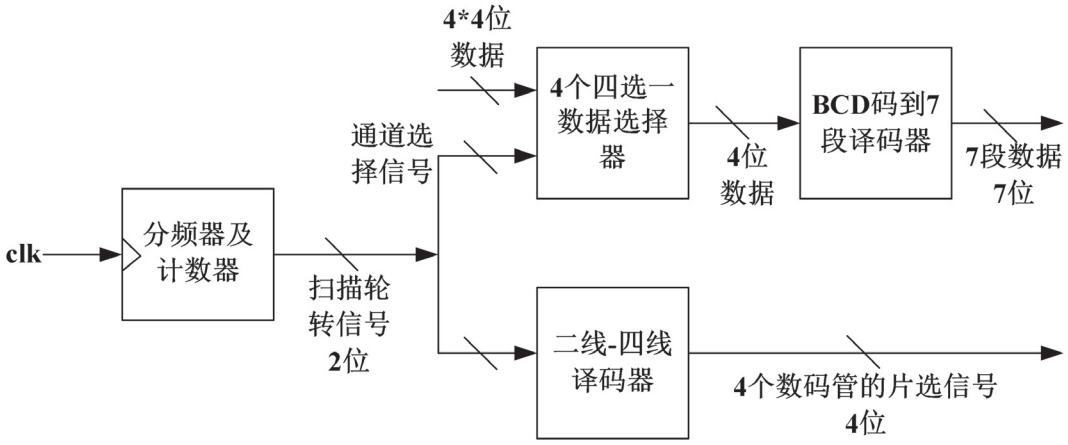


图 9-2 数码管扫描显示电原理图

实验原理二：按键消抖

按键抖动在单片机、CPLD、FPGA 等很多设计中都会遇到，因为任何机械按键在按动时都会产生若干毛刺，从而影响程序的正确判断，可能会被误以为有多次按键按下。图 9-3 是按键按下时产生波形的示意图。

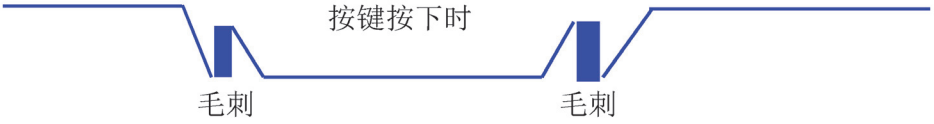


图 9-3 按键抖动示意图

按键消抖功能，也叫防颤、去抖动，就是要去除毛刺，使得一次按键按下只产生一个脉冲。其部分参考源代码（Verilog 语言）如下：

```
parameter s0=2'b01,s1=2'b10; //定义两种状态
//分频产生周期为 10ms 左右的时钟 keyclk，因为一般机械按键的抖动约持续 10ms 左右；也可直接用低频时钟
//状态机激励
```

```
//pre_s 和 next_s 即为状态机指针，分别表示现状态和次状态
always @(posedge keyclk)
begin
    if(RESET==0) pre_s=s0;
    else pre_s=next_s;
end

//按键检查
always @(posedge keyclk) //din 是作为输入的消抖前按键信号
begin
    case(pre_s)
        s0:
            begin
                if(din==0) next_s=s1; //din 是低电平有效，独立按键按下为 0
                else begin next_s=s0;dout=1; end //dout 是作为输出的消抖后按键信号
            end
        s1:
            begin
                if(din==0) dout=0; //连着两拍 din 均为 0 即为正常按键而非抖动
                else begin next_s=s0;dout=1; end
            end
        default: next_s=s0;
    endcase
end
```

另外，也可用电原理图形式来实现按键消抖电路，例如多级 D 触发器方案、计数器延时方案等。注意：按键的抖动时间大概是 10ms 左右，消抖电路时钟可用 128Hz 或 64Hz，更低的消抖时钟消抖效果更好，但按键的响应会显得更慢些。同时注意：CPLD 实验板上的独立按键按下为 0，弹起即常态为 1。几种消抖电路参考方案如图 9-4 所示。

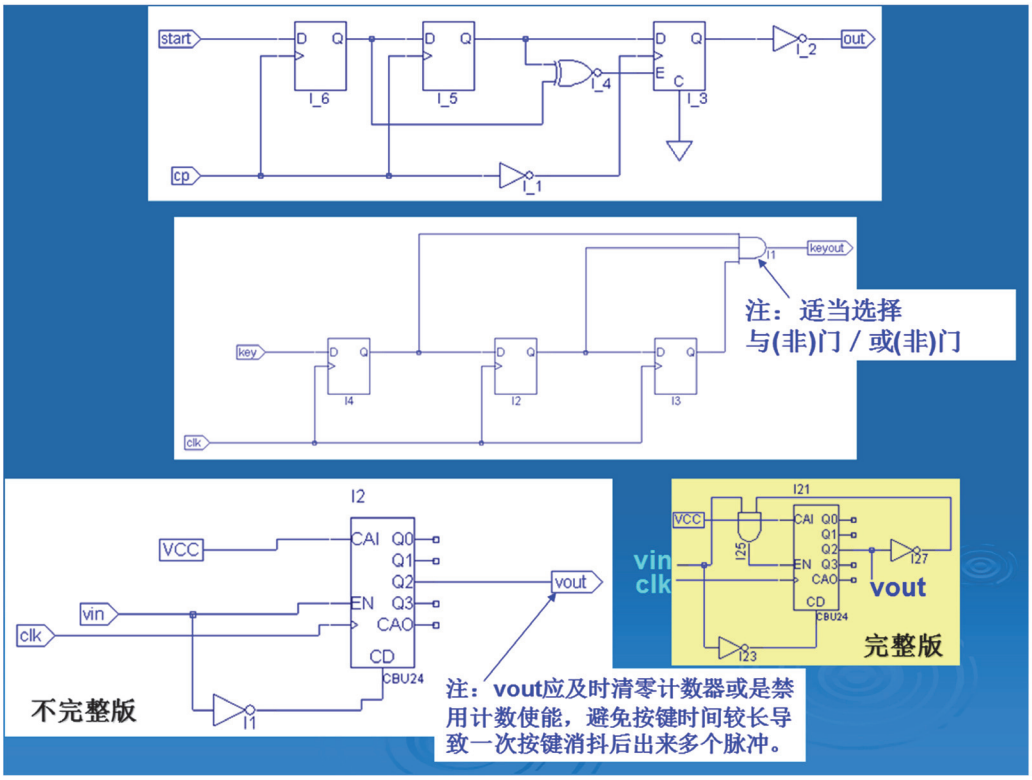


图 9-4 按键消抖电路参考方案

实验三 模拟掷骰子游戏系统的设计

要求：设计一个模拟掷骰子游戏系统：

1. 利用基本 ISP 实验板（即 MAGIC3100 CPLD 实验板），通过按键触发模拟掷骰子，在数码管上显示两个随机数。若这两个随机数之和为 7 或 11，则判为赢，发光管亮，其余判为输，发光管不亮。掷 7 次为一轮，统计 7 次中赢的次数，分别在数码管上显示出所掷的次数和赢的次数。
2. 有清零功能：每一轮结束后所掷次数和赢的次数均清零，准备重新开始下一轮的计数（提示：可以设置一个所掷次数和赢的次数均为 0 的初始/过渡状态；也可以在上一轮 7 次掷完后，再掷一次即认为新的一轮开始）。
3. 设计消抖电路，消除在按键过程中产生的抖动现象。
4. 以纯电原理图 Schematic 方式或者以电原理图 Schematic 和 HDL 语言混合方式输入设计，也可以纯 Verilog 语言或纯 VHDL 语言方式输入设计，并编辑模拟测试输入激励波形文件（.wdl）进行波形仿真，验证仿真结果是否符合设计要求。
5. 整个设计完成后，将所产生的熔丝图文件（.jed）下载到基本 ISP 实验板上，进行调试验证和操作演示。

实验四 数字 8 循环移位显示系统的设计

要求：设计一个数字 8 循环移位显示系统：

1. 利用基本 ISP 实验板（即 MAGIC3100 CPLD 实验板），设计实现一个数字 8 在几个数码管上自动循环移位显示，可通过一个或两个按键来切换左移或右移两种方向，即可采用下列两种方式之一：
   (1). 一键方式：一个键按一下，8 自动循环左移；这个键再按一下，8 再自动循环右移；
   (2). 两键方式：一个键按一下，8 自动循环左移；另一键再按一下，8 再自动循环右移。
2. 设计消抖电路，消除在按键过程中产生的抖动现象。
3. 要求分别用以下两种方法来设计实现：(1). 纯电原理图 Schematic 方式；(2). 纯 Verilog 语言或纯 VHDL 语言方式。并编辑模拟测试输入激励波形文件（.wdl）进行波形仿真，查看结果是否符合所做设计。
4. 整个设计完成后，将所产生的熔丝图文件（.jed）下载到基本 ISP 实验板上，进行调试验证和操作演示。

提示：

✓ 在设计方案上，可用一键方式，也可用两键方式；按键可用边沿触发，也可用电平触发。
建议用一键边沿触发方式！

✓ 在实现方案设计时，注意实验板上的数码管显示是扫描方式显示的；注意思考实验板上的数码管何时有显示、何时全灭无显示；注意实验板上的数码管都是七段逐段驱动的。
要求实现 4 个数码管的数字 8 循环移位显示！

✓ **要求分别用电原理图方式和纯 HDL 语言方式实现！**
(1). **电原理图方式：**以双向移位寄存器为核心，配合 T 触发器，实现一键边沿触发（而非电平触发）切换左右移，注意左右移时补入位的设计及置数功能的应用。边沿触发按键需要加消抖电路。
(2). **纯 HDL 语言方式：**实际上比电原理图方式更简单也更易实现。

实验五 十字路口交通灯控制系统的设计

要求：设计一个十字路口交通灯控制系统：

1. 利用基本 ISP 实验板（即 MAGIC3100 CPLD 实验板），参考《电子系统设计》教材第 2 章中的“十字路口交通灯控制系统”设计举例，设计一个十字路口交通灯控制系统。
2. 用数码管显示倒计时的时间。
3. 如用到按键，则需设计消抖电路，消除在按键过程中产生的抖动现象。
4. 以纯电原理图 Schematic 方式或者以电原理图 Schematic 和 HDL 语言混合方式输入设计，也可以纯 Verilog 语言或纯 VHDL 语言方式输入设计，并编辑模拟测试输入激励波形文件（.wdl）进行波形仿真，验证仿真结果是否符合设计要求。
5. 整个设计完成后，将所产生的熔丝图文件（.jed）下载到基本 ISP 实验板上，进行调试验证和操作演示。

提示：本实验也可参考《电子系统设计》（第四、五版）2.6.3 节“应用可编程逻辑器件设计交通灯控制系统”中的部分内容，但要注意以下地方需修改：

P.69 图 2-76 中，S_i 从上到下应为 S_0、S_1（在右边）、S_2、S_3、S_4（在右边）、S_5；

P.71 第 5 行 if(state==2 && REQUA && num[7:4]>4'd2)应改为：

if(state==2 && LAMP[1]==1 && REQUA && num[7:4]>4'd1)

P.71 第 8 行 if(state==5 && REQUB && num[7:4]>4'd2)应改为：

if(state==5 && LAMP[1]==1 && REQUB && num[7:4]>4'd1)

注：《电子系统设计》（第四、五版）以前印刷的版本才需作上述修改，最新印刷的版本上述地方均已修改完毕。

实验六 数字电压表的设计

要求：熟悉扩展实验板（注：扩展实验板详见第八章“结合扩展实验板的数模混合电子系统设计”），掌握扩展实验板上的硬件电路，并以该扩展实验板与基本 ISP 实验板（即 MAGIC3100 CPLD 实验板）相结合，以纯 Verilog 语言或纯 VHDL 语言方式输入设计，也可以电原理图 Schematic 和 HDL 语言混合方式输入设计，来实现一个具有数字电压表等功能的数模混合的测控电子系统，具体要求能测量显示扩展实验板上运放输出端 J4 的直流电压值，并可适当添加发挥功能，然后仿真验证、下载调试和操作演示。注意：要求显示电压值至少保留小数点后两位有效数字，并且显示电压值与万用表实测电压值之间的误差不大于 0.02V。

提示：

1、 实验题所述功能：

- ■ 扩展板是受控对象，基本板上的 ISP 芯片是数字控制器，二者连成一个数模测控系统（测：电压；控：数码管显示）。
- ■ 实质上要实现的具体要求：电压表实现（对照万用表测量运放输出值即 ADC 的输入值；用按键来切换 4051 的通道，调节电位器，显示三位有效数字的电压）；同时加上切换 4051 通道号的按键消抖，并用一个数码管显示其通道号。
- ■ 主要实现难点在于数制转换：送显时。可用查表法或直接计算法。
- ✓ 一般同学都是先把例如 255/256*500 的值算出来之后（假设不考虑四舍五入），再想办法把这个结果值由二进制转换成十进制送去显示。
- ✓ 往届同学的巧妙方法：一步到位，在计算上式值的同时就完成数制转换。提示：255 * 5 = 4 * 256 + (0.9x) * 256。

2、扩展板简介：

- ■ 输入直流电压、电位器（调输入电压）、电阻网络（自动通过 4051 调节增益——通道号 C、B、A 由 ISP 实验板上的 ISP 芯片里送过来调节反馈电阻；或通过 S2 拨码开关手工调节反馈电阻；从而调输出电压）、AD0809 电路图重要（注意转换时间、8 位数字量、CLOCK、START）。
- ■ 另外扩展板上有两组灯，一组是直接驱动，一组是继电器驱动（注意：继电器工作需要三极管进行电流驱动的，其有效电平与直接驱动的刚好相反：直接驱动是电平为 1 时亮，而继电器驱动是电平为 0 时亮）。注：如果扩展接口引脚不够用的话，可以只实现电压表功能，而不用控制扩展板上 LED 灯的亮灭。

3、注意事项：

1)、 AD0809 输出的 8 位的高低位不要搞反，如搞反则旋转电位器时显示电压值会跳变得很厉害。

2)、 使用万用表，配合对比观察本设计显示的电压值与万用表实测电压值之间的误差。

3)、 第八章第五节的例程对比三档时不够准确（仅实现电压表时可不用比较模块）。

4)、 注意在第八章第四节中提到的电平转换板的反向驱动问题，即从扩展板送到基本板去的信号电平已含取反，而从基本板去扩展板的信号电平也已含取反，所以在设计时可能需要再加反门来得到原信号电平。

实验七 乐曲演奏器的设计

要求：利用基本 ISP 实验板（即 MAGIC3100 CPLD 实验板），以纯 Verilog 语言或纯 VHDL 语言方式输入设计，也可以电原理图 Schematic 和 HDL 语言混合方式输入设计，实现一个乐曲的自动演奏器，同时 LED 灯会随着音高而亮灭，并可适当添加发挥功能。

重点：灵活的分频操作以及 LED 灯的显示技术。

- **实现原理：**

1、音调的控制

频率的高低决定了音调的高低。音乐的十二平均律规定，每两个 8 度音（如简谱中的中音 1 与高音 1）之间的频率相差一倍。在两个 8 度音之间，又可分为 12 个半音，每两个半音的频率比为 2^(1/12)（即 2 的十二分之一次方），约等于 1.059463；另外还规定，音名 A（简谱中的低音 6）的频率为 440Hz，音名 B 到 C 之间、E 到 F 之间为半音，其余为全音。由此可以计算出从 A（简谱中的低音 6）到 al（简谱中的高音 6）之间每个音名的频率依次为：A－440Hz，B－493.88Hz，C－523.25Hz，D－587.33Hz，E－659.25Hz，F－698.46Hz，G－783.99Hz，a（简谱中的中音 6）－880Hz，b－987.76Hz，c－1046.50Hz，d－1174.66Hz，e－1318.51Hz，f－1396.92Hz，g－1567.98Hz，al－1760Hz。

2、音长的控制

音符的持续时间必须根据乐曲的速度及每个音符的节拍数来确定。例如当乐曲的最短音符为 4 分音符时，如果将全音符的持续时间设为 1s 的话，则只需再提供一个 4Hz 的时钟频率即可产生 4 分音符的时长。

- **部分参考源代码（Verilog 语言）：**

```
//音阶和频率的对应关系（通过音阶查找相应频率）
always @(tone,reset)
begin
    if(reset==0)
    begin
        EnSp<=0; //EnSp 为蜂鸣器的使能信号，为 0 时为停顿
        led<=8'b11111111; //LED 灯全灭
    end
    else
        begin
            case(tone) //tone 为曲谱中的音符
                0:begin led<=8'b11111111; EnSp<=0; end //停顿，灭灯
                1:begin origin<=11468; led<=8'b11111110; EnSp<=1; end //音符 1
                2:begin origin<=10215; led<=8'b11111100; EnSp<=1; end //音符 2
                3:begin origin<=9099; led<=8'b11111000; EnSp<=1; end //音符 3
                4:begin origin<=8591; led<=8'b11110000; EnSp<=1; end //音符 4
                5:begin origin<=7653; led<=8'b11100000; EnSp<=1; end //音符 5
                6:begin origin<=6818; led<=8'b11000000; EnSp<=1; end //音符 6
                7:begin origin<=6074; led<=8'b10000000; EnSp<=1; end //音符 7
                8:begin origin<=5733; led<=8'b00000000; EnSp<=1; end //高音 1
                default:begin led<=8'b11111111; EnSp<=0; end //停顿，灭灯
            endcase
        end
end

//蜂鸣器产生各音节基准频率
always @(posedge clk or negedge reset) //clk 为时钟源 6MHz
begin
```

```
    if(reset==0)
        begin
            CntSp<=0; //CntSp 为产生蜂鸣器发声频率的计数器
            sp<=1; //sp 为输出给蜂鸣器发声的信号
        end
    else if(EnSp==1)
        //origin 越大，减的时间越长，sp 的周期越长，频率就越低，音调越低越粗
        //origin 越小，减的时间越短，sp 的周期越短，频率就越高，音调越高越尖
        begin
            if(CntSp==0) begin CntSp<=origin; sp<=~sp; end
            else CntSp<=CntSp-1;
        end
    else sp<=1;
end

//曲谱
//音律就是通过曲谱顺序演奏的，而每个节拍都有相应的音阶
//以下为《小星星》的音乐
always @(posedge Clock4Hz or negedge reset) //Clock4Hz 为 4Hz 时钟信号，控制音符播放一拍的时间长短
begin
    if(reset==0) CntSone<=0; //CntSone 是控制歌曲进度的信号
    else
        begin
            case(CntSone)
                0:      tone<=1;
                2:      tone<=1;
                4:      tone<=5;
                6:      tone<=5;
                8:      tone<=6;
                10:     tone<=6;
                12:     tone<=5;
                13:     tone<=5;

                16:     tone<=4;
                18:     tone<=4;
                20:     tone<=3;
                22:     tone<=3;
                24:     tone<=2;
                26:     tone<=2;
                28:     tone<=1;
                29:     tone<=1;

                32:     tone<=5;
                34:     tone<=5;
                36:     tone<=4;
                38:     tone<=4;
```

```
                40:     tone<=3;
                42:     tone<=3;
                44:     tone<=2;
                45:     tone<=2;

                48:     tone<=5;
                50:     tone<=5;
                52:     tone<=4;
                54:     tone<=4;
                56:     tone<=3;
                58:     tone<=3;
                60:     tone<=2;
                61:     tone<=2;

                default: tone<=0;
            endcase

            if(CntSone==67) CntSone<=0;
            else CntSone<=CntSone+1;

        end
end
```

实验八 简易计算器的设计

要求：利用基本 ISP 实验板（即 MAGIC3100 CPLD 实验板），以纯 Verilog 语言或纯 VHDL 语言方式输入设计，也可以电原理图 Schematic 和 HDL 语言混合方式输入设计，实现一个简易计算器，按照键盘阵列上的数字标记，完成一位数的加“+”、减“-”运算，利用“Z”复位，并可适当添加发挥功能。

重点：键盘阵列扫描、数码管扫描显示技术及状态机的应用。

实验原理一：键盘阵列扫描

- **功能现象：**

在键盘阵列中，从第 1 个按键到第 16 个按键，每一个按键代表了一个二进制数，可通过 LED 灯发亮的方式表现出来。

- **实现原理：**

键盘扫描在实际应用中很常见，无论是在单片机还是 FPGA/CPLD 应用中都能看到它的身影。在单片机和 FPGA/CPLD 应用中，键盘扫描的原理其实是一样的，只是在实现方法上有所不同。它最大的好处就是能节约 I/O 口，比如在本例中，16 个按键如果按照普通的方法来解决，那需要 16 个 I/O 口，但通过扫描的方式来完成，却只需要 8 个 I/O 口。

扫描方法的原理是：轮流扫描每一行，在扫描一行的时候读取四列的电平，判断有无按键按下，如图 9-5 所示。

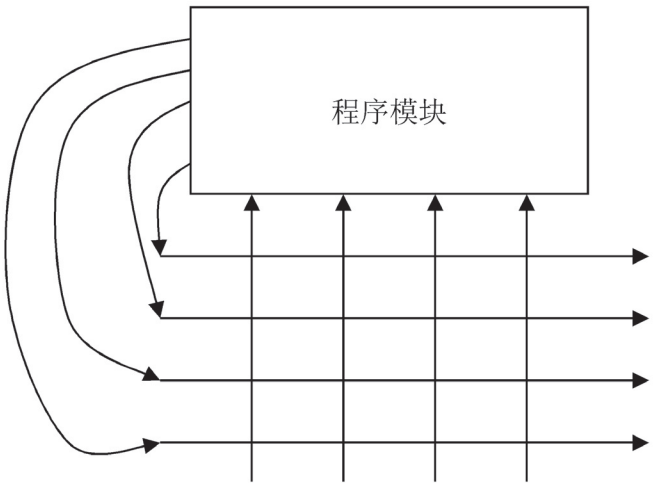


图 9-5 键盘扫描方法原理图

键盘阵列模块的每个交叉点处都是一个按下后能接通列与行的开关。假设四列为输入（从左到右依次为 0、1、2、3），四行为输出（从上到下依次为 0、1、2、3）。**在程序中，输出行值是在程序模块中人为制造赋予的，因为就是要逐行扫描；然后输入列值是要靠行值在按键按下交叉点接通的情况下转送过来输入到程序模块中的。**试想，行也作为输入，不行，因为程序不能人为给输入赋予值。又试想，列也作为输出，似乎也可以，输出的值也是可以读取的，这些列的值是被动地决定于输入行以及外界按键是否按下的，应该也可以读的；但是作为输入显得更自然一点，以读取外界的输入值。

在程序中，控制四行输出值依次为 0001、0010、0100、1000，这样每个扫描周期（如 10ms）内就只有一行输出 1，如果在这一周期内有按键按下，相应的那个交叉点接通，那么行输出的那个 1 就会传给接通的那一列，作为输入传进程序模块去，所以这时那一列输入也肯定为 1，所以整个 8 位（列在前、行在后）就有两个 1，其中有一个对应列的 1 和一个对应行的 1。

比如左上角按键按下时：

①. 第一个扫描周期，行是 0001，第 0 列得到这个 1 故也为 1，而其余列因为没有按下，故仍为 0，所以这时的 8 位（列在前，行在后）是：00010001，识别出了这个按键，一个灯亮。

②. 第二个扫描周期，行是 0010，左上角按键按下也无法让列的 0 得到这个 1，因为这时第 0 行是 0，所以第 0 列只能在按键按下时也得到 0，其余不变，所以这时的 8 位是：00000010，落在下述 Verilog 参考源程序的 case default 中，全部灯都不亮。

③. 同理，第三个扫描周期时的 8 位是 00000100、第四个扫描周期时的 8 位是 00001000，全部灯也不亮。

由于扫描周期为 10ms 左右，扫描很快，所以在按下左上角按键时，虽然相对应的灯只亮了 40ms 左右中的 1/4

时间，其余的若干 ms 没有亮，但是仍然可以看到它是会发亮的，只不过连续下来后觉得它有点闪，即时亮时灭的，而其余无关灯则一直保持不亮。另外，在全部没按键接通时，默认的列值均为 0。

与下面的部分参考源代码相对应的扫描过程如下（其示意图如图 9-6 所示）：

1、CLK(扫描时间)，仅 KEY_OUT(0)输出 1 的同时，读 KEY_IN 电平，然后进行译码；

2、CLK+1(扫描时间)，仅 KEY_OUT(1)输出 1 的同时，读 KEY_IN 电平，然后进行译码；

3、CLK＋2(扫描时间)，仅 KEY_OUT(2)输出 1 的同时，读 KEY_IN 电平，然后进行译码；

4、CLK＋3(扫描时间)，仅 KEY_OUT(3)输出 1 的同时，读 KEY_IN 电平，然后进行译码。

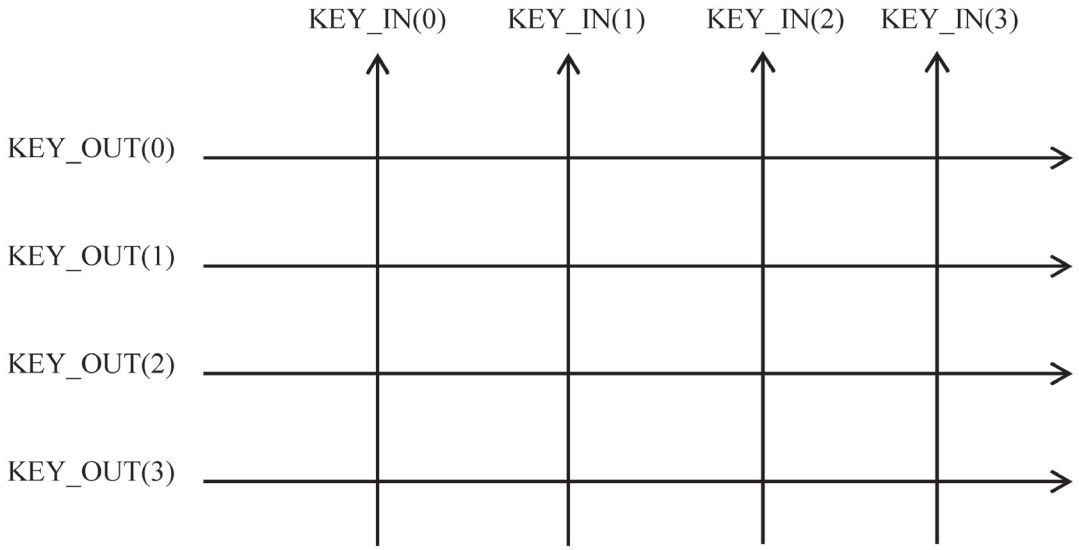


图 9-6 键盘扫描方法示意图

- **部分参考源代码（Verilog 语言）：**

```
always @(posedge clk or negedge reset) //clk 为低频时钟源如 128Hz
begin
    if(reset==0)
        begin
            scanvalue<=1; //scanvalue 记录扫描数据
            led<=5'b11111; //5 位 led 的亮灭显示对应所按下的键；其值为 0 时有效，LED 发亮
            combvalue<=0; //combvalue 为行扫描和列扫描的组合值
        end
    else
        begin //每若干 ms 进行一次键盘扫描
            key_out<=scanvalue; //输出扫描值；key_out 对应 row 行
//注意：对于未按下键时默认电平为 0 的阵列（如 MAGIC3100 板），扫描有效电平应取为 1，例如 0001；
//对于未按下键时默认电平为 1 的阵列（如 EDA-E 板），扫描有效电平应取为 0，例如 1110。
//请思考为什么需要这样？
            case(scanvalue)
                4'b0001: scanvalue<=4'b0010;
                4'b0010: scanvalue<=4'b0100;
                4'b0100: scanvalue<=4'b1000;
                4'b1000: scanvalue<=4'b0001;
                default: scanvalue<=4'b0001;
            endcase
            combvalue<={key_in,key_out}; //key_in 对应 column 列
            case(combvalue)
                8'b00010001: led<=5'b11110; //对应键盘“1”
                8'b00100001: led<=5'b11101; //对应键盘“2”
```

```
                8'b01000001: led<=5'b11100; //对应键盘“3”
                8'b10000001: led<=5'b11011; //对应键盘“X”

                8'b00010010: led<=5'b11010; //对应键盘“4”
                8'b00100010: led<=5'b11001; //对应键盘“5”
                8'b01000010: led<=5'b11000; //对应键盘“6”
                8'b10000010: led<=5'b10111; //对应键盘“Y”

                8'b00010100: led<=5'b10110; //对应键盘“7”
                8'b00100100: led<=5'b10101; //对应键盘“8”
                8'b01000100: led<=5'b10100; //对应键盘“9”
                8'b10000100: led<=5'b10011; //对应键盘“Z”

                8'b00011000: led<=5'b10010; //对应键盘“0”
                8'b00101000: led<=5'b10001; //对应键盘“+”
                8'b01001000: led<=5'b10000; //对应键盘“-”
                8'b10001000: led<=5'b01111; //对应键盘“=”

                default: led<=5'b11111; //无键盘按下，全部 LED 灯都不亮
            endcase
        end
    end
```

实验原理二：简易计算器的部分参考源代码（**Verilog** 语言）：

```
//产生 10ms 时钟的进程
always @(posedge clk or negedge reset) //clk 为系统时钟信号 6MHz
begin
    if(reset==1'b0) timecnt<=0; //timecnt 为分频计数器，用来得到 10ms 时钟
    else if(timecnt==29999)
        begin
            time10ms<=~time10ms; //time10ms 为 10ms 时钟
            timecnt<=0;
        end
    else timecnt<=timecnt+1;
end

//键盘扫描进程
always @(posedge time10ms or negedge reset)
begin
    if(reset==0)
        begin
            scanvalue<=1; //scanvalue 用于记录扫描数据
            flag_Data<=0; //flag_Data 为输入数据顺序标志
            flag_Over<=0; //flag_Over 为完成计算标志
            flag_Cal<=0; //flag_Cal 用于标记运算方式，1－加法，2－减法
            int_Data0<=0;
            int_Data1<=0; //int_Data0 和 int_Data1 为待运算的两个整数
            PointTime<=1; //PointTime 为数码管中间的“：”，在这里用于指示“+”或“-”运算符的输入
            led_int_Data2<=10;
```

```
        //led_int_Data2 为送去第 2 个数码管显示的数据，为 10 表示正数不显示，为 11 表示负数显示“-”
        int_Data<=0; //int_Data 为运算结果
    end
else
    begin
        key_out<=scanvalue; //输出扫描值

        case(scanvalue) //扫描值移位
            4'b0001: scanvalue<=4'b0010;
            4'b0010: scanvalue<=4'b0100;
            4'b0100: scanvalue<=4'b1000;
            4'b1000: scanvalue<=4'b0001;
            default: scanvalue<=4'b0001;
        endcase

        case({key_in,key_out}) //翻译扫描结果
            8'b00010001: //对应键盘“1”
                begin
                    case(flag_Data)
                        0: begin led_int_Data2<=10;int_Data0<=1;int_Data<=1;flag_Data<=1; end //第 1 个数
                        2: begin int_Data1<=1;int_Data<=1;flag_Data<=3; end //第 2 个数
                        default:;
                    endcase
                end

            8'b00100001: //对应键盘“2”
                begin
                    case(flag_Data)
                        0: begin led_int_Data2<=10;int_Data0<=2;int_Data<=2;flag_Data<=1; end //第 1 个数
                        2: begin int_Data1<=2;int_Data<=2;flag_Data<=3; end //第 2 个数
                        default:;
                    endcase
                end

            //对应键盘“3”到“9”及“0”的源代码与 1 和 2 类似，注意行列扫描码的不同即可，故略

            8'b10000001:; //对应键盘“X”

            8'b10000010:; //对应键盘“Y”

            8'b10000100: //对应键盘“Z”，复位
                begin
                    flag_Over<=0;
                    flag_Data<=0;
                    int_Data0<=0;
                    int_Data1<=0;
                    PointTime<=1;
                    led_int_Data2<=10;
                    int_Data<=0;
```

```
                end

            8'b00101000: //对应键盘“+”，准备做加法
                begin
                    if(flag_Data==1) begin flag_Cal<=1;PointTime<=0;flag_Data<=2; end
                end

            8'b01001000: //对应键盘“-”，准备做减法
                begin
                    if(flag_Data==1) begin flag_Cal<=2;PointTime<=0;flag_Data<=2; end
                end

            8'b10001000: //对应键盘“=”，进行运算
                begin
                    case(flag_Cal)
                        1: begin int_Data<=int_Data0+int_Data1;led_int_Data2<=10; end //加法
                        2: begin
                            if(int_Data0>=int_Data1)
                                begin
            int_Data<=int_Data0-int_Data1;led_int_Data2<=10;
            //减法，差为正数，第 2 个数码管不显示表示正数（为 10 时设计译码使对应数码管全暗）
                                end
                            else
                                begin
            int_Data<=int_Data1-int_Data0;led_int_Data2<=11;
            //减法，差为负数，第 2 个数码管显示“-”号表示负数（为 11 时设计译码使对应数码管显示“-”）
                                end
                            end
                        default:;
                    endcase

                    PointTime<=1;
                    flag_Cal<=0;
                    flag_Data<=0;
                end

            default:; //无键盘按下
        endcase
    end
end

//给数码管传输数据
//其中数码管显示译码部分略，参见实验二中的数码管扫描显示原理
always @(clk,reset)
begin
    if(reset==0)
        begin
            led_int_Data1<=0;led_int_Data0<=0; //第 1 个数码管不显示，第 3、4 个数码管为 0
        end
```

```
    else
        begin
            if(int_Data>9)
                begin
                    led_int_Data1<=1;led_int_Data0<=int_Data-10; //分解成 BCD 码
                end
            else
                begin
                    led_int_Data1<=10; //第 3 个数码管不显示
                    led_int_Data0<=int_Data;
                end
        end
end
```

实验九 PC 串口通信系统的设计

要求：利用基本 ISP 实验板（即 MAGIC3100 CPLD 实验板），以纯 Verilog 语言或纯 VHDL 语言方式输入设计，也可以电原理图 Schematic 和 HDL 语言混合方式输入设计，实现一个 PC 串口通信的系统，完成实验板与 PC 机串口之间的数据发送和接收功能，并可适当添加发挥功能。

重点：串口通信的发送和接收的实现技术，以及数码管扫描显示、按键消抖技术和状态机的应用。

- **功能现象：**

在 PC 机上打开“串口调试助手”软件（***注：此软件可从实验 FTP 的“实验相关软件”目录下载，也可在实验室电脑桌面上“电子系统设计实验”文件夹中的“ISP”文件夹中找到）***，选中“十六进制显示”和“十六进制发送”两个复选框，波特率设为 9600，串口根据情况设置（如图 9-7 所示）。在下面的发送框中输入两位 BCD 码，点击“手动发送”，则数字由 PC 机通过串口发给基本 ISP 实验板，这时会在实验板的数码管上显示出来这两个数。而按下基本 ISP 实验板上的独立按键 A，则数字由实验板通过串口发给 PC 机，这时在串口调试助手上面的接收框中会显示“56”。

图 9-7 串口通信实验示意图

- **实现原理：**

串口控制器分为发送和接收两个部分。

✓ 串口发送部分：其默认配置波特率为 9600、8 位数据、1 位起始位、1 位停止位、无奇偶校验位。在代码中发送部分出现的“1010101100”，左边第一位‘1’是停止位，右边第一位‘0’是起始位，中间的数据是 56（可以看成是十六进制或 BCD 码）。发送过程是，在串口波特率 9600 的激励下，将 10 位二进制码从右到左依次发送。

✓ 串口接收部分：比串口发送部分复杂一些。首先要有一个时钟 Clock16，其频率是波特率时钟的 16（约 625/39）倍，也就是说一个数据位有 16 个 Clock16 周期，这是串口控制器惯用的采样时钟，目的是在位数据波形的中间进行取样。串口接收部分的时序示意图如图 9-8 所示，其中的①②③④⑤⑥⑦⑧为采样点。

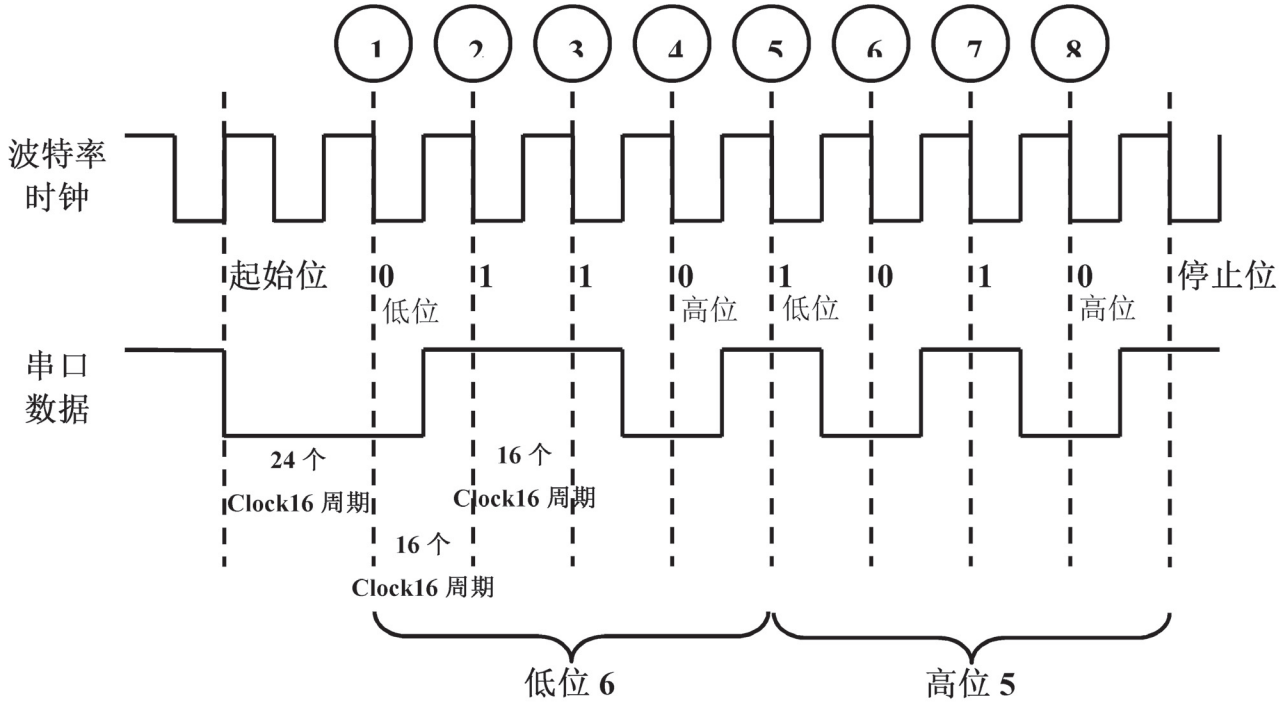


图 9-8 串口通信接收部分的时序示意图

- **部分参考源代码（Verilog 语言）：**

```
//时钟产生进程
always @(posedge clk or negedge reset)
begin
    if(reset==0);
    else
    begin
        if(ClockCount==624)begin Clock9600<=1; ClockCount<=0;end
        else begin Clock9600<=0; ClockCount<=ClockCount+1;end
        //Clock9600 为 9600 波特率时钟，其频率为 6MHz/625=9600Hz
        //ClockCount 为产生 Clock9600 的计数器

        if(ClockCount_Rx==38) begin Clock16<=1;ClockCount_Rx<=0;end
        else begin Clock16<=0; ClockCount_Rx<=ClockCount_Rx+1;end
        //Clock16 为 9600 波特率时钟的 16 倍频采样时钟，在接收时使用，因 625/16≈39（虽为约数，但经测试可用）
        //ClockCount_Rx 为产生 Clock16 的计数器
    end
end

//串口发送部分，即从 ISP 实验板发送到 PC 上，用串口调试助手显示
//Tx 为串口发送数据，锁定在实验板 ISP 芯片上的 97 脚，已通过硬件连线与串口模块的发送端连接
//Send_en 为串口发送使能，0 有效，0 表示发送使能
//Send_over 为串口发送完成，0 有效，0 表示发送完成
//Send_count 为串口发送时的位发送计数
//Send_data 为发送寄存器
always @(posedge Clock9600 or posedge Send_en)
```

```
begin
    if(Send_en==1) begin Send_count<=0;Tx<=1;Send_over<=1;end //发送未使能，发送未完成
    else if(Send_count==9)begin Tx<=Send_data[9];Send_over<=0;end //发送完成，总共发送 10 次（1 位起始位，8 位数据位，1 位停止位，共 10 位，无奇偶校验位）
    else begin Tx<=Send_data[Send_count];Send_count<=Send_count+1;end
end
//发送激励
always @(posedge dout or negedge reset or negedge Send_over) //dout 是按键消抖后的输出，即按键按一下触发开始发送
begin
    if(reset==0)begin Send_en<=1;Send_data<=10'b1000000000;end //初始化不使能发送，发送数据为全 0
    else if(Send_over==0)begin Send_en<=1;Send_data<=10'b1000000000;end //发送完毕，则恢复发送不使能
    else
        begin
        Send_en<=0; //使能发送
        Send_data<=10'b1010101100; //发送数据为：停止位 1，数据 56，起始位 0
                                   //因为从第[0]位开始发，最后发第[9]位，所以正好先发低位，后发高位
        end
end

//串口接收部分，即从 PC 上的串口调试助手发送到 ISP 实验板上，用数码管显示
//串口接收检测起始位，即发送过来的第[0]位为 0，一旦检测到即开始接收数据
//Rx 为串口接收数据，锁定在板上的 98 脚，已通过硬件连线与串口模块的接收端连接
//Rx_Hold 为串口接收到起始位的标志信号，1 有效，表示接收到了
//Rx_Valid 标志接收到一字节有效数据，1 有效，表示接收到了 8 位数据
//Rx_Data 为接收寄存器
always @(Rx or reset)
begin
    if(reset==0) Rx_Hold<=0; //初始化时不接收保持
    else if(Rx==0) Rx_Hold<=1; //通过 Rx_Hold 信号启动接收进程
    else Rx_Hold<=0;
end
//接收进程
always @(posedge Clock16 or negedge reset)
begin
    if(reset==0) begin Rx_Valid<=0;m<=0;end //初始化时接收 8 位数据尚为无效，用 m 计 Clock16 个数为 0
    else
        begin
            case(m)
                0: //检测启动信号
                    begin
                        if(Rx_Hold==1) //接收启动信号已有效，即已收到起始位了
                            begin
                                m<=m+1;
                            end
                        else
                            begin
```

```
                    m<=0; //无效
                    Rx_Valid<=0; //无效
                end
            end

            //经过 24 个 Clock16 周期后，采第 1 位数据即最低位[0]
            24: begin Rx_Data[0]<=Rx;m<=m+1;end

            //经过 16 个 Clock16 周期后，采第 2 位数据即次低位[1]；后面依此类推
            40: begin Rx_Data[1]<=Rx;m<=m+1;end

            56: begin Rx_Data[2]<=Rx;m<=m+1;end
            72: begin Rx_Data[3]<=Rx;m<=m+1;end
            88: begin Rx_Data[4]<=Rx;m<=m+1;end
            104: begin Rx_Data[5]<=Rx;m<=m+1;end
            120: begin Rx_Data[6]<=Rx;m<=m+1;end

            //经过 16 个 Clock16 周期后，采第 8 位数据即最高位[7]
            136: begin Rx_Data[7]<=Rx;m<=m+1;end

            //Rx_Valid 赋 1，标志串口数据接收完成；这时经过 16 个 Clock16 周期应收到了停止位
            152: begin Rx_Valid<=1;m<=m+1;end

            168: begin m<=0;Rx_Valid<=0;end //恢复为初始态，接收个数为 0，接收完成无效
            default: m<=m+1; //每隔一个 Clock16 周期来一个上升沿，其它数时 m 一直在加 1
        endcase
    end
end
//输出接收到的串口数据，送到数码管上去显示
always @(negedge Rx_Valid or negedge reset) //Rx_Valid 的下降沿说明一次接收已经完成才由 1 变 0 的
begin
    if(reset==0) begin Data1<=0;Data2<=0;end
    else
        begin
            Data1<={Rx_Data[3],Rx_Data[2],Rx_Data[1],Rx_Data[0]}; //接收到的低 4 位
            Data2<={Rx_Data[7],Rx_Data[6],Rx_Data[5],Rx_Data[4]}; //接收到的高 4 位
        end
end
```

实验十 PS/2 键盘打字系统的设计

要求：利用基本 ISP 实验板（即 MAGIC3100 CPLD 实验板），以纯 Verilog 语言或纯 VHDL 语言方式输入设计，也可以电原理图 Schematic 和 HDL 语言混合方式输入设计，实现一个 PS/2 键盘打字的系统，当在连接好的 PC 机标准 PS/2 键盘上打数字时，在实验板的数码管上显示出相应数字，并可适当添加发挥功能。

重点：PS/2 通讯协议的帧结构及其应用，以及数码管扫描显示技术。

- **功能现象：**

连接好计算机标准 PS/2 键盘，打开实验板电源，随意按下 PS/2 键盘上面一横排主键盘区的 0~9 数字键，在实验板的数码管上会显示相应的数字。

注意：首先请确认实验板上的拨码开关均拨向数字“1”、“2”、“3”、“4”一端，因为 PS/2 与拨码开关共享了相同的 CPLD 引脚。只要将拨码开关拨向数字一端也就不会影响 PS/2 了。

注意：在使用 PS/2 键盘时，如果出现实验板上对应数码管不断闪烁的情况，请马上关闭实验板电源，这是因为一些非常古老的 PS/2 键盘功率过大，使实验板电源衰落而不断复位。

- **实现原理：**

下面主要针对目标外设的特性进行分析，介绍标准 PS/2 键盘的时序图、操作方法以及编码形式。大家一定要把 90% 的精力放到对目标外设的分析上，因为真正的开发能力不是指你对语句的熟悉程度，而是指你对目标特性的把握和分析能力。

下面整理了一些 PS/2 的资料。希望大家在开发中也养成一种习惯：在做任何项目之前，都上网搜搜，网上资源丰富多元，有大量资料可供参考，别人遇到的问题也可引以为戒，许多源码也不妨借鉴一下。开发的目的不仅是出新产品，而且是在最短的时间内开发出新产品。

1. 物理连接器

一般，具有五脚连接器的键盘称之为 AT 键盘，而具有六脚 mini-DIN 连接器的键盘则称之为 PS/2 键盘。其实这两种连接器都只有四个脚有意义。它们分别是 Clock（时钟脚）、Data（数据脚）、+5V（电源脚）和 Ground（电源地）。在 PS/2 键盘与 PC 机的物理连接上只要保证这四根线一一对应就可以了。PS/2 键盘靠 PC 的 PS/2 端口提供+5V 电源；另外两个脚 Clock（时钟脚）和 Data（数据脚）都是集电极开路的，所以必须接大阻值的上拉电阻，它们平时保持高电平，有输出时才被拉到低电平，之后自动上浮到高电平。

2. 电气特性

PS/2 通讯协议是一种双向同步串行通讯协议。通讯的两端通过 Clock（时钟脚）同步，并通过 Data（数据脚）交换数据。通讯的两端中任何一方如果想抑制另外一方通讯时，只需要把 Clock（时钟脚）拉到低电平。如果是 PC 机和 PS/2 键盘间的通讯，则 PC 机必须做主机，也就是说，PC 机可以抑制 PS/2 键盘发送数据，而 PS/2 键盘则不会抑制 PC 机发送数据。注：在本实验中，由 ISP 实验板来充当主机。

一般两设备间传输数据的最大时钟频率是 33kHz，大多数 PS/2 设备工作在 10~20kHz，推荐值在 15kHz 左右，也就是说，Clock（时钟脚）高、低电平的持续时间都为 40μs 左右（频率约为 12.5kHz）。每一数据帧包含 11~12 个位（注：设备发到主机时为 11 位，如本实验中所示；而主机发到设备时为 12 位），具体含义如图 9-9 所示。

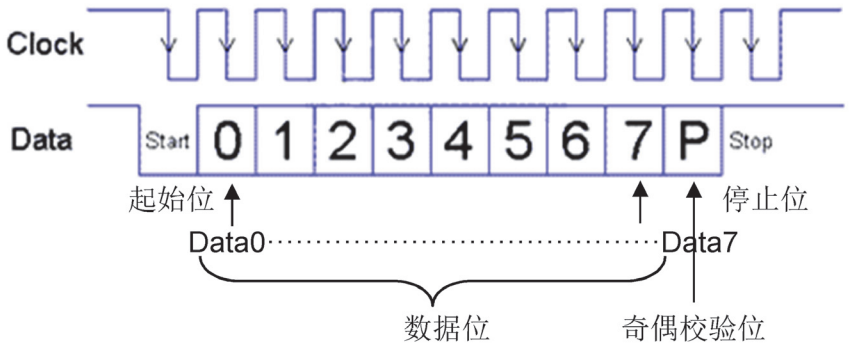


图 9-9 数据帧格式说明

3. PS/2 设备与 PC 机的通讯

PS/2 设备（如键盘或鼠标）的 Clock（时钟脚）和 Data（数据脚）都是集电极开路的，平时都是高电平。当 PS/2 设备等待发送数据时，它首先检查 Clock（时钟脚）以确认其是否为高电平，只有为高电平才可以开始发送数据。如果是低电平，则认为是 PC 机抑制了通讯，此时它必须缓冲需要发送的数据直到重新获得总线的控制权(一般 PS/2 键盘有 16 个字节的缓冲区，而 PS/2 鼠标只有 1 个缓冲区即仅存储最后一个要发送的数据)。如果 Clock（时钟脚）为高电平，PS/2 设备便开始将数据发送到 PC 机。一般都是由 PS/2 设备产生时钟信号。发送时一般都是按照数据帧格式顺序发送，其中数据位在 Clock（时钟脚）为高电平时准备好，在 Clock（时钟脚）的下降沿被 PC 机读入，如图 9-9 所示。

当时钟频率为 15kHz 时，从 Clock（时钟脚）的上升沿到数据位变化的时间至少要 5μs。而数据变化到 Clock（时钟脚）下降沿的时间至少也要 5μs，但不能大于 25μs，这是由 PS/2 通讯协议的时序规定的。如果时钟频率是其它值，参数的值应稍作调整。

上述讨论中传输的数据是指对特定键盘的编码或者对特定命令的编码。一般采用第二套扫描码集所规定的码值来编码，如表 9-1~表 9-4 所示。其中键盘码分为通码（Make）和断码（Break）。通码是按键接通时所发送的编码，用两位十六进制数来表示；断码通常是按键断开时所发送的编码，用四位十六进制数来表示。

表 9-1 101、102、104 键键盘（第二套扫描码集）

键 名	通 码	断 码	键 名	通 码	断 码	键 名	通 码	断 码
A	1C	F0,1C	9	46	F0,46	[	54	F0,54
B	32	F0,32	`	0E	F0,0E	INSERT	E0,70	E0,F0,70
C	21	F0,21	-	4E	F0,4E	HOME	E0,6C	E0,F0,6C
D	23	F0,23	=	55	F0,55	PG UP	E0,7D	E0,F0,7D
E	24	F0,24	\	5D	F0,5D	DELETE	E0,71	E0,F0,71
F	2B	F0,2B	BKSP	66	F0,66	END	E0,69	E0,F0,69
G	34	F0,34	SPACE	29	F0,29	PG DN	E0,7A	E0,F0,7A
H	33	F0,33	TAB	0D	F0,0D	U ARROW	E0,75	E0,F0,75
I	43	F0,43	CAPS	58	F0,58	L ARROW	E0,6B	E0,F0,6B
J	3B	F0,3B	L SHFT	12	F0,12	D ARROW	E0,72	E0,F0,72
K	42	F0,42	L CTRL	14	F0,14	R ARROW	E0,74	E0,F0,74
L	4B	F0,4B	L GUI	E0,1F	E0,F0,1F	NUM	77	F0,77
M	3A	F0,3A	L ALT	11	F0,11	KP /	E0,4A	E0,F0,4A
N	31	F0,31	R SHFT	59	F0,59	KP *	7C	F0,7C
O	44	F0,44	R CTRL	E0,14	E0,F0,14	KP -	7B	F0,7B
P	4D	F0,4D	R GUI	E0,27	E0,F0,27	KP +	79	F0,79
Q	15	F0,15	R ALT	E0,11	E0,F0,11	KP EN	E0,5A	E0,F0,5A
R	2D	F0,2D	APPS	E0,2F	E0,F0,2F	KP .	71	F0,71
S	1B	F0,1B	ENTER	5A	F0,5A	KP 0	70	F0,70
T	2C	F0,2C	ESC	76	F0,76	KP 1	69	F0,69
U	3C	F0,3C	F1	5	F0,05	KP 2	72	F0,72
V	2A	F0,2A	F2	6	F0,06	KP 3	7A	F0,7A

表 9-1（续） 101、102、104 键键盘（第二套扫描码集）

键 名	通 码	断 码	键 名	通 码	断 码	键 名	通 码	断 码
W	1D	F0,1D	F3	4	F0,04	KP 4	6B	F0,6B
X	22	F0,22	F4	0C	F0,0C	KP 5	73	F0,73
Y	35	F0,35	F5	3	F0,03	KP 6	74	F0,74
Z	1A	F0,1A	F6	0B	F0,0B	KP 7	6C	F0,6C
0	45	F0,45	F7	83	F0,83	KP 8	75	F0,75
1	16	F0,16	F8	0A	F0,0A	KP 9	7D	F0,7D
2	1E	F0,1E	F9	1	F0,01	]	5B	F0,5B
3	26	F0,26	F10	9	F0,09	;	4C	F0,4C
4	25	F0,25	F11	78	F0,78	'	52	F0,52
5	2E	F0,2E	F12	7	F0,07	,	41	F0,41
6	36	F0,36	PRNTSCRN	E0,12,E0,7C	E0,F0,7C,E0,F0,12	.	49	F0,49
						/	4A	F0,4A
7	3D	F0,3D	SCROLL	7E	F0,7E			
8	3E	F0,3E	PAUSE	E1,14,77,E1,F0,14,F0,77	-NONE-			

表 9-2 ACPI 功能键扫描码（第二套扫描码集）

键 名	通 码	断 码
Power	E0, 37	E0, F0, 37
Sleep	E0, 3F	E0, F0, 3F
Wake	E0, 5E	E0, F0, 5E

表 9-3 多媒体功能键扫描码（第二套扫描码集）

键 名	通 码	断 码
Next Track	E0, 4D	E0, F0, 4D
Previous Track	E0, 15	E0, F0, 15
Stop	E0, 3B	E0, F0, 3B
Play/Pause	E0, 34	E0, F0, 34
Mute	E0, 23	E0, F0, 23
Volume Up	E0, 32	E0, F0, 32
Volume Down	E0, 21	E0, F0, 21
Media Select	E0, 50	E0, F0, 50
E-Mail	E0, 48	E0, F0, 48
Calculator	E0, 2B	E0, F0, 2B
My Computer	E0, 40	E0, F0, 40
WWW Search	E0, 10	E0, F0, 10
WWW Home	E0, 3A	E0, F0, 3A
WWW Back	E0, 38	E0, F0, 38
WWW Forward	E0, 30	E0, F0, 30
WWW Stop	E0, 28	E0, F0, 28
WWW Refresh	E0, 20	E0, F0, 20
WWW Favorites	E0, 18	E0, F0, 18

表 9-4 常用的键盘命令字

命令字	说明
FF	使键盘复位。
FE	重新发送，当主机在接收中出现错误就可以用些命令，键盘接收后回应最后发送的扫描码或命令回应，但是 0xFE 绝不会作为键盘的回应而被发送。
F6	载入缺省的设置。
F5	键盘停止扫描，同时载入缺省值键。
F4	用上 0xF5 命令禁止键盘后重新使能键盘。
ED	主机在发送本命令后跟随发送一个参数字节用于指示键盘上Num Lock, Caps Lock,各 Scroll Lock LED 的状态。参数字节的 bit0 位为 Scroll Lock，bit1 位为 Num Lock，bit2 位 Caps Lock，这些位为 1 时相应的 LED 点亮，为 0 则熄灭。

- **部分参考源代码（Verilog 语言）：**

```
//KeyData 为 PS/2 的数据线，而 KeyClock 为 PS/2 的时钟线
//KeyData 即 KBDATA 对应 CPLD 的 4 脚，与实验板上拨码开关 3 复用，拨到数字一侧即不影响 PS/2 键盘
//KeyClock 即 KBCLK 对应 CPLD 的 3 脚，与实验板上拨码开关 4 复用，拨到数字一侧即不影响 PS/2 键盘

always @(posedge clk or negedge reset) //clk 为系统高频时钟（如 6MHz）信号，reset 为系统复位信号
begin
    if(!reset)
        begin
            KeyClock_r0 <= 1'b0;
            KeyClock_r1 <= 1'b0;
            KeyClock_r2 <= 1'b0;
        end
    else
        begin //锁存状态，进行滤波消抖
            KeyClock_r0 <= KeyClock; //KeyClock_r0 是最新的
            KeyClock_r1 <= KeyClock_r0;
            KeyClock_r2 <= KeyClock_r1; //KeyClock_r2 是最旧的
        end
end
assign neg_KeyClock = ~KeyClock_r1 & KeyClock_r2;
//neg_KeyClock 标志着 KeyClock 下降沿，前 1 后 0，或者说 r2 为 1 而 r1 为 0

//接收扫描码
always @(posedge clk or negedge reset)
begin
    if(reset==0)
        begin
            counter=0; //counter 为接收计数器
            RxEn<=1; //RxEn 为接收完成标志信号，下降沿有效；这里初始化为接收尚未完成
        end
    else if(neg_KeyClock)
        begin
            RxData[counter]<=KeyData;
            //RxData 用于临时存储接收到的 PS/2 数据
            //显然是先收低位，后收高位（因为发过来时是先发低位，后发高位的）
            if(counter>=10)RxEn<=0;
            else RxEn<=1;
            //接收完毕，共接收了 11 次
            //从 0 到 10，包括 1 个起始位，8 个数据位，1 个奇偶校验位，1 个停止位
            if(counter>=10)counter=0;
            else counter=counter+1;
        end
end
```

```
//避免一个按键产生两个数
always @(negedge RxEn or negedge reset)
begin
    if(reset==0)
        begin
            LastRxData<=8'b01000101;
            //LastRxData 用于记录上一次接收到的 PS/2 数据；初始化为 45h，即 0 的通码
            LedEn<=0; //LedEn 为 0 则允许数码管显示，为 1 则不允许显示
        end
    else if(RxEn==0) //接收完毕
        begin
            if(LastRxData==8'b11110000)LedEn<=1;
            //F0h 是断码标志，这个码就不再显示了（用 LedEn=1 来决定），以免按一次键产生两个相同的数
            else LedEn<=0; //允许数码管显示
            LastRxData<=RxData[8:1];
            //存储在 RxData 中新收到的 8 位数据送给 LastRxData，[0]是起始位就不送了
        end
end

//翻译扫描码
always @(LastRxData or reset or LedEn)
begin
    if(reset==0)
        PS_Data<=0;
    else if(LedEn==0) //当允许显示时
        begin
            case(LastRxData)
                //使用一个数码管来显示通码所对应的数字；这里先把通码 LastRxData 翻译成数字 PS_Data
                //save 用于记住刚才的按键值，以便在按其它非数字键时保持原值
                //注：直接用 save<=PS_Data 的话会有问题，因为是非阻塞式赋值
                8'b01000101: begin PS_Data<=4'b0000;save<=4'b0000; end //45h 为 0 的通码
                8'b00010110: begin PS_Data<=4'b0001;save<=4'b0001; end //16h 为 1 的通码
                8'b00011110: begin PS_Data<=4'b0010;save<=4'b0010; end //1Eh 为 2 的通码
                8'b00100110: begin PS_Data<=4'b0011;save<=4'b0011; end //26h 为 3 的通码
                8'b00100101: begin PS_Data<=4'b0100;save<=4'b0100; end //25h 为 4 的通码
                8'b00101110: begin PS_Data<=4'b0101;save<=4'b0101; end //2Eh 为 5 的通码
                8'b00110110: begin PS_Data<=4'b0110;save<=4'b0110; end //36h 为 6 的通码
                8'b00111101: begin PS_Data<=4'b0111;save<=4'b0111; end //3Dh 为 7 的通码
                8'b00111110: begin PS_Data<=4'b1000;save<=4'b1000; end //3Eh 为 8 的通码
                8'b01000110: begin PS_Data<=4'b1001;save<=4'b1001; end //46h 为 9 的通码
                //如果需要识别更多键盘按键可以在这里继续添加
                default: PS_Data<=save; //其它键包括断码标志 F0 均无效，保持原值
            endcase
        end
end
```

（请思考：上面这两个 always 块可否归并成一个 always 块？LedEn 信号可否去掉？）

实验十一 I^2C EEPROM 读写系统的设计

要求：利用基本 ISP 实验板（即 MAGIC3100 CPLD 实验板），以纯 Verilog 语言或纯 VHDL 语言方式输入设计，也可以电原理图 Schematic 和 HDL 语言混合方式输入设计，实现一个 I^2C EEPROM 读写的系统，按键选择实验板键盘阵列中的任意一个数字，将其写入实验板上的 EEPROM 24LC02 中，并显示在数码管上；掉电后重新上电，再从实验板上的 EEPROM 24LC02 中读取写入的数字，将其显示在数码管上，并可适当添加发挥功能。

重点：I^2C 总线双向传输协议的格式及其应用，以及数码管扫描显示、键盘扫描及按键消抖技术。

在学习这个实验时希望大家能多多在网上查找相关资料，看看别人的实现方法，比较一下各种实现的思路，这在学习和开发当中都是非常必要的。开发人员不是理论家，绝不能闭门造车，作为一个开发人员，你的使命是以最快的速度实现你的设计，能借鉴别人的地方一定要去用，要学会站在巨人的肩膀上；但同时别人的东西也绝对不是完美的，你必须读懂它，然后再去使用它、改进它。

- **功能现象：**

按动实验板键盘阵列中的任意一个数字键，随后再按动实验板上的独立按键 A，这时将刚才选择的数字写入实验板上的 EEPROM 24LC02 中，同时将该数字显示在数码管上。为了验证数据是否真正地写入了 EEPROM 里，这时关闭实验板电源，等待一会儿再开启电源，按动实验板上的独立按键 B，这时从实验板上的 EEPROM 24LC02 中读取数据，同时将该所读数据（应该即为刚才键入的数字）显示在数码管上，以表明数据的确存入了 EEPROM，而且确实是掉电保持的。也就是说独立按键 A 就是写命令，独立按键 B 就是读命令。

注意：首先请确认实验板上的拨码开关均拨向数字“1”、“2”、“3”、“4”一端，因为 EEPROM 与拨码开关共享了相同的 CPLD 引脚。只要将拨码开关拨向数字一端也就不会影响 EEPROM 了。

- **实现原理：**

1、I^2C 总线的基本概念

I^2C 总线（Inter-Integrated Circuit Bus，中文翻译成内置集成电路总线或内部整合电路总线），是 20 世纪 80 年代由 PHILIPS 公司为了让主板、嵌入式系统或手机能连接低速周边装置而开发的一种两线式串行通讯总线，使用多主从架构，主要用于连接微控制器及其外围设备，是微电子通信控制领域广泛采用的一种总线标准。它是同步通信的一种特殊形式，具有接口线少、控制方式简单、器件封装形式小、通信速率较高等优点。I^2C 的正确读法为“I-squared-C”，而“I-two-C”则是另一种错误但被广泛使用的读法，在中国则多以“I 方 C”称之。其基本概念包括：

- ✓ 发送器（Transmitter）：发送数据到总线的器件
- ✓ 接收器（Receiver）：从总线接收数据的器件
- ✓ 主机（Master）：初始化发送、产生时钟信号和终止发送的器件（无论读写均为主机终止）
- ✓ 从机（Slave）：被主机寻址的器件

I^2C 总线是双向传输的总线，因此主机和从机都可能成为发送器和接收器。如果主机向从机发送数据，则主机是发送器，而从机是接收器；如果主机从从机读取数据，则主机是接收器，而从机是发送器。不论主机是发送器还是接收器，时钟信号 SCL 都要由主机来产生。

图 9-10 是 I^2C 总线信号连接的示意图。其中，Master 为主机，Slave1 和 Slave2 为从机。SDA 和 SCL 是 I^2C 总线的两根信号线，一根是串行数据线 SDA，另一根是串行时钟线 SCL，通常这两个管脚都是漏极开路（或集电极开路）输出结构，因此在实际使用时，SDA 和 SCL 信号线都必须要加上拉电阻 Rp（Pull-Up Resistor）接到电源 Vdd（如图 9-10 中所示），上拉电阻一般取值 3~10kΩ。开漏结构的好处是：(1)、当总线空闲时，这两条信号线都保持高电平，不会消耗电流；(2)、电气兼容性好。上拉电阻接 5V 电源就能与 5V 逻辑器件进行接口相连，上拉电阻接 3.3V 电源又能与 3.3V 逻辑器件进行接口相连。因为是开漏结构，所以不同器件的 SDA 与 SDA 之间、SCL 与 SCL 之间可以直接相连，不需要额外的转换电路。

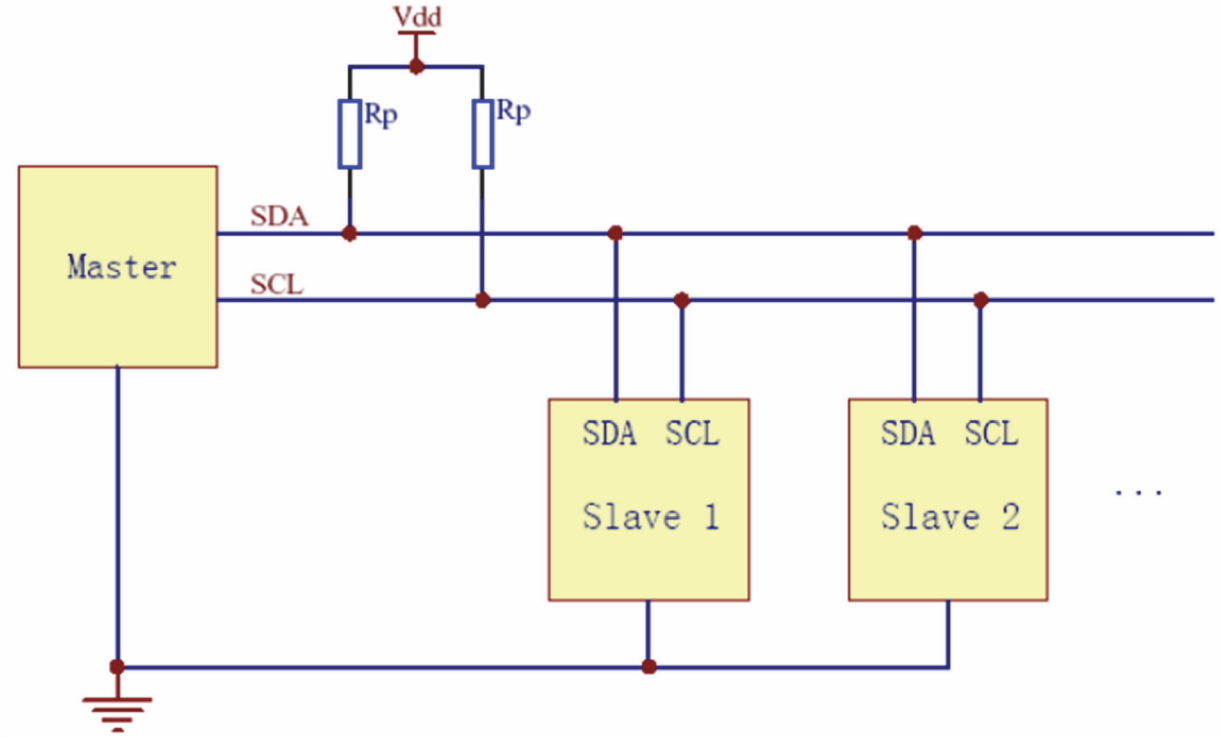


图 9-10 I²C 总线信号连接示意图

2、EEPROM 器件：24LC02

EEPROM 器件通常都是兼容的，24LC02、24LC04 一直到 24LC256 管脚都是兼容的，其管脚图如图 9-11 所示。其中，A0、A1、A2 是从机地址，因为总线上如果有多个 EEPROM 器件，这时就要通过地址引脚加以区分；Vcc 是正电源，根据型号不同，电压范围也不同，比如 24LC02 电压范围为 2.5V~5.5V，24AA02 电压范围为 1.8V~5.5V；WP 为写保护使能引脚（Write Protect），当其为高电平时，主机就只能读而不能写了（即处于写保护状态）。

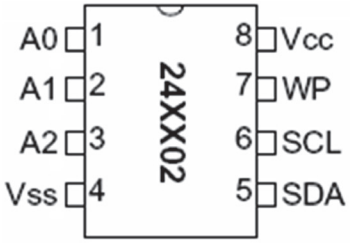


图 9-11 EEPROM 器件的管脚图

3、I²C 总线的通信速率

I²C 总线的通信速率受主机控制，能快能慢。但是最高速率是有限制的，I²C 总线上数据的传输速率在标准模式下为 100kbps，在快速模式下为 400kbps。按照后来修订的版本，位速率最高可达 3.4Mbps。

4、I²C 总线上数据的有效性（Data validity）

I²C 总线上数据有效性的示意图如图 9-12 所示。数据线 SDA 的电平状态必须在时钟线 SCL 处于高电平期间保持稳定不变。SDA 的电平状态只有在 SCL 处于低电平期间才允许改变，但在 I²C 总线的起始和结束时例外。注：某些其它的串行总线协议，如 SPI（Serial Peripheral Interface，串行外设接口），可能规定数据在时钟信号的边沿（上升沿或下降沿）有效，而 I²C 总线则是电平有效。

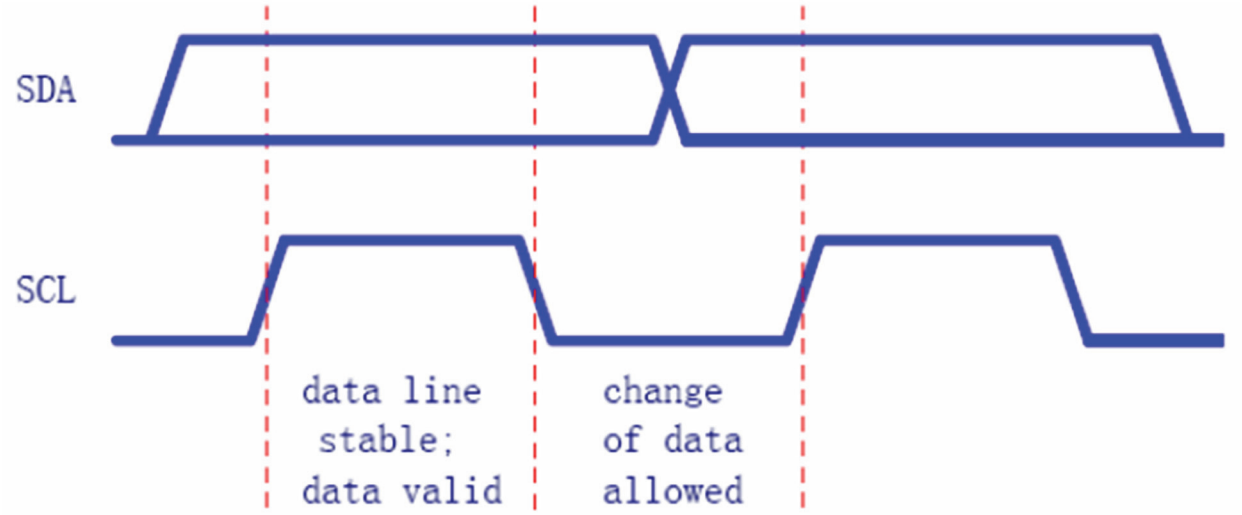


图 9-12 I²C 总线上数据有效性的示意图

5、起始条件和停止条件（START and STOP conditions）

I²C 总线数据传输的起始条件和停止条件示意图如图 9-13 所示。

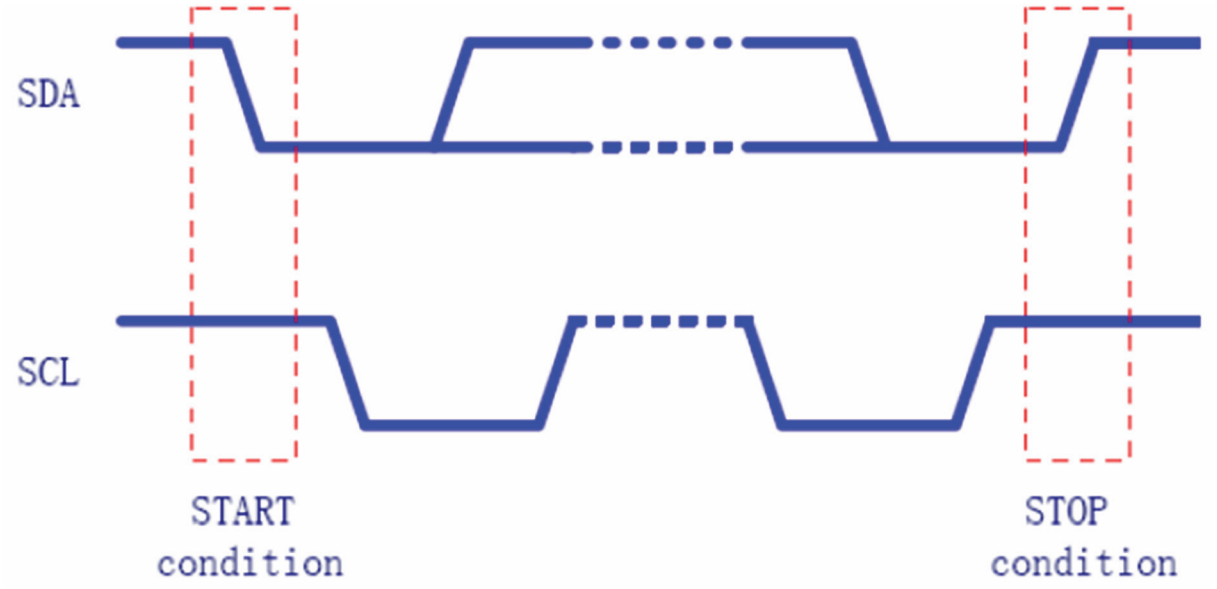


图 9-13 I²C 起始条件和停止条件示意图

- ✓ 起始条件：当 SCL 处于高电平期间，SDA 从高电平向低电平跳变时产生起始条件。总线在起始条件产生后便处于忙的状态。起始条件常简记为 S（即 START）。
- ✓ 停止条件：当 SCL 处于高电平期间时，SDA 从低电平向高电平跳变时产生停止条件。总线在停止条件产生后处于空闲状态。停止条件常简记为 P（即 STOP）。

6、从机地址（Slave Address）

I²C 总线不需要额外的地址译码器和片选信号。多个具有 I²C 总线接口的器件都可连接到同一条 I²C 总线上，它们之间通过器件地址来区分。主机是主控器件，它不需要器件地址；其它器件都属于从机，需有器件地址。必须保证同一条 I²C 总线上所有从机的地址都是唯一确定的，不能有重复，否则 I²C 总线将不能正常工作。一般从机地址由 7 位地址位和 1 位读写标志 R/W 组成，7 位地址占据高 7 位，读写位在最后，如图 9-14 所示。读写位是 0，表示主机将要向从机写入数据；读写位是 1，则表示主机将要从从机读取数据。例如一种 EEPROM 器件的控制字节（CONTROL BYTE）分配格式如图 9-14 所示，其中，B1~B0 就是从机地址管脚上的地址信号，1010 是器件类型的分类编号。

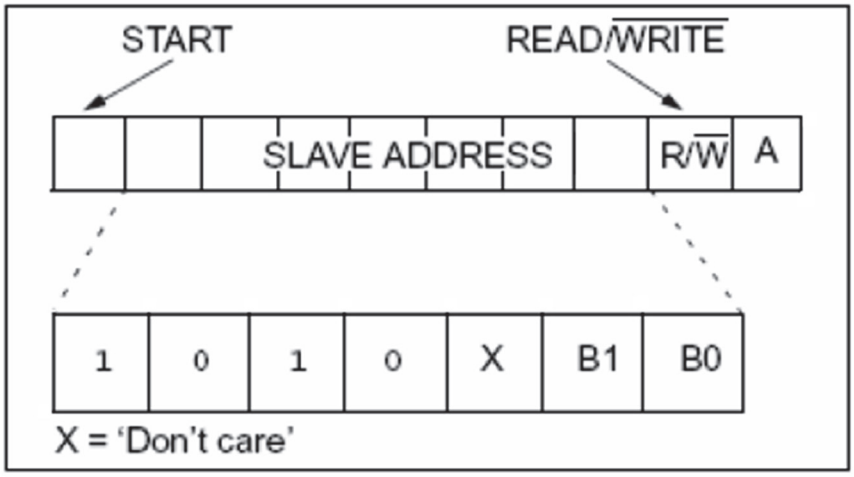


图 9-14 EEPROM 器件的控制字节（CONTROL BYTE）分配格式示意图

7、数据是按字节传输的

I^2C 总线总是以字节(Byte)为单位收发数据。每次传输的字节数量没有严格限制。首先传输的是数据的最高位MSB，第 7 位)，最后传输的是最低位（LSB，第 0 位)。另外，每个字节之后还要跟一个响应位，称为应答。

8、应答（Acknowledge）

在 I^2C 总线传输数据过程中，每传输一个字节，都要在其后跟上一个应答状态位。接收器接收数据的情况可以通过应答位来告知发送器。应答位的时钟脉冲仍由主机产生，而应答位的数据状态则遵循“谁接收谁产生”的原则，即总是由接收器产生应答位，也就是说，主机向从机发送数据时，应答位由从机产生；主机从从机接收数据时，应答位由主机产生。I^2C 总线标准规定：应答位为 0，表示接收器应答（ACK），常简记为 A；应答位为 1，则表示非应答（NACK），常简记为 $\overline{A}$。发送器发送完 LSB 之后，应当释放 SDA 线（拉高 SDA，输出晶体管截止），以等待接收器产生应答位。如果在接收器接收完最后一个字节的数据，或者不能再接收更多的数据时，应当产生非应答来通知发送器。发送器如果发现接收器产生了非应答状态，则应当终止发送。

9、基本的数据传输格式

基本的数据传输格式如图 9-15、图 9-16 所示。

图 9-15 主机向从机发送数据的基本格式

图 9-16 主机从从机接收数据的基本格式

其中，

S：起始位（START）

SA：从机地址（Slave Address），7 位从机地址

W：写标志位（Write），1 位写标志

R：读标志位（Read），1 位读标志

A：应答位（Acknowledge），1 位应答

$\overline{A}$：非应答位（Not Acknowledge），1 位非应答

D：数据（Data），每个数据都必须是 8 位

P：停止位（STOP）

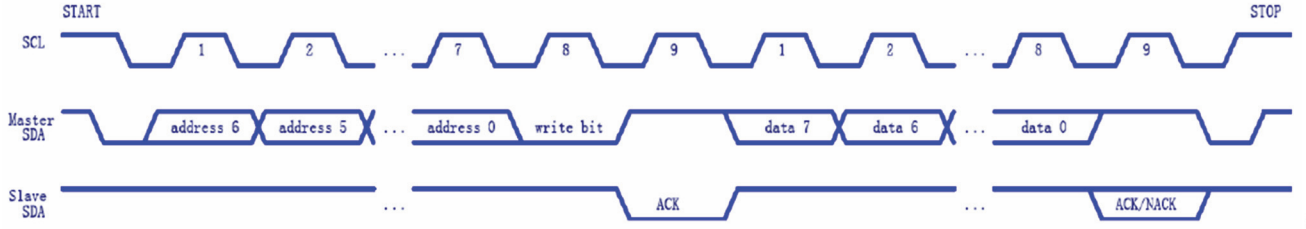
阴影：主机产生的信号

无阴影：从机产生的信号

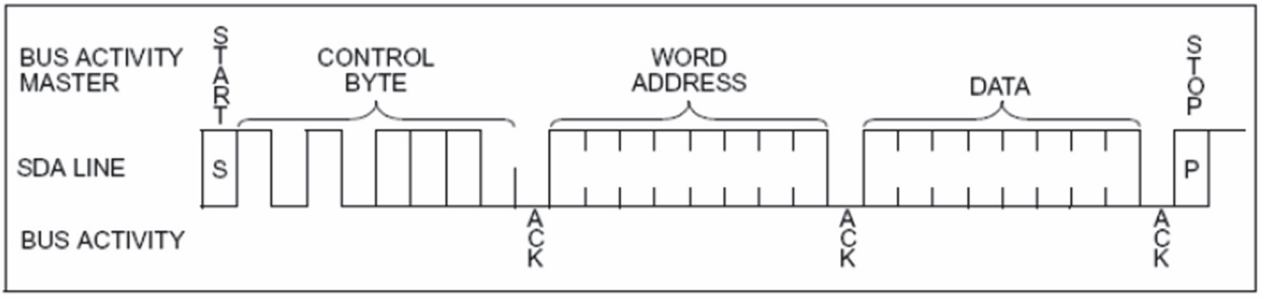
应当注意的是，与图 9-16 中的情况不同的是，在图 9-15 中，主机向从机发送最后一个字节的数据时，从机可能应答也可能非应答，但不管怎样主机都可以产生停止条件。如果主机在向从机发送数据（甚至包括从机地址在内）时检测到从机非应答，则应当及时停止传输。

10、传输一个字节数据的时序图

为了更清楚地了解 I^2C 总线的基本数据传输过程，下面画出了只传输一个字节数据的时序图，如图 9-17、图 9-18 所示（其中的(a)和(b)图实际上是一致的），这是最基本的传输方式。**其中，CONTROL BYTE 即为包含从机地址的控制字节，WORD ADDRESS 即为 EEPROM 中的数据存储地址**。在图 9-17 和图 9-18 中的(a)图里，SDA 信号线被画成了两条，一个是主机产生的（Master SDA），另一个是从机产生的（Slave SDA）。实际上主机和从机的 SDA 信号线总是连接在一起的，是同一根 SDA。在这里画成两个 SDA 主要是有助于深刻理解在 I^2C 总线上主机和从机的不同行为。**另外注意：按照如图 9-18 所示的 I^2C 协议要求，CONTROL BYTE 需要先后写两次。**

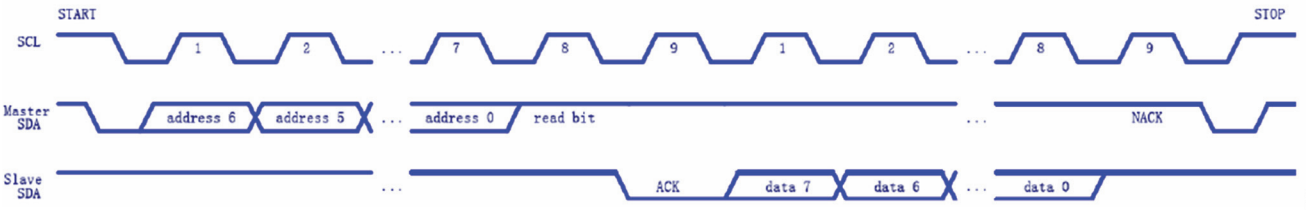


(a)

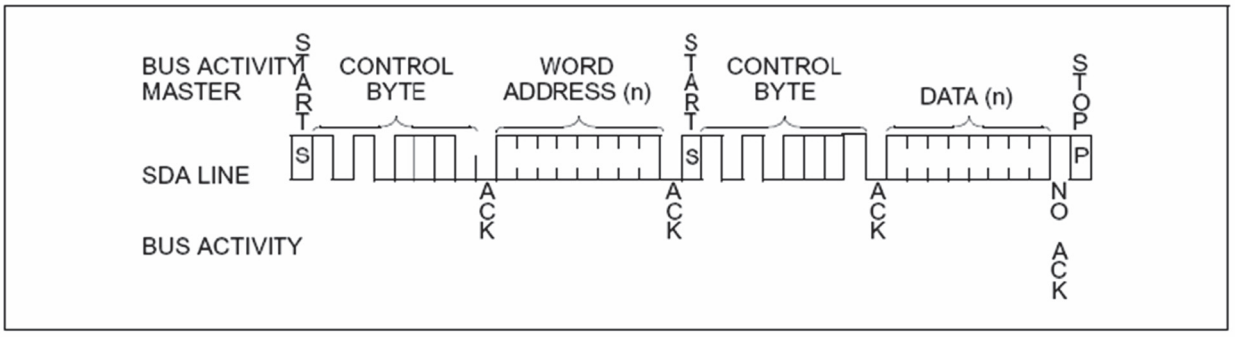


(b)

图 9-17 主机向从机发送 1 个字节数据（即 BYTE WRITE）的时序图



(a)

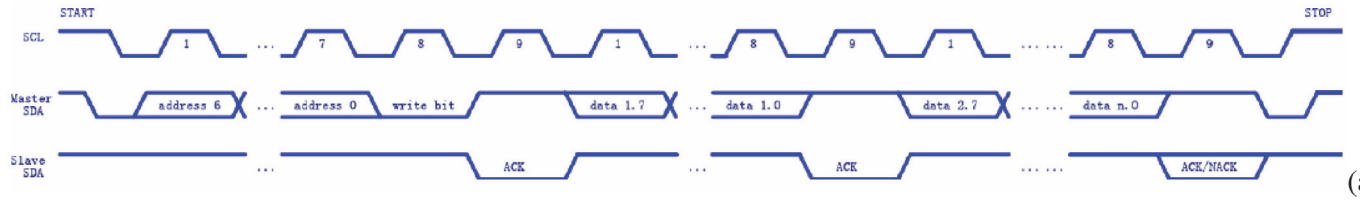


(b)

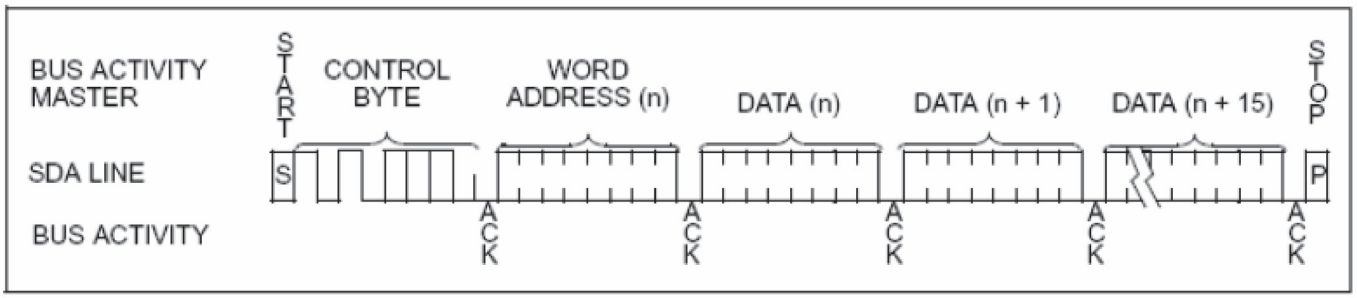
图 9-18 主机从从机接收 1 个字节数据（即 RANDOM READ）的时序图

11、传输多个字节数据的时序图

主机连续向从机发送或从从机接收多个字节数据的情况也很容易理解，其时序图如图 9-19、图 9-20 所示（其中的(a)和(b)图实际上是一致的）。**其中，CONTROL BYTE 即为包含从机地址的控制字节，WORD ADDRESS 即为 EEPROM 中的数据存储地址**。

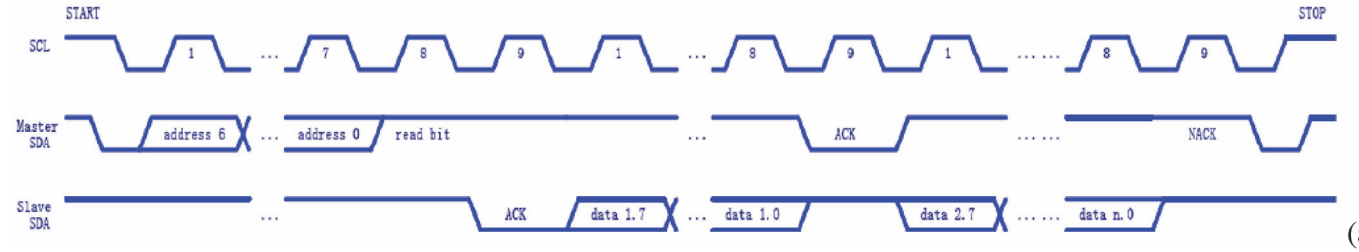


(a)

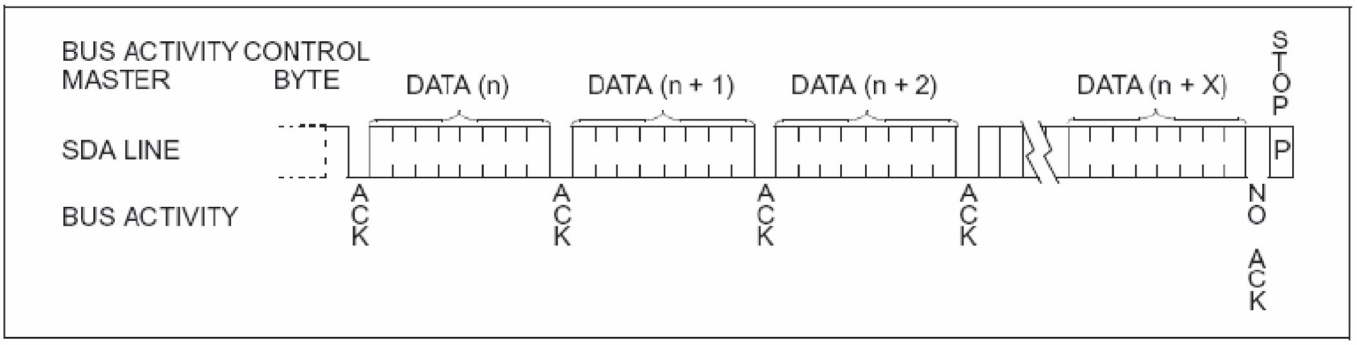


(b)

图 9-19 主机向从机连续发送多个字节数据（即 PAGE WRITE）的时序图



(a)



(b)

图 9-20 主机从从机连续接收多个字节数据（即 SEQUENTIAL READ）的时序

- 部分参考源代码（**Verilog** 语言）：

```
//SDA 为 I2C 数据总线，对应 CPLD 的 5 脚，与实验板上拨码开关 2 复用，拨到数字一侧即不影响 I2C
//SCL 为 I2C 时钟线，对应 CPLD 的 6 脚，与实验板上拨码开关 1 复用，拨到数字一侧即不影响 I2C

//CMD_RD 和 CMD_WR 为按键命令
//CMR_WR 为写，对应独立按键 A；CMR_RD 为读，对应独立按键 B
```

```
//phase0, phase1, phase2, phase3 为四个阶段的状态变量
//phase0 对应 SCL 的上升沿时刻，phase2 对应 SCL 的下降沿时刻
//phase1 对应从 SCL 高电平的中间时刻，phase3 对应从 SCL 低电平的中间时刻
//phase1 为数据的有效时段，此时数据线不允许变化
//phase3 为数据的无效时段，允许更改数据

//参数设置，相当于 C 语言中的宏定义
parameter i2c_addr=7'b1010000,eeprom_addr=8'b00000110;
//i2c_addr 为 EEPROM 器件从机地址
//eeprom_addr 为 EEPROM 存储空间地址，在这里固定存储地址为 00000110

assign SCL=scl_buf; //scl_buf 为 I2C 时钟线缓冲
assign SDA=(Flag_RW)?sda_buf:1'bz; //sda_buf 为 I2C 数据线缓冲
//Flag_RW 为 1 则将 SDA 设置为写状态，为 0 则将 SDA 设置为读状态

//产生采样的 4 个阶段
always @(posedge CLK or negedge RST) //CLK 为 6MHz 时钟源信号
begin
    if(RST==0)
    begin
        phase0 <= 0;
        phase1 <= 0;
        phase2 <= 0;
        phase3 <= 0;
        clk_div<= 0;
    end
    else
    begin
        if(clk_div!=4'b1111) clk_div <= clk_div + 4'b0001;
        else clk_div <= 0;
        //phase0：上升沿
        if(phase0==1) phase0 <= 0;
        else if(clk_div==4'b1111) phase0 <= 1;
        //phase1：‘1’的中间时刻
        if(phase1==1) phase1 <= 0;
        else if(clk_div==4'b0011) phase1 <= 1;
        //phase2：下降沿
        if(phase2==1) phase2 <= 0;
        else if(clk_div==4'b0111) phase2 <= 1;
        //phase3：‘0’的中间时刻
        if(phase3==1) phase3 <= 0;
        else if(clk_div==4'b1011) phase3 <= 1;
    end
end
//总体效果是：4 拍为一个 phase，所以 16 拍经历四个 phase
//0 时是 phase0=1，4 时是 phase1=1，8 时是 phase2=1，12 时是 phase3=1
```

```
//用于按键消抖的延时进程
always @(posedge CLK or negedge RST)
begin
    if(RST==0) key_delay<=0; //key_delay 为按键消抖计数器
    else if(start_delay==1) key_delay<=key_delay+1; // start_delay 为按键消抖计数器计数开始信号，1 有效
    else key_delay<=0;
end

//I2C 进程
always @(posedge CLK or negedge RST)
begin
    if(RST==0)
    begin
        start_delay<=0;
        eeprom_state<=0; //eeprom_state 为操作状态，0 无操作，1 写操作状态，2 读操作状态
    end
    else
        begin
            case(eeprom_state)
                0: //无操作命令状态
                    begin
                        WrData<={4'b0000,DATA_IN};//合并成一个 8 位数
                        //DATA_IN 为输入的 4 位数据；WrData 为待写入 EEPROM 的 8 位数据
                        RdData<=0; //RdData 用于存放从 EEPROM 读出的 8 位数据
                        bit_state<=0; //bit_state 为发送数据或接收数据时的位状态
                        Flag_RW<=0; //读

                        //按键防抖动
                        if((key_delay==0)&&((CMD_WR==0)||(CMD_RD==0)))start_delay<=1;
                        else if(key_delay==10'b1000000000)
                            begin
                                if(CMD_WR==0)eeprom_state<=1; //写操作状态
                                else if(CMD_RD==0)eeprom_state<=2; //读操作状态
                                else eeprom_state<=0; //无操作命令状态
                                start_delay<=0; //停止按键消抖计数器工作
                            end
                    end

                1: //写 EEPROM 操作（其时序参见图 9-17 中的(b)图）
                    begin
                        if(phase0==1)scl_buf<=1;
                        else if(phase2==1)scl_buf<=0;
                        //1 到 0 跳变，产生开始 START 信号
```

```
case(bit_state)
    0: //帧开始，从机地址的第 0 位
      //i2c_addr 从机地址是从高位开始的，这与存储地址和数据发送形式相反
      //具体可参考 24LC02 数据手册
        begin
            if(phase1==1)begin sda_buf<=0;Flag_RW<=1;end
            if((phase3&Flag_RW)==1)
                begin sda_buf<=i2c_addr[6];Flag_RW<=1;bit_state<=1;end
        end
    1: //从机地址的第 1 位
        if(phase3==1)begin sda_buf<=i2c_addr[5];Flag_RW<=1;bit_state<=2;end
    //略
    6: //从机地址的第 6 位
        if(phase3==1)begin sda_buf<=i2c_addr[0];Flag_RW<=1;bit_state<=7;end
    7: //表示写从机（控制字节 CONTROL BYTE 中的 RW’为 0）
        if(phase3==1)begin sda_buf<=0;Flag_RW<=1;bit_state<=8;end
    8: //准备读应答 ACK
        if(phase3==1)begin Flag_RW<=0;bit_state<=9;end
    9: //读应答 ACK
        begin
            if(phase0==1)sda_tem<=SDA; //读 ACK
            //sda_tem 为 I2C 数据线缓冲，临时存储读 SDA 数据

            if((phase1==1)&&(sda_tem==1))eeprom_state<=0;
            //判断 ACK 信号是否有效，为 1 表示无效，返回初始状态

            if(phase3==1)begin sda_buf<=eeprom_addr[7];Flag_RW<=1;bit_state<=10;end
            //发送数据存储地址 WORD ADDRESS，写地址 bit0
        end
    10: //写地址 bit1
        if(phase3==1)begin sda_buf<=eeprom_addr[6];Flag_RW<=1;bit_state<=11;end
    //略
    16: //写地址 bit7
        if(phase3==1)begin sda_buf<=eeprom_addr[0];Flag_RW<=1;bit_state<=17;end
    17: //准备读 ACK
        if(phase3==1)begin Flag_RW<=0;bit_state<=18;end
    18: //读 ACK，然后开始写数据
        begin
            if(phase0==1)begin sda_tem<=SDA;end
            if((phase1==1)&&(sda_tem==1))eeprom_state<=0;
            //开始写数据[7]
            if(phase3==1)begin sda_buf<=WrData[7];Flag_RW<=1;bit_state<=19;end
        end
    19: //写入数据[6]
        if(phase3==1)begin sda_buf<=WrData[6];Flag_RW<=1;bit_state<=20;end
    //略
```

```
                25: //写入数据[0]
                    if(phase3==1)begin sda_buf<=WrData[0];Flag_RW<=1;bit_state<=26;end
                26: //准备读 ACK
                    if(phase3==1)begin Flag_RW<=0;bit_state<=27;end
                27: //读 ACK
                    begin
                        if(phase0==1)sda_tem<=SDA;
                        if((phase1==1)&&(sda_tem==1))eeprom_state<=0;
                        //下面 SDA 产生一个由低到高的跳变，结束一帧
                        if(phase3==1)begin sda_buf<=0;Flag_RW<=1;bit_state<=28;end
                    end
                28:
                    begin
                        if(phase1==1)sda_buf<=1; //SDA 从 0 到 1 的跳变，发送停止位 STOP
                        if(phase3==1)
                            begin bit_state<=0;eeprom_state<=0;DataLED<=DATA_IN;end
                            //DataLED 为送数码管显示的数
                    end
                default: begin bit_state<=0;eeprom_state<=0;end
            endcase
        end

2: //读 EEPROM 操作（其时序参见图 9-18 中的(b)图）
        begin
            if(phase0==1)scl_buf<=1;
            else if(phase2==1)scl_buf<=0;
            //1 到 0 跳变，产生开始 START 信号
            case(bit_state)
                0: //写 EEPROM 从机地址的第 0 位，因为 i2c_addr 从机地址是从高位开始的
                    begin
                        if(phase1==1)begin sda_buf<=0;Flag_RW<=1; end
                        if((phase3&Flag_RW)==1)
                            begin sda_buf<=i2c_addr[6];Flag_RW<=1;bit_state<=1;end
                    end
                1: //从机地址的第 1 位
                    if(phase3==1)begin sda_buf<=i2c_addr[5];Flag_RW<=1;bit_state<=2;end
                //略
                6: //从机地址的第 6 位
                    if(phase3==1)begin sda_buf<=i2c_addr[0];Flag_RW<=1;bit_state<=7;end
                7: //表示写从机（控制字节 CONTROL BYTE 中的 RW’为 0）
                    if(phase3==1)begin sda_buf<=0;Flag_RW<=1;bit_state<=8;end
                8: //准备读 ACK
                    if(phase3==1)begin Flag_RW<=0;bit_state<=9;end
```

```
9: //读 ACK
    begin
        if(phase0==1)sda_tem<=SDA;
        if((phase1==1)&&(sda_tem==1))eeprom_state<=0;
        if(phase3==1)begin sda_buf<=eeprom_addr[7];Flag_RW<=1;bit_state<=10;end
    end
10: //写地址 bit1
    if(phase3==1)begin sda_buf<=eeprom_addr[6];Flag_RW<=1;bit_state<=11;end
//略
16: //写地址 bit7
    if(phase3==1)begin sda_buf<=eeprom_addr[0];Flag_RW<=1;bit_state<=17;end
17: //准备读 ACK
    if(phase3==1)begin Flag_RW<=0;bit_state<=18;end
18: //读 ACK
    begin
        if(phase0==1)begin sda_tem<=SDA;end
        if((phase1==1)&&(sda_tem==1))begin eeprom_state<=0;end
        if(phase3==1)begin sda_buf<=1;Flag_RW<=1;bit_state<=19;end
    end
//再次发送从机地址，即图 9-18(b)图中的第二个 CONTROL BYTE
19: //从机地址的第 0 位
    begin
        if(phase1==1)begin sda_buf<=0;Flag_RW<=1; end
        if((phase3&Flag_RW)==1)
            begin sda_buf<=i2c_addr[6]; Flag_RW<=1;bit_state<=20;end
    end
20: //从机地址的第 1 位
    if(phase3==1)begin sda_buf<=i2c_addr[5];Flag_RW<=1;bit_state<=21;end
//略
25: //从机地址的第 6 位
    if(phase3==1)begin sda_buf<=i2c_addr[0];Flag_RW<=1;bit_state<=26;end
26: //表示读从机（控制字节 CONTROL BYTE 中的 RW’为 1）
    if(phase3==1)begin sda_buf<=1;Flag_RW<=1;bit_state<=27;end
27: //准备读 ACK
    if(phase3==1)begin Flag_RW<=0;bit_state<=28;end
28: //读 ACK
    begin
        if(phase0==1)sda_tem<=SDA;
        if((phase1==1)&&(sda_tem==1))eeprom_state<=0;
        if(phase3==1)begin Flag_RW<=0;bit_state<=29;end//准备读数据
    end
```

```
                    //开始读数据
                    29:
                        begin
                            if(phase1==1)RdData[7]<=SDA;
                            if(phase3==1)bit_state<=30;
                        end
                    30:
                        begin
                            if(phase1==1)RdData[6]<=SDA;
                            if(phase3==1)bit_state<=31;
                        end
                    //略
                    36:
                        begin
                            if(phase1==1)RdData[0]<=SDA;
                            if(phase3==1)bit_state<=37;
                        end
                    37: //进行 ACK 应答
                        if(phase3==1)begin sda_buf<=0;Flag_RW<=1;bit_state<=38;end
                    38:
                        begin
                            if(phase1==1)sda_buf <= 1; //SDA 从 0 到 1 的跳变，发送停止位 STOP
                            if(phase3==1)
                                begin bit_state<=0;eeprom_state<=0;DataLED<=RdData[3:0];end
                        end
                    default: begin bit_state<=0;eeprom_state<=0;end
                endcase
            end
        default: eeprom_state<=0;
    endcase
end
end
```

实验十二 LED 点阵显示系统的设计

要求：利用 EDA-E LC4256V CPLD 开发板，以纯 Verilog 语言或纯 VHDL 语言方式输入设计，也可以电原理图 Schematic 和 HDL 语言混合方式输入设计，实现一个 LED 点阵显示的系统，在开发板上的 8*8 LED 点阵屏上显示十六进制数字 0~F，并可适当添加发挥功能。

重点：LED 点阵屏显示的原理及其应用，其中包括点阵屏的动态扫描原理和任意自定义字符软件的应用。

- **功能现象：**

打开开发板电源，在开发板上的 8*8 LED 点阵屏上依次显示数字 0~F。

- **实现原理：**

1. 8*8 点阵屏介绍

8*8 点阵屏由 8 行 8 列共 64 个 LED 点阵组成，其外观及引脚图如图 9-21 所示。

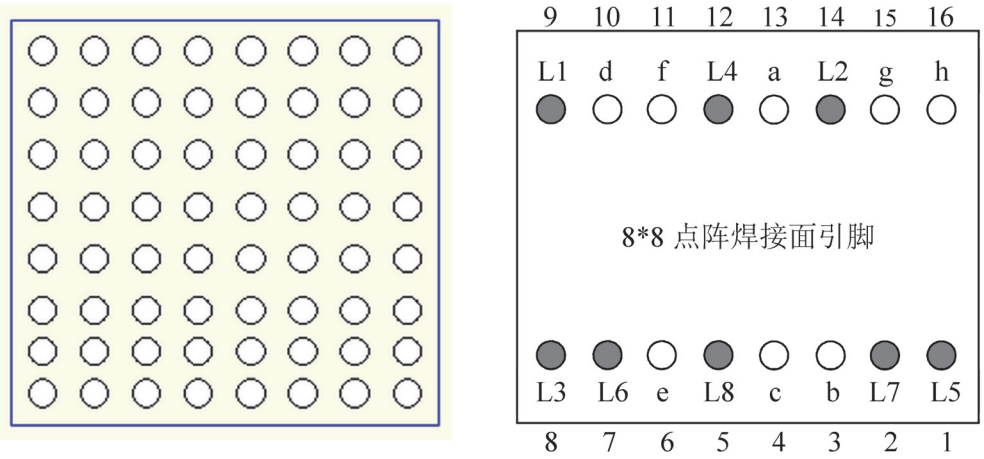


图 9-21 8*8 点阵屏的外观及引脚图

8*8 点阵屏的等效电路图如图 9-22 所示。

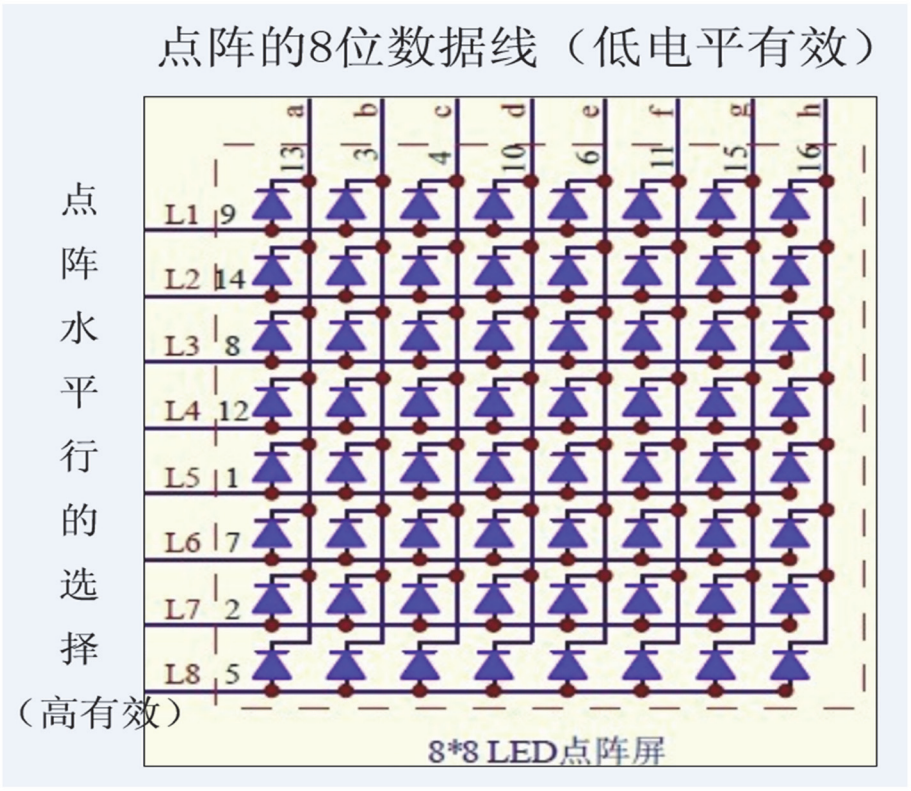


图 9-22 8*8 点阵屏的等效电路图

为了便于理解，可以定义：

- 点阵的 8 列即 8 位数据线信号为：a、b、c、d、e、f、g、h（低电平有效）。
- 点阵的 8 行即 8 位行选择线信号为：L1、L2、L3、L4、L5、L6、L7、L8（高电平有效）。

当点阵的数据线与点阵的行选择线同时有效时，二者交叉处对应的 LED 被点亮。

8*8 点阵屏硬件电路图如图 9-23 所示。

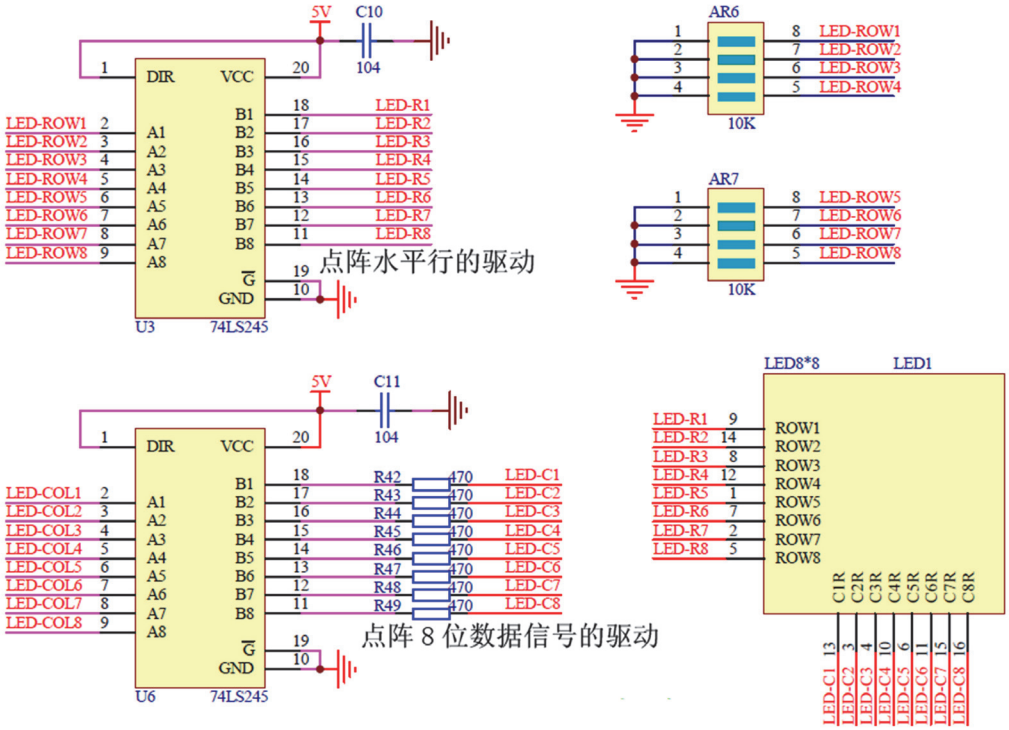


图 9-23 8*8 点阵屏的硬件电路图

其中，用两片 CMOS 型三态缓冲门 74HC245（图中的 U6、U3）分别驱动点阵的数据线和点阵的行选择线。为了防止干扰，在 U3 的前端加上 10kΩ的下拉电阻（图中的排阻 AR6 和 AR7）。74HC245 输出的高电平达到 5V，要加 470Ω的限流电阻。这里有三个问题值得一提。第一个问题是为什么要加下拉电阻？加下拉电阻的作用是防止外界干扰，使得高低电平清晰稳定，即使当输入行信号如 LED-ROW1 为高阻态 Z（相当于断开）时也可保证输入送给 74HC245 的是下拉的稳定低电平；而数据信号即列信号的 74HC245 处下拉电阻可加可不加，本来 LED 一般用三极管驱动，这里用 74HC245 本身就有驱动作用，且列信号要么 1 要么 0 较为稳定。第二个问题是为什么要加限流电阻？这是因为 5V 电压加在 LED 上导通时电流过大，会造成 LED 吃不消，所以需要加适当大小的限流电阻来减小导通时的电流。第三个问题是数据信号即列信号端加了限流电阻，那为什么行信号端没有加限流电阻？行端和列端在 LED 导通时其实是连在一条电路上的，所以行和列均可加限流电阻，但因为行是公共端，它要拖这一行上的 8 列，假如当 8 个灯全亮时电流负载大，需要吃得住 8 个灯去分这一电流，这时如果行上加限流电阻就无法支撑 8 个灯同时亮起来了；而且在行前端加限流电阻的话，会造成该行 1 个灯亮跟 2 个灯亮时的亮度有差异，明显不好看，所以限流电阻最好是加在列后端，这样每个灯如果亮的话其亮度将是一致的。

2. 点阵屏的动态扫描显示原理

LED 阵列的显示方式是按显示编码的顺序，一行一行地显示。每一行的显示时间大约为 1ms 左右，由于人眼的视觉暂留效应，将感觉到 8 行 LED 是在同时显示的。若显示的时间太短，则亮度不够；若显示的时间太长，将会感觉到闪烁。本实验采用高电平逐行扫描（指点阵的行选择线），低电平输出显示信号（指点阵的数据线），即轮流给行信号输出高电平，在任意时刻只有一行发光二极管是处于可以被点亮的状态 ，而其它行都处于熄灭状态。

3. 利用小软件得到要显示的任意自定义字符

通过 fang.exe 这个小软件（如图 9-24 所示；***注：此软件可从实验 FTP 的“实验相关软件”目录下载，也可在实验室电脑桌面上“电子系统设计实验”文件夹中的“ISP”文件夹中找到***），用对号勾选的方式，可以得到要显示的任意自定义字符或图案的每行的编码。有对号的点表示点亮，输出数据相应的位为 1；无对号的点表示熄灭，输出数据相应的位为 0。勾选后再点击该软件的“点到值”按钮即可得到相应编码。如图 9-24 所示得到的是字符“0”的编码。

图 9-24 得到任意自定义字符编码的小软件

据此，相应的 Verilog 语言源代码中字符“0”对应的部分如下所示：

```
case(scan_data)
    4'd0:
        begin
            col1<=8'h0e;  //第一行有效时的 8 位数据编码
            col2<=8'h11;  //第二行有效时的 8 位数据编码
            col3<=8'h13;  //第三行有效时的 8 位数据编码
            col4<=8'h15;  //第四行有效时的 8 位数据编码
            col5<=8'h19;  //第五行有效时的 8 位数据编码
            col6<=8'h11;  //第六行有效时的 8 位数据编码
            col7<=8'h0e;  //第七行有效时的 8 位数据编码
            col8<=8'h00;  //第八行有效时的 8 位数据编码
            //“0”字符的编码
            //从第一行到第八行一直循环显示（速度足够快），在点阵屏上就能显示字符“0”
        end
```

- **部分参考源代码（Verilog 语言）：**

```
//显示 16 进制数字 0~F
always@(posedge clk_50m or negedge reset_n) //clk_50m 为 50MHz 时钟源
    begin
        if(!reset_n) //异步复位
            begin
                cnt_scan<=0; //行扫描计数器清零
                row_buf<=8'b10000000; //点阵的行选择线的初始状态（有效第八行，L8 最高位，L1 最低位）
            end
        else if(cnt_scan!=16'hffff) //计数器时间约为 1.3ms（一行显示停留的时间）
            cnt_scan<=cnt_scan+1; //行扫描计数器加 1 计数
        else //当 cnt_scan 计满 16'hffff 时
            begin
                row_buf[7:1]<=row_buf[6:0];
```

```
                row_buf[0]<=row_buf[7]; //点阵的行选择线信号一次循环左移 1 位（逐个导通每一行）
                cnt_scan<=0; //行扫描计数器重新计数
            end
    end

always@(posedge clk_50m or negedge reset_n)
    begin
        if(!reset_n) //异步复位
            begin
                cnt_next<=0; //显示下一帧的时间计数器（一帧 8*8 点阵显示停留的时间）清零
                scan_data<=0; //表示当前帧显示字符编码（从“0”开始）
            end
        else if(cnt_next!=39999999) //一帧显示停留的时间为 0.8s（在这一时间内 scan_data 不变）
                cnt_next<=cnt_next+1;
        else //当 cnt_next 计满 39999999 时
            begin
                cnt_next<=0;
                scan_data<=scan_data+1'b1; //显示下一个字符（从“0”一直显示到“F”）
            end
    end

//为要显示的字符进行点阵数据编码
always@ (scan_data)
    begin
        case(scan_data)
            4'd0:  //0
                begin
                    col1<=8'h0e;  //第一行有效时的 8 位数据编码
                    col2<=8'h11;  //第二行有效时的 8 位数据编码
                    col3<=8'h13;  //第三行有效时的 8 位数据编码
                    col4<=8'h15;  //第四行有效时的 8 位数据编码
                    col5<=8'h19;  //第五行有效时的 8 位数据编码
                    col6<=8'h11;  //第六行有效时的 8 位数据编码
                    col7<=8'h0e;  //第七行有效时的 8 位数据编码
                    col8<=8'h00;  //第八行有效时的 8 位数据编码
                end
            4'd1:  //1
                begin
                    col1<=8'h04;
                    col2<=8'h0c;
                    col3<=8'h04;
                    col4<=8'h04;
                    col5<=8'h04;
                    col6<=8'h04;
                    col7<=8'h0e;
                    col8<=8'h00;
```

```
                end
            //略
            4'd14: //E
                begin
                    col1<=8'h1f;
                    col2<=8'h10;
                    col3<=8'h10;
                    col4<=8'h1e;
                    col5<=8'h10;
                    col6<=8'h10;
                    col7<=8'h1f;
                    col8<=8'h00;
                end
            4'd15: //F
                begin
                    col1<=8'h1f;
                    col2<=8'h10;
                    col3<=8'h10;
                    col4<=8'h1e;
                    col5<=8'h10;
                    col6<=8'h10;
                    col7<=8'h10;
                    col8<=8'h00;
                end
            default: //全暗
                begin
                    col1<=8'b00000000;
                    col2<=8'b00000000;
                    col3<=8'b00000000;
                    col4<=8'b00000000;
                    col5<=8'b00000000;
                    col6<=8'b00000000;
                    col7<=8'b00000000;
                    col8<=8'b00000000;
                end
        endcase
    end

always@(row_buf) //逐个导通每一行
    begin
        case(row_buf) //每导通新的一行，重新装载这行对应的 8 列数据
            8'b0000_0001://此处下划线无意义不编译，只是为了便于阅读
                col_buf=col1; //第 1 行选通时，将现在要显示字符的第 1 行数据输出到点阵数据线
            8'b0000_0010:
                col_buf=col2; //第 2 行选通时，将现在要显示字符的第 2 行数据输出到点阵数据线
            8'b0000_0100:
```

```
                col_buf=col3; //第 3 行选通时，将现在要显示字符的第 3 行数据输出到点阵数据线
            8'b0000_1000:
                col_buf=col4; //第 4 行选通时，将现在要显示字符的第 4 行数据输出到点阵数据线
            8'b0001_0000:
                col_buf=col5; //第 5 行选通时，将现在要显示字符的第 5 行数据输出到点阵数据线
            8'b0010_0000:
                col_buf=col6; //第 6 行选通时，将现在要显示字符的第 6 行数据输出到点阵数据线
            8'b0100_0000:
                col_buf=col7; //第 7 行选通时，将现在要显示字符的第 7 行数据输出到点阵数据线
            8'b1000_0000:
                col_buf=col8; //第 8 行选通时，将现在要显示字符的第 8 行数据输出到点阵数据线
            default:
                col_buf=8'h00;
        endcase
    end

assign led_row=row_buf; //输出到点阵的行选择线（高电平有效）
assign led_col=~col_buf; //输出到点阵数据线（低电平有效）
//注意行、列变量的高低位顺序，顺序不对将会导致显示的左右或上下颠倒
```

实验十三 LCD 液晶显示系统的设计

要求：利用 EDA-E LC4256V CPLD 开发板，以纯 Verilog 语言或纯 VHDL 语言方式输入设计，也可以电原理图 Schematic 和 HDL 语言混合方式输入设计，实现一个 LCD1602 液晶显示的系统，在开发板上的 LCD1602 液晶屏上分两排显示十六进制数字 0~F，并可适当添加发挥功能。

重点：LCD1602 液晶屏显示的原理及其应用，其中包括 LCD1602 的引脚、指令集及现有字符和自定义字符显示的原理和实现方法。

- **功能现象：**

打开开发板电源，在开发板上的 LCD1602 液晶屏上，第一行滚动显示十六进制数字 0~7，第二行滚动显示十六进制数字 8~F，两行显示结束之后清屏并重新循环显示以上字符。

- **实现原理：**

1. LCD1602 简介

LCD1602 是工业字符型液晶，能够同时显示 16×2 即 32 个字符（16 列 2 行），如图 9-25 所示。

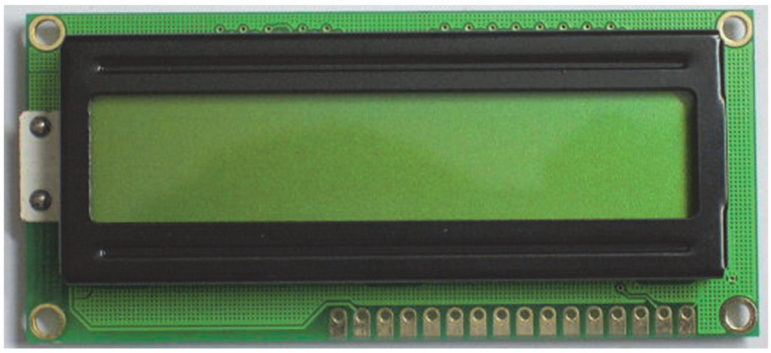


图 9-25 1602 的实物照片图

LCD1602 是一种专门用来显示字母、数字、符号等的点阵型液晶模块。它由若干个 5×8 或 5×10 的点阵字符位组成（注：5 为宽度，8 或 10 为高度；也有的 1602 点阵是 5×7 或者 5×11），每个点阵字符位都可以显示一个字符，每位之间有一个点距的间隔，每行之间也有间隔，起到了字符间距和行间距的作用，正因为如此所以它不能很好地显示图形（用自定义 CGRAM 即 Character Generator RAM 字符产生 RAM，显示效果也不好）。市面上字符液晶大多数是基于 HD44780 液晶芯片的，控制原理是完全相同的，因此基于 HD44780 写的控制程序可以很方便地应用于市面上大部分的字符型液晶。第 1 行 16 个黑方块，第 2 行空白无显示，这是 1602 上电复位成功的标志，说明 1602 液晶本身是正常的。由于 1602 是慢速器件，所以时序要注意参考其手册，操作需加入合适的延时，而且初始化很重要。

2. LCD1602 的引脚说明

字符型 LCD 通常有 14 条引脚线或 16 条引脚线的 LCD，多出来的 2 条线是背光电源线 VCC（15 脚）和地线 GND（16 脚），16 脚 LCD 的控制原理与 14 脚的完全一样，定义如图 9-26 及表 9-5 所示。

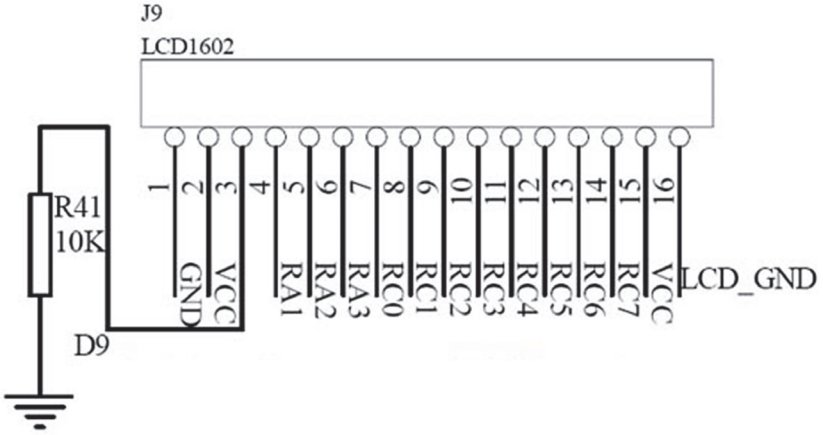


图 9-26 LCD1602 的引脚图

表 9-5 LCD1602 的引脚定义

引脚号	引脚名	电平	输入/输出	功能说明
1	VSS			电源地，一般接地
2	VDD			接电源（+5V）
3	V0（或 Vee 或 VL）			对比调整电压，为液晶显示器对比度调整端，接正电源时对比度最弱，接地电源时对比度最高（对比度过高时会产生“鬼影”，使用时可以通过一个 10kΩ的电位器调整对比度）。
4	RS	0/1	输入	寄存器选择：1 时选择数据寄存器、输入数据；0 时选择指令寄存器、输入指令。
5	R/W	0/1	输入	读写信号线：1 时进行读操作、从 LCD 读取信息；0 时进行写操作、向 LCD 写入指令或数据。
6	E	1；1→0	输入	使能(EN 即 Enable)端：1 时读取信息；1→0 下降沿时执行指令。
7	DB0	0/1	输入/输出	低 4 位三态、 双向数据总线 0 位（最低位）
8	DB1	0/1	输入/输出	低 4 位三态、 双向数据总线 1 位
9	DB2	0/1	输入/输出	低 4 位三态、 双向数据总线 2 位
10	DB3	0/1	输入/输出	低 4 位三态、 双向数据总线 3 位
11	DB4	0/1	输入/输出	高 4 位三态、 双向数据总线 4 位
12	DB5	0/1	输入/输出	高 4 位三态、 双向数据总线 5 位
13	DB6	0/1	输入/输出	高 4 位三态、 双向数据总线 6 位
14	DB7	0/1	输入/输出	高 4 位三态、 双向数据总线 7 位（最高位）（也是忙标志 busy flag，当 busy flag 为 1 时，LCD 忙，将无法再处理其他的指令要求）
15	BLA	+Vcc		LCD 背光电源正极（或空脚）
16	BLK	接地		LCD 背光电源负极（或空脚）

3. LCD1602 的字符集

LCD1602（HD44780）内置了 DDRAM、CGROM 和 CGRAM。

DDRAM（Display Data RAM）就是显示数据 RAM，用来寄存待显示的字符代码。共 80 个字节，其地址和屏幕的对应关系如表 9-6 所示：

表 9-6 地址和屏幕的对应关系表

	显示位置	1	2	3	4	5	6	7	……	40
DDRAM 地 址	第一行	00H	01H	02H	03H	04H	05H	06H	……	27H
	第二行	40H	41H	42H	43H	44H	45H	46H	……	67H

也就是说想要在 LCD1602 屏幕的第一行第一列显示一个“A”字，就要向 DDRAM 的 00H 地址写入“A”字的代码就行了。但具体的写入是要按 LCD 模块的指令格式来进行的，后面将会介绍到。那么一行有 40 个地址，怎么写？在 1602 中我们就用前 16 个就行了，第二行也一样用前 16 个地址。对应关系如表 9-7 所示：

表 9-7 DDRAM 地址与显示位置的对应关系

00H	01H	02H	03H	04H	05H	06H	07H	08H	09H	0AH	0BH	0CH	0DH	0EH	0FH
40H	41H	42H	43H	44H	45H	46H	47H	48H	49H	4AH	4BH	4CH	4DH	4EH	4FH

我们知道文本文件中每一个字符都是用一个字节的代码记录的，而一个汉字是用两个字节的代码记录。在 PC 上我们只要打开文本文件就能在屏幕上看到对应的字符是因为在操作系统里和 BIOS 里都固化有字符字模。什么是字模？就代表了在点阵屏幕上点亮和熄灭的信息数据。例如“A”字的字模如图 9-27 所示：

01110 ○■■■○
10001 ■○○○■
10001 ■○○○■
10001 ■○○○■
11111 ■■■■■
10001 ■○○○■
10001 ■○○○■
10001 ■○○○■

图 9-27 “A”字的字模示意图

图中左边的数据就是字模数据，右边就是将左边数据用“○”代表 0，用“■”代表 1。看出是个“A”字了吧？在文本文件中“A”字的代码是 41H，PC 收到 41H 的代码后就去字模文件中将代表“A”字的这一组数据送到显卡去点亮屏幕上相应的点，你就看到“A”这个字了。

刚才介绍了想要在 LCD1602 屏幕的第一行第一列显示一个“A”字，就要向 DDRAM 的 00H 地址写入“A”字的代码 41H，可 41H 这一个字节的代码如何才能让 LCD 模块在屏幕的点阵上显示“A”字呢？跟 PC 机上类似，在 LCD 模块上也固化了字模存储器，这就是 CGROM 和 CGRAM。LCD1602（HD44780）内置了 192 个常用字符的字模，存于字符产生器 CGROM（Character Generator ROM）中，另外还有 8 个允许用户自定义的字符产生 RAM，称为 CGRAM（Character Generator RAM）。图 9-28 说明了 CGROM 和 CGRAM 与字符的对应关系，其中已经存储了 160 个不同的点阵字符图形，这些字符有：阿拉伯数字、英文字母的大小写、常用的符号、和日文假名等，每一个字符都有一个固定的代码。从图中可以看出，“A”字的对应高位代码（上面）为 0100，对应低位代码（左边）为 0001，合起来就是 01000001，也就是 41H。可见它的代码与我们 PC 中的字符代码是基本一致的，都是 ASCII 码。字符代码 0x00~0x0F 为用户自定义的字符图形 RAM 即 CGRAM，对于 5×8 点阵的字符，可以存放 8 组；对于 5×10 点阵的字符，可以存放 4 组。0x20~0x7F 为标准的 ASCII 码，0xA0~0xFF 为日文字符和希腊文字符，其余字符码（0x10~0x1F 及 0x80~0x9F）没有定义。

图 9-28 CGROM 和 CGRAM 中字符码与字符字模关系对照表

4. LCD1602 的指令集

LCD1602（HD44780）的指令集共有 11 条指令，如表 9-8 所示。

表 9-8 LCD1602 的指令集一览表

指令	指令码										说明
	RS	R/W	D7	D6	D5	D4	D3	D2	D1	D0	
清屏	0	0	0	0	0	0	0	0	0	1	清除显示，光标回原位
光标返回	0	0	0	0	0	0	0	0	1	*	ADD=0，光标回原位
输入方式	0	0	0	0	0	0	0	1	I/D	S	是否移动及移动方向
显示开关	0	0	0	0	0	0	1	D	C	B	D：显示、C：光标、B：光标闪烁
移位	0	0	0	0	0	1	S/C	R/L	*	*	移位光标及整体显示
功能设置	0	0	0	0	1	DL	N	F	*	*	数据位数、行数、字体
CGRAM 地址设置	0	0	0	1	A5	A4	A3	A2	A1	A0	设置 CGRAM 的地址
DDRAM 地址设置	0	0	1	A6	A5	A4	A3	A2	A1	A0	设置 DDRAM 的地址
忙标志/读地址计数器	0	1	BF	AC6	AC5	AC4	AC3	AC2	AC1	AC0	读出忙标志位（BF）及 AC 值
CGRAM/DDRAM 数据写	1	0	写数据								将内容写入 RAM 中
CGRAM/DDRAM 数据读	1	1	读数据								将内容从 RAM 中读出
说明	I/D：1——增量方式、0——减量方式； S：1——位移、0－－不位移； D：1——显示开、0——显示关； C：1——光标开、0——光标关； B：1——光标闪烁开、0——光标闪烁关； S/C：1——显示移位、0——光标移位； R/L：1——右移、0——左移； DL：1——8 位、0——4 位； N：1——2 行、0——1 行； F：1——5x10 字体、0——5x8 字体； BF：1——正在执行内部操作、0——可接受指令。										DDRAM：显示数据 RAM CGRAM：字符发生器 RAM ACG：CGRAM 地址 ADD：DDRAM 地址及光标地址

(1)、清屏指令（如表 9-9 所示）

表 9-9 清屏指令

指令功能	指令编码										执行时间 /ms
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
清屏	0	0	0	0	0	0	0	0	0	1	1.64

功能：

<1> 清除液晶显示器，即将 DDRAM 的内容全部填入“空白”的 ASCII 码 20H；

<2> 光标归位，即将光标撤回液晶显示屏的左上方；

<3> 将地址计数器 AC（Address Counter）的值设为 0。

(2)、光标归位指令（如表 9-10 所示）

表 9-10 光标归位指令

指令功能	指令编码										执行时间 /ms
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
光标归位	0	0	0	0	0	0	0	0	1	X	1.64

功能：
<1> 把光标撤回到显示器的左上方；
<2> 把地址计数器 AC 的值设置为 0；
<3> 保持 DDRAM 的内容不变。
X 表示任意值均可。

(3)、输入方式（进入模式）设置指令（如表 9-11 所示）

表 9-11 输入方式设置指令

指令功能	指令编码										执行时间 /us
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
进入模式设置	0	0	0	0	0	0	0	1	I/D	S	40

功能：设定每次输入 1 位数据后光标的移位方向，并且设定每次写入一个字符后是否移动。参数设定的情况如下所示：
I/D：0=写入新数据后光标左移，1=写入新数据后光标右移；
S：0=写入新数据后显示屏不移动，1=写入新数据后显示屏整体右移 1 个字符。

(4)、显示开关控制指令（如表 9-12 所示）

表 9-12 显示开关控制指令

指令功能	指令编码										执行时间 /us
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
显示开关控制	0	0	0	0	0	0	1	D	C	B	40

功能：控制显示器开/关、光标显示/关闭以及光标是否闪烁。参数设定的情况如下：
D：0=显示功能关，1=显示功能开；
C：0=无光标，1=有光标；
B：0=光标闪烁，1=光标不闪烁。

(5)、设定显示屏或光标移动方向指令（如表 9-13 所示）

表 9-13 设定显示屏或光标移动方向指令

指令功能	指令编码										执行时间 /us
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
设定显示屏或光标移动方向	0	0	0	0	0	1	S/C	R/L	X	X	40

功能：使光标移位或使整个显示屏幕移位。参数设定的情况如表 9-14 所示：

表 9-14 显示屏及光标移动方向指令的具体设置

S/C	R/L	设定情况
0	0	光标左移 1 格，且 AC 值减 1
0	1	光标右移 1 格，且 AC 值加 1
1	0	显示器上字符全部左移一格，但光标不动
1	1	显示器上字符全部右移一格，但光标不动

(6)、功能设定指令（如表 9-15 所示）

表 9-15 功能设定指令

指令功能	指令编码										执行时间 /us
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
功能设定	0	0	0	0	1	DL	N	F	X	X	40

功能：设定数据总线位数、显示的行数及字型。参数设定的情况如下：

DL：0=数据总线为 4 位，1=数据总线为 8 位；

N：0=显示 1 行，1=显示 2 行；

F：0=5×8 点阵/每字符，1=5×10 点阵/每字符。

(7)、设定 CGRAM 地址指令（如表 9-16 所示）

表 9-16 设定 CGRAM 地址指令

指令功能	指令编码										执行时间 /us
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
设定CGRAM 地址	0	0	0	1	CGRAM的地址(6位)						40

功能：设定下一个要存入数据的 CGRAM 的地址。DB5、DB4、DB3 为字符号，也就是你将来要显示该字符时要用到的 CGRAM 字符地址（000~111，能定义八个字符）；DB2、DB1、DB0 为行号（000~111，例如 5×8 的 8 行中的哪一行）。

(8)、设定 DDRAM 地址指令（如表 9-17 所示）

表 9-17 设定 DDRAM 地址指令

指令功能	指令编码										执行时间 /us
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
设定DDRAM 地址	0	0	1	DDRAM的地址(7位)							40

功能：设定下一个要存入数据的 DDRAM 的地址。00~0FH 及 40~4FH，故最多为 7 位。

(9)、读取忙信号或 AC 地址指令（如表 9-18 所示）

表 9-18 读取忙信号或 AC 地址指令

指令功能	指令编码										执行时间 /us
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
读取忙碌信号或AC地址	0	1	FB	AC内容(7位)							40

功能：

<1> 读取忙碌信号 BF 的内容，BF=1 表示液晶显示器忙，暂时无法接收控制器送来的数据或指令；当 BF=0 时，液晶显示器可以接收控制器送来的数据或指令；

<2> 读取地址计数器 AC 的内容。

(10)、数据写入 DDRAM 或 CGRAM 指令（如表 9-19 所示）

表 9-19 数据写入 DDRAM 或 CGRAM 指令

指令功能	指令编码										执行时间 /us
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
数据写入到 DDRAM或 CGRAM	1	0	要写入的数据 D7～D0								40

功能：

<1> 将字符码写入 DDRAM，以使液晶显示屏显示出相对应的字符（例如写入 41H，即显示“A”）；

<2> 将使用者自己设计的图形即自定义字符存入 CGRAM。DB7、DB6、DB5 可为任何数据，一般取“000”。DB4、DB3、DB2、DB1、DB0 对应于每行 5 点的字模数据（例如 5×8 或 5×10 点阵每行的 5 点）。

注：对 DDRAM 写入数据时是写入字符码（如 ASCII 码），即字符在字模库表中的位置；而对 CGRAM 写入数据时是写入字模点阵数据。

(11)、从 CGRAM 或 DDRAM 读出数据指令（如表 9-20 所示）

表 9-20 从 CGRAM 或 DDRAM 读出数据指令

指令功能	指令编码										执行时间 /us
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
从CGRAM或 DDRAM读出 数据	1	1	要读出的数据 D7～D0								40

功能：读取 DDRAM 或 CGRAM 中的内容。

综合上述所有指令，LCD 读写的基本操作时序和功能如表 9-21 所示：

表 9-21 LCD 读写的基本操作时序和功能

RS	R/W	操作说明
0	0	写入指令寄存器（清除屏等）
0	1	读 busy flag（DB7），以及读取位址计数器（DB0~DB6）值
1	0	写入数据寄存器（显示各字型等）
1	1	从数据寄存器读取数据

相关说明如下：

- 写指令：输入：RS=L，RW=L，E=下降沿脉冲，DB0~DB7=指令码，输出：无。
- 读状态：输入：RS=L，RW=H，E=H，输出：DB0~DB7=状态字。
- 写数据：输入：RS=H，RW=L，E=下降沿脉冲，DB0~DB7=数据，输出：无。
- 读数据：输入：RS=H，RW=H，E=H，输出：DB0~DB7=数据。

5. 显示字符

下面举例说明如何显示字符，以 51 单片机 C 语言程序为例说明如何控制 LCD1602 屏幕上第一行第一列显示一个“A”字。

```
//先定义接口
# include <AT89x51.h>
/****************************************
```

```
P1------DB0~DB7，P2.0------RS，P2.1------RW，P2.2------E
**************************************/
# define LCD_DB P1
sbit LCD_RS=P2^0;
sbit LCD_RW=P2^1;
sbit LCD_E=P2^2;
//******定义函数***************
# define uchar unsigned char
# define uint unsigned int
void LCD_init(void); //初始化函数
void LCD_write_command(uchar command); //写指令函数
void LCD_write_data(uchar dat); //写数据函数
void LCD_disp_char(uchar x, uchar y, uchar dat); //在某个屏幕位置上显示一个字符，x 为列（0~15），y 为行（1~2）
//void LCD_check_busy(void); //检查忙函数。一般不用此函数，因其通过率极低。
void delay_n40us(uint n); //延时函数
//*******************************
//*******初始化函数***************
void LCD_init(void)
{
LCD_write_command(0x38); //设置 8 位格式，2 行，5x8（00111000，DL=1，N=1，F=0，D5=1）
LCD_write_command(0x0c); //整体显示，关光标，不闪烁（00001100，C=0，B=0，D=1）
LCD_write_command(0x06); //设定输入方式，增量不移位（00000110，I/D=1，S=0）
LCD_write_command(0x01); //清除屏幕显示（00000001）
delay_n40us(100); //实践证明，一般用 for 循环 200 次就能可靠完成清屏指令
}
//*******************************
//********写指令函数************
void LCD_write_command(uchar dat)
{
LCD_DB=dat;
LCD_RS=0; //指令
LCD_RW=0; //写入
LCD_E=1; //允许
LCD_E=0; //产生下跳变边沿以执行
delay_n40us(1); //实践证明，一般用 for 循环 1 次就能完成普通写指令
}
//*******************************
//********写数据函数*************
void LCD_write_data(uchar dat)
{
LCD_DB=dat;
LCD_RS=1; //数据
LCD_RW=0; //写入
LCD_E=1; //允许
LCD_E=0; //产生下跳变边沿以执行
delay_n40us(1);
```

```
}
//*****************************
//*******显示一个字符函数*********
void LCD_disp_char(uchar x,uchar y,uchar dat)
{
uchar address;
if(y==1) address=0x80+x;
else address=0xc0+x;
LCD_write_command(address);
LCD_write_data(dat);
}
//*****************************
/*******检查忙函数*************
void LCD_check_busy()
//实践证明，一般此检查忙指令通过率极低，以致于不能正常使用 LCD，
//因此一般不用检查忙函数，而使用延时的方法，延时还是非常好用的，
//用 for 循环做延时，普通指令只要 1 次循环就可完成，清屏指令用 200 次循环便能完成。
{
do
{
LCD_E=0;
LCD_RS=0;
LCD_RW=1;
LCD_DB=0xff;
LCD_E=1;
}while(LCD_DB^7==1);
}
*****************************/
//********延时函数***************
void delay_n40us(uint n)
{
uint i;
uchar j;
for(i=n;i>0;i--)
      for(j=0;j<2;j++); //在这个延时循环中只做了 2 次循环
}
//*****************************
//*********主函数*****************
void main(void)
{
LCD_init();
LCD_disp_char(0,1,"A");
while(1);
}
```

6. 显示自定义字符

上面只是显示了“A”这一个字符（即将 CGROM 中的“A”写入 DDRAM）。下面介绍一下显示自定义字符的步骤：(1)、先将自定义字符写入 CGRAM；(2)、将 CGRAM 中的自定义字符送到 DDRAM 中显示。

查看 LCD1602 的 CGROM 字符代码表，可以发现从 00000000B~00000111B（00H~07H）地址的内容是没有定义的，它就是留给用户自己定义的 CGRAM。用户可以通过先定义 LCD1602 的 CGRAM 中的内容，然后再同调用 CGROM 中的字符一样来调用自定义好的 CGRAM 字符（自定义字符最多可写 8 个）。那么如何设定 CGRAM 中的内容呢？首先我们要把所要编写的字符对应于 5×8 点阵的“字模”提取出来，我们可以通过相关的软件来提取（类似于前面的实验十二 LED 点阵显示设计实验中的 fang.exe 小软件），也可以手工提取。说白了也就是将点阵的某一行中有显示的点用 1 表示，无显示的点用 0 表示，以此形成该行对应的字模数据。

设定 CGRAM 的内容，要一行一行地设定，每一行对应一个 CGRAM 地址，每个地址对应一个 5×8 点阵，每行 5 点，共 8 行，因此要将 8 行的字模数据都写入 CGRAM。写好后，就可像调用 CGROM 字符一样来调用它了。

定义一行的内容，分两步：

(1)、设定行地址（CGRAM 地址）：（即前面的第(7)条设定 CGRAM 地址指令）

用到的指令如下：

RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
0	0	0	1	DATA					

其中：DB5、DB4、DB3 为字符号，也就是你将来要显示该字符时要用到的字符在 CGRAM 中的地址（0~7）；DB2、DB1、DB0 为行号（点阵的 8 行中的哪一行，0~7）。

(2)、设定 CGRAM 数据（内容）：（即前面的第(10)条数据写入 DDRAM 或 CGRAM 指令）

指令码如下：

RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
1	0	DATA							

其中：DB4、DB3、DB2、DB1、DB0 对应于点阵的 5 行中每行 5 点的字模数据；DB7、DB6、DB5 可为任何数据，一般取“000”。

下面仍以 51 单片机 C 语言程序为例，说明如何显示自定义字符。

```
#include<reg52.h>
#include <intrins.h>
sbit lcd1602_rs=P0^7;
sbit lcd1602_rw=P0^6;
sbit lcd1602_e=P0^5;
#define lcd1602_sent_dat P2
#define lcd1602_dat lcd1602_rs=1;
#define lcd1602_com lcd1602_rs=0;
#define lcd1602_write lcd1602_rw=0;
#define lcd1602_read lcd1602_rw=1;
#define lcd1602_e_able lcd1602_e=1;
#define lcd1602_e_unable lcd1602_e=0;

//此函数定义八个字符分别写入 CGRAM 的八个地址
unsigned char code pic[8][8]={
/*--调入了五幅图像--*/
/*--宽度 x 高度=5x8--*/
/*--宽度不是 8 的倍数，现调整为：宽度 x 高度=8x8--*/
/*--因为前面的指令(10)中 DB7~5 为 000，所以这里左 3 列即高 3 位均为 0 已有了不用再加了--*/
{0x04,0x0E,0x15,0x04,0x04,0x04,0x04,0x00}, //向上箭头
{0x00,0x04,0x04,0x04,0x04,0x15,0x0E,0x04}, //向下箭头
```

```
{0x00,0x04,0x08,0x1F,0x08,0x04,0x00,0x00}, //向左箭头（实际上在 CGROM 中已有左右箭头）
{0x00,0x04,0x02,0x1F,0x02,0x04,0x00,0x00}, //向右箭头
{0x04,0x04,0x0A,0x1F,0x1F,0x0A,0x04,0x04}, //占位符
{0,0,0,0,0,0,0,0}, //全灭无显示
{0,0,0,0,0,0,0,0},
{0,0,0,0,0,0,0,0}
};

unsigned char lcd1602_read_dat()
{
    unsigned char dat;
    lcd1602_read;
    lcd1602_dat;
    lcd1602_e_able;
    dat=lcd1602_sent_dat;
    lcd1602_e_unable;
    return dat;
}

void read_busy()
{
while(lcd1602_read_dat()&0x80); //检查 DB7 即 busyflag
}

void lcd1602_write_dat(unsigned char dat)
{
    char i;
    lcd1602_sent_dat=dat;
    lcd1602_dat;
    lcd1602_write;
    for(i=0;i<80;i++)
        _nop_();
    lcd1602_e_able;
    for(i=0;i<80;i++)
        _nop_();
    lcd1602_e_unable;
    read_busy();
}
void lcd1602_write_com(unsigned char com)
{
    char i;
    lcd1602_sent_dat=com;
    lcd1602_com;
    lcd1602_write;
    for(i=0;i<80;i++)
        _nop_();
```

```
    lcd1602_e_able;
    for(i=0;i<80;i++)
        _nop_();
    lcd1602_e_unable;
    read_busy();
}

//初始化
void init()
{
    read_busy();
    lcd1602_write_com(0x38); //指令(6)
    lcd1602_write_com(0x0c); //指令(4)
    lcd1602_write_com(0x06); //指令(3)
    lcd1602_write_com(0x01); //指令(1)
    lcd1602_write_com(0x02); //指令(2)
}


//显示字符串
void lcd1602_write_character(unsigned char add,unsigned char *p)
{
    lcd1602_write_com(add);
    while (*p!='\0')
    {
        lcd1602_write_dat(*p++);
    }
}

//将自定义字符写入 CGRAM
//add 是 CGRAM 的地址（0~7）
//*pic_num 是指向一个 8 位数组的首地址
void lcd1602_write_pic(unsigned char add,unsigned char *pic_num)
{
    unsigned char i;
    add=add<<3; //左移 3 位，因为在 DB5、DB4、DB3 位置
    for(i=0;i<8;i++) //5*8，共 8 行
    {
        lcd1602_write_com(0x40|add+i); //指令(7)
        lcd1602_write_dat(*pic_num++); //指令(10)
    }
}

void main()
{
    int i=0;
```

```
    init();
    //显示 CGROM 中的字符
    lcd1602_write_com(0x80);
    lcd1602_write_dat('6');
    //写八个自定义字符到 CGRAM
    for(i=0;i<8;i++)
        lcd1602_write_pic(i,pic[i]);
    //第一行显示八个自定义字符
    lcd1602_write_com(0x80+0x01); //指令(8)
    for(i=0;i<0x08;i++)
        lcd1602_write_dat(i);
    //第二行显示“test!success!”
    lcd1602_write_character(0x80+0x40,"test!success!");
    while(1);
}
```

7. 用 CPLD/FPGA 控制 1602 显示字符

LCD1602 液晶的读/写操作、屏幕和光标的操作都是通过指令编程来实现的，为了方便控制，在 CPLD/FPGA 中可以采用状态机实现设计。在设计中设置 8 个状态，分别是起始状态 CLEAR、设置 DDRAM 地址状态 SETDDRAM、工作方式设置状态 SETFUNCTION、显示方式设置状态 SWITCHMODE、输入方式设置状态 SETMODE、光标归位状态 RETURNCURSOR、字符换行状态 SHIFT、写 RAM 状态 WRITERAM。其控制的状态转移图如图 9-29 所示：

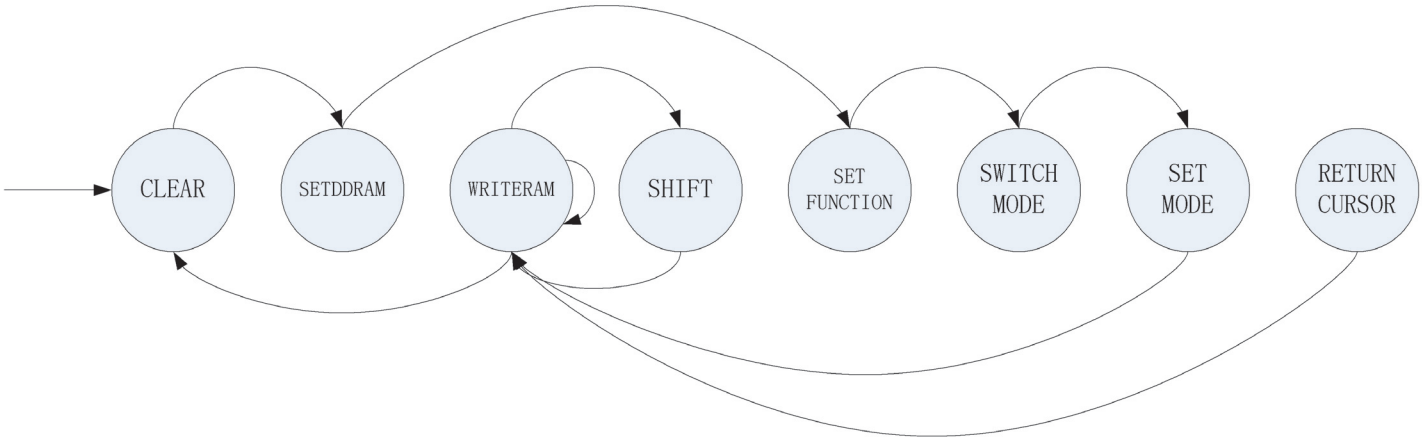


图 9-29 PLD 控制 1602 的状态转移图

- **部分参考源代码（Verilog 语言）：**

```
parameter CLEAR          =8'b00000001;     //清屏
parameter SETDDRAM       =8'b00000010;     //设置 DDRAM 地址
parameter SETFUNCTION    =8'b00000100;     //工作方式设置
parameter SWITCHMODE     =8'b00001000;     //显示状态设置
parameter SETMODE        =8'b00010000;     //输入方式设置
parameter RETURNCURSOR=8'b00100000;     //光标归位
parameter SHIFT          =8'b01000000;     //换行
parameter WRITERAM       =8'b10000000;     //写 DDRAM
parameter CUR_INC        =1; //完成一个字符码传送后，地址计数器 AC 自动加 1
parameter CUR_NOSHIFT    =0; //显示不发生移位
parameter OPEN_DISPLAY   =1; //开显示
```

```
    parameter OPEN_CUR      =0; //光标不显示
    parameter BLANK_CUR     =0; //光标不闪烁
    parameter DATAWIDTH8    =1; //数据总线的宽度设置为 8 位
    parameter DATAWIDTH4    =0; //数据总线的宽度设置为 4 位
    parameter TWOLINE       =1; //显示行数为 2 行
    parameter ONELINE       =0; //显示行数为 1 行
    parameter FONT5X10      =1; //显示格式为 5*10 点阵
    parameter FONT5X8       =0; //显示格式为 5*8 点阵
    parameter DATA_IN       ="    01234567        89ABCEDF    ";
    //此 DATA_IN 的前面 4 个字符、中间 8 个字符及后面 4 个字符均为空格
    //注意：此句更换不同显示内容时可能超规模（占用宏单元个数超过 LC4256V 的 256 个）

//从 50MHz 系统时钟得到低频时钟 clkhz
    always @(posedge clk_50m)
        begin
        if(cnt==20'hfffff)
            begin
                cnt<=0;
                clkhz<=~clkhz;
            end
        else
            cnt<=cnt+1;
        end
    assign lcd_en=clkhz;

//1602 工作状态机
    always @(posedge clkhz or negedge reset_n)
        if(!reset_n) //异步复位
            begin
                lcd_rs<=0;
                lcd_rw<=0; //写指令寄存器
                lcd_d<=8'b0000_0001; //清屏指令
                state<=CLEAR;
                disp_count<=6'b0;
            end
        else
            begin
                case(state)
                    CLEAR: //清屏
                            begin
                                lcd_rs<=0;
                                lcd_rw<=0; //写指令寄存器
                                lcd_d<=8'b0000_0001; //清屏指令
                                data_in_buf<=DATA_IN;
                                state<=SETDDRAM;
                            end
```

```
SETDDRAM: //地址设置
        begin
            lcd_rs<=0;
            lcd_rw<=0;
            lcd_d<=8'b1000_0000;//起始地址
            state<=SETFUNCTION;
        end
SETFUNCTION: //工作方式设置
        begin
            lcd_rs<=0;
            lcd_rw<=0;
            lcd_d[7:5]<=3'b001;
            lcd_d[4]<=DATAWIDTH8;
            lcd_d[3]<=TWOLINE;
            lcd_d[2]<=FONT5X8;
            lcd_d[1:0]<=2'b00;
            state<=SWITCHMODE;
        end
SWITCHMODE: //显示开关控制
        begin
            lcd_rs<=0;
            lcd_rw<=0;
            lcd_d[7:3]<=5'b00001;
            lcd_d[2]<=OPEN_DISPLAY;
            lcd_d[1]<=OPEN_CUR;
            lcd_d[0]<=BLANK_CUR;
            state<=SETMODE;
        end
SETMODE: //设置输入模式
        begin
            lcd_rs<=0;
            lcd_rw<=0;
            lcd_d[7:2]<=6'b000001;
            lcd_d[1]<=CUR_INC;
            lcd_d[0]<=CUR_NOSHIFT;
            state<=WRITERAM;
        end
RETURNCURSOR: //光标归位
        begin
            lcd_rs<=0;
            lcd_rw<=0;
            lcd_d<=8'b00000010; //归位指令设地址计数器 AC=0，
            //把光标及光标所在的字符回原点；DDRAM 内容不变
            state<=WRITERAM;
        end
SHIFT: //换行后写 DDRAM 显示第二行第一个字符
```

```
                begin
                    lcd_rs<=1; //数据
                    lcd_rw<=0;
                    lcd_d<=data_in_buf[255:248];
                    data_in_buf<=(data_in_buf<<8); //移位准备显示下一个字符
                    disp_count<=disp_count+1'b1; //已显示字符个数计数值加 1
                    state<=WRITERAM;
                end
            WRITERAM: //写 DDRAM 显示
                begin
                    lcd_rs<=1;
                    lcd_rw<=0;
                    if(disp_count==32)
                        begin
                            disp_count<=4'b0;
                            state<=CLEAR; //显示完 32 个字符重新清屏显示
                        end
                    else  if(disp_count==16) //16 时先换行再写 DDRAM 显示，注意要做两件事
                        begin
                            lcd_rs<=0; //指令
                            lcd_rw<=0;
                            lcd_d<=8'b11000000; //16 个字符发完换到第二行，地址 40H
                            state<=SHIFT;
                        end
                    else
                        begin
                            lcd_d<=data_in_buf[255:248];
                            data_in_buf<=(data_in_buf<<8);
                            disp_count<=disp_count+1'b1;
                            state<=WRITERAM;
                        end
                end
        endcase
    end
```

实验十四 VGA 显示系统的设计

要求：利用 EDA-E LC4256V CPLD 开发板，以纯 Verilog 语言或纯 VHDL 语言方式输入设计，也可以电原理图 Schematic 和 HDL 语言混合方式输入设计，实现一个在 VGA 显示器屏幕上显示彩条信号的系统，并可适当添加发挥功能。这里设 VGA 显示器为 800×600@75Hz 格式。同时设置一个显示方式控制开关，开关置 OFF 时屏幕显示水平条幅，开关置 ON 时屏幕显示垂直条幅，从左至右、从上至下的彩条颜色分别为红、绿、黄、蓝、品红、青、白、黑。

重点：VGA 扫描显示的工作原理，以及行、帧同步电路的设计方法。

- **功能现象：**

通过 VGA 连接线将开发板连接至 VGA 显示器，拨码开关 CS1 置 OFF，屏幕显示水平条幅；拨码开关 CS1 置 ON，屏幕显示垂直条幅。从左至右、从上至下的彩条颜色分别为红、绿、黄、蓝、品红、青、白、黑，其三基色分量如表 9-22 所示。

表 9-22 彩条的三基色合成

彩条	蓝基色 (blue)分量	绿基色 (green)分量	红基色 (red)分量	彩条	蓝基色 (blue)分量	绿基色 (green)分量	红基色 (red)分量
红	0	0	255	品红	255	0	255
绿	0	255	0	青	255	255	0
黄	0	255	255	白	255	255	255
蓝	255	0	0	黑	0	0	0

- **实验原理：**

1. VGA 显示器原理

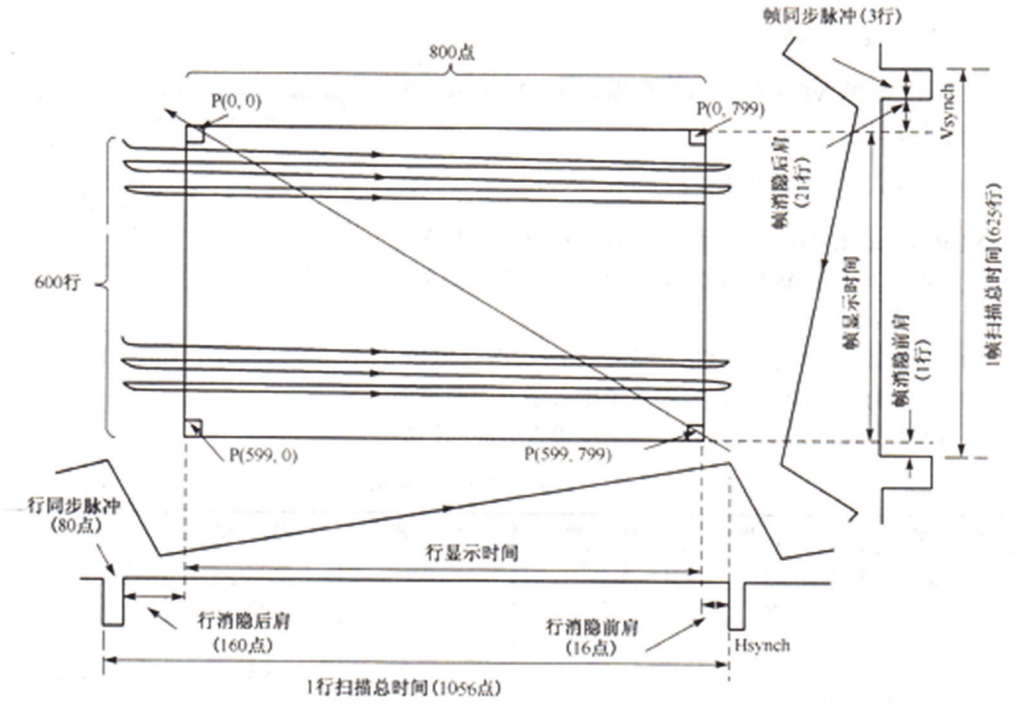
常见的 VGA 彩色显示器一般由 CRT（阴极射线管）构成，彩色是由 R、G、B，即红、绿、蓝三基色组成。大部分 CRT 采用逐行扫描方式实现图像显示，由 VGA 控制模块产生的水平同步信号和垂直同步信号控制阴极射线枪产生电子束，打在涂有荧光粉的荧光屏上，产生 R、G、B 三基色，合成一个彩色像素。其中，水平同步即行同步，垂直同步即帧同步（有时也称为场同步）。

如图 9-30 所示，扫描从屏幕的左上方开始，由左至右，由上到下，逐行进行扫描，每扫完一行，电子束回到屏幕下一行的起始位置。在回扫期间，CRT 对电子束进行消隐，每行结束是用行同步脉冲 Hsynch 进行行同步。扫描完所有行，再由帧同步脉冲 Vsynch 进行帧同步，并使扫描回到屏幕的左上方，同时进行帧消隐，预备下一帧的扫描。在回扫期间消隐信号 blank 输出低电平，对电子束进行消隐。图 9-30 中的(a)和(b)小图实际上是一样的，(b)小图是标准说明书中的官方版本。

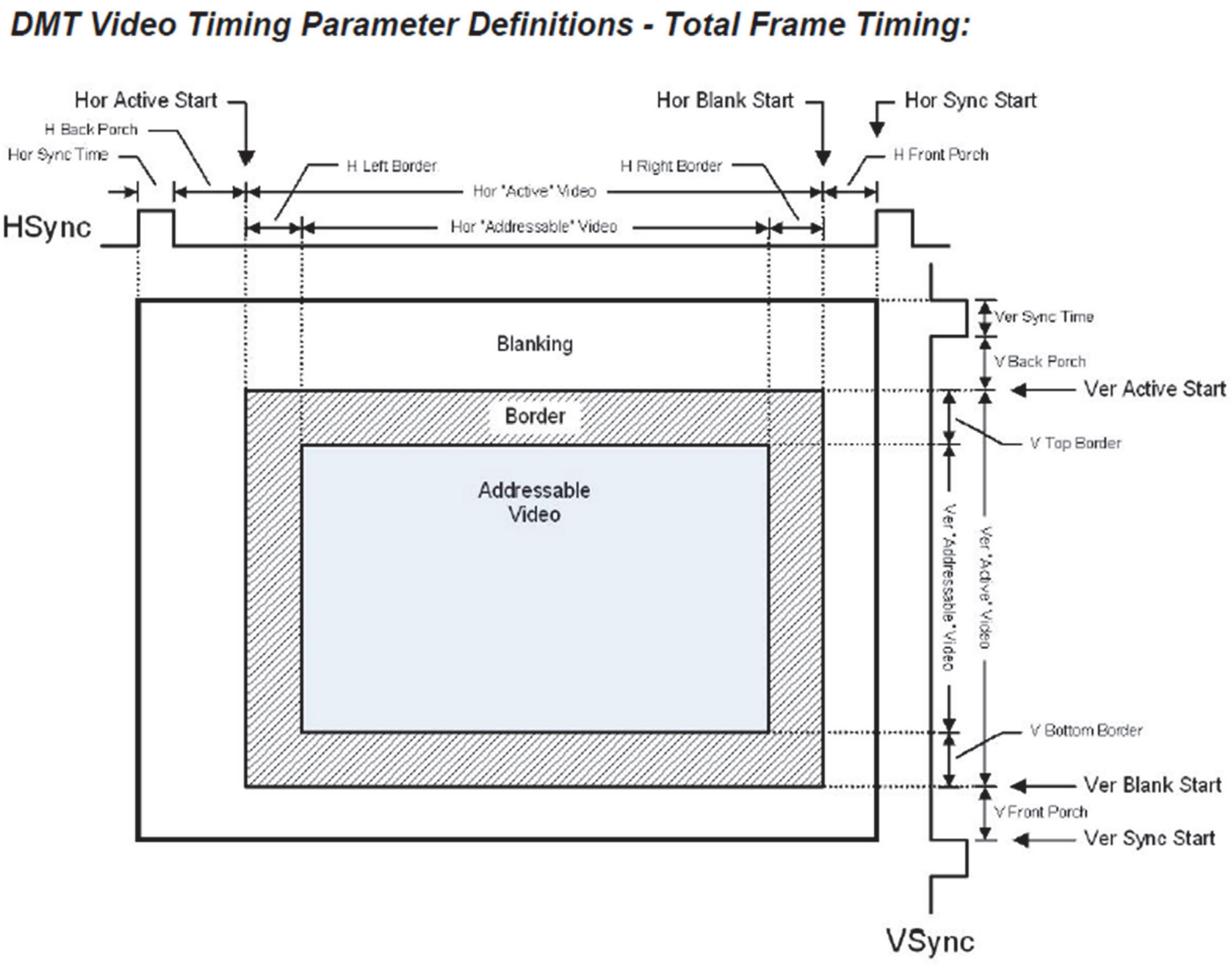
行同步信号 Hsync 和帧同步信号 Vsync 是两个重要的信号。在显示过程中，图像格式 800×600@75Hz 的行同步信号 Hsync 和帧同步信号 Vsync 的时序关系如图 9-30 及图 9-31 所示。行同步信号 Hsync 和帧同步信号 Vsync 均由同步脉冲(synch)、消隐前肩(fp，Front Porch，也称前沿)、消隐后肩(bp，Back Porch，也称后沿)和有效显示区域(active)组成，其中行同步信号 Hsync 包含行同步脉冲 Hsynch，帧同步信号 Vsync 包含帧同步脉冲 Vsynch。nblank 信号为消隐信号，在回扫期间输出低电平以对电子束进行消隐。图 9-31 中的(a)和(b)小图实际上是一样的，(b)小图是标准说明书中的官方版本。各个时序参数如表 9-23 所示。像素时钟 clk 频率约为 50MHz(1056×625×75)。

表 9-23 图像格式 800×600@75Hz 的时序参数（时钟：50MHz(1064×626×75)）

行同步信号 HS 参数（单位：像素）					帧同步信号 VS 参数（单位：行）				
Hactive	Hfp	Hsynch	Hbp	Htotal	Vactive	Vfp	Vsynch	Vbp	Vtotal
800	16	80	160	1056	600	1	3	21	625

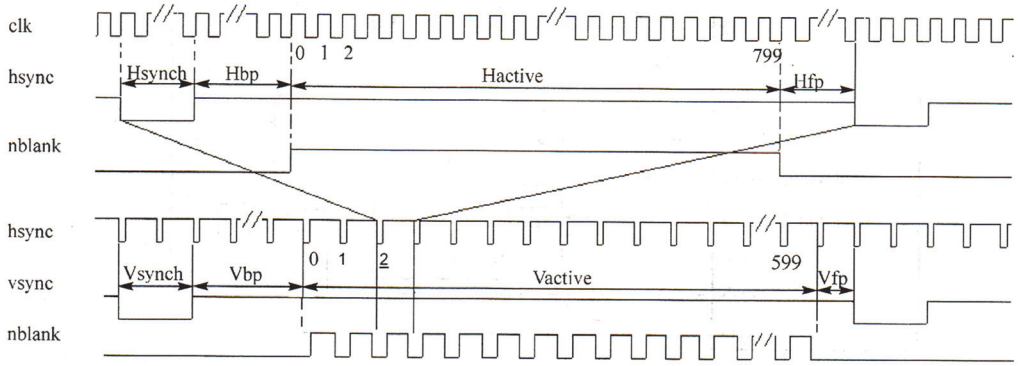


(a)

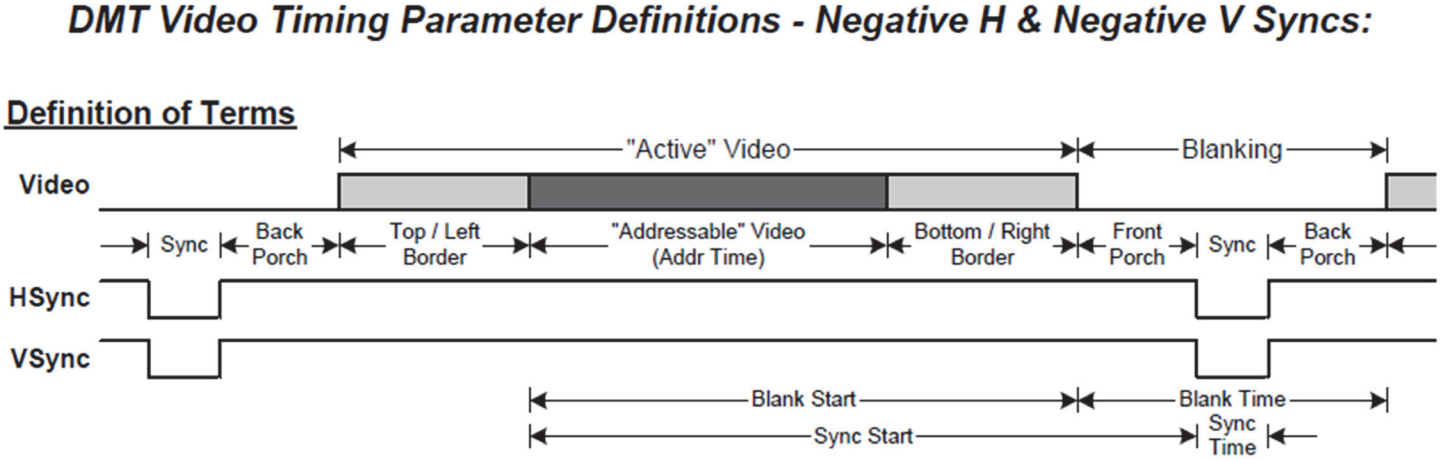


(b)

图 9-30 显示器屏幕与时序对应关系



(a)



(b)

图 9-31 行、帧同步信号的时序图

以下根据 VGA 时序标准给出其他几种图像格式的时序参数及时钟供参考（如表 9-24 所示）。

表 9-24 其他几种图像格式的时序参数

图像格式：640×480@75Hz					**时钟：31.5MHz(840×500×75)**				
Hactive	Hfp	Hsynch	Hbp	Htotal	Vactive	Vfp	Vsynch	Vbp	Vtotal
640	16	64	120	840	480	1	3	16	500
图像格式：800×600@60Hz					**时钟：40MHz(1056×628×60)**				
Hactive	Hfp	Hsynch	Hbp	Htotal	Vactive	Vfp	Vsynch	Vbp	Vtotal
800	40	128	88	1056	600	1	4	23	628
图像格式：1024×768@60Hz					**时钟：65MHz(1344×806×60)**				
Hactive	Hfp	Hsynch	Hbp	Htotal	Vactive	Vfp	Vsynch	Vbp	Vtotal
1024	24	136	160	1344	768	3	6	29	806

VGA 显示原理的详情请参见 Coodinated Video Timings (VESA-CVT) STANDARD PROPOSAL 2002（vesa_timing_2002.pdf）及 Proposed VESA and Industry Standards and Guidelines for Computer Display Monitor Timing (DMT) 2008（vesa_timing.pdf）。

2. VGA 接口连接

标准 15 针的 VGA 输出接口引脚如图 9-32 所示，其硬件电路如图 9-33 所示，各引脚的定义如表 9-25 所示。

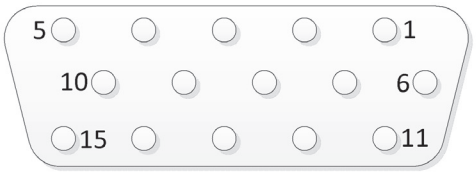


图 9-32 VGA 输出接口引脚

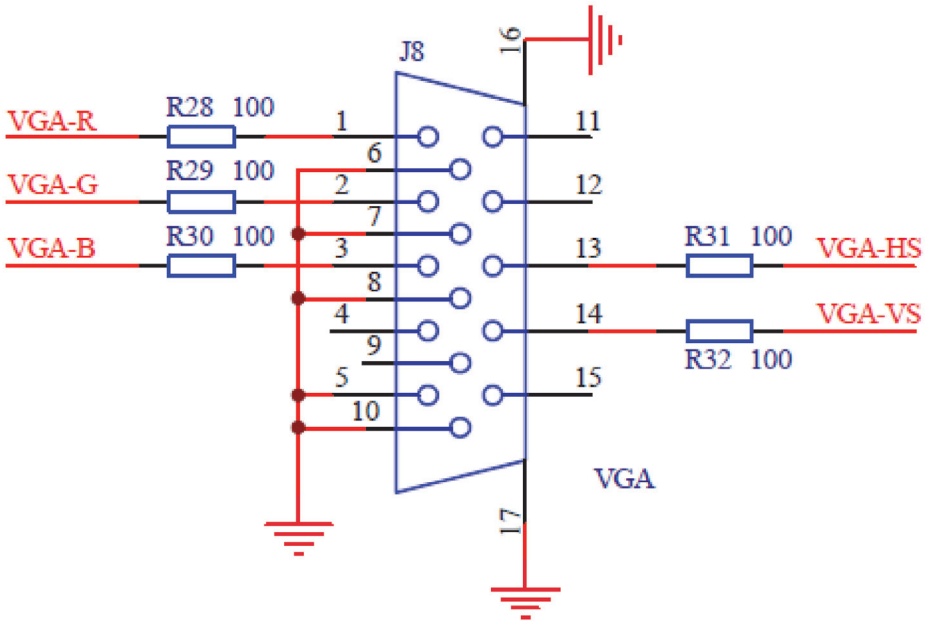


图 9-33 VGA 接口的硬件电路

表 9-25 VGA 接头引脚的定义

引脚	名称	功能	引脚	名称	功能
1	red	红基色	9	V_{DC}	5V 直流供电 （保留，各家定义不同）
2	green	绿基色	10	GND	数字地
3	blue	蓝基色	11	ID Bit	地址码（ID0 显示器标示位 0）
4	ID Bit	地址码（ID2 显示器标示位 2）	12	ID Bit	地址码（ID1 显示器标示位 1）
5	GND	自测试（各家定义不同） （一般为 GND）	13	hsync	行同步信号
6	red_gnd	红基色地	14	vsync	帧同步信号
7	green_gnd	绿基色地	15	ID Bit	地址码（ID3 显示器标示位 3） （各家定义不同）
8	blue_gnd	蓝基色地			

3. VGA 显示控制电路

VGA 显示控制电路的主要作用是产生三基色分量的数据（vga_d[blue,green,red]，即图 9-33 中的 VGA-B、VGA-G、VGA-R）、行和帧两个方向的同步信号（vga_hs、vga_vs 即图 9-33 中的 VGA-HS、VGA-VS）、消隐信号（blank）等。

根据实验任务可画出如图 9-34 所示的原理框图。vga_ctrl 模块产生行同步信号 hsync、帧同步信号 vsync、有效显示区的扫描像素值 hcnt、vcnt 和消隐信号 nblank。vga_data 模块产生三基色（vga_d）信号。sys_clk 为像素时钟 clk，orient 为垂直/水平彩条显示控制开关。

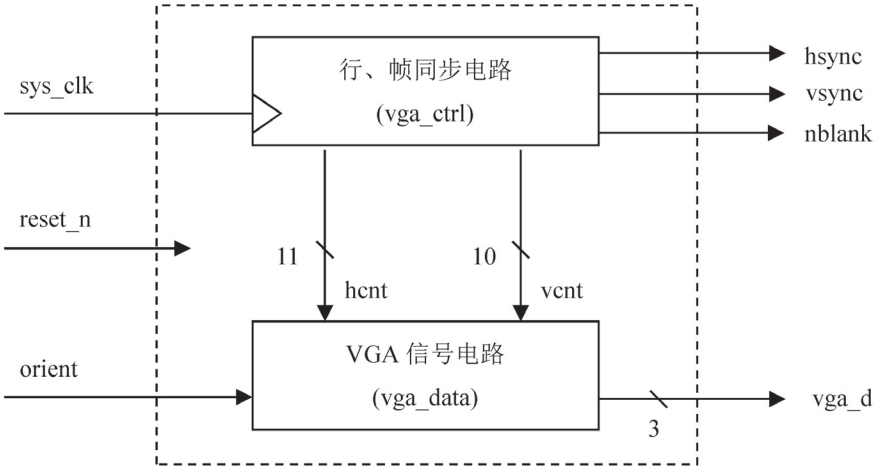


图 9-34 VGA 显示控制电路的原理框图

4. VGA 信号电路的设计

VGA 信号电路为组合电路，水平彩条信号只与帧像素有关，垂直彩条信号只与行像素有关。VGA 信号电路的功能表如表 9-26 所示，其中 vcnt 和 hcnt 分别为帧同步信号和行同步信号的扫描像素计数器，即下文中的垂直行计数器和水平像素计数器。

表 9-26 VGA 信号电路功能表

orient=1(置 ON 位)：水平条幅				
vcnt	**blue**	**green**	**red**	**颜色**
0~74	0	0	255	红
75~149	0	255	0	绿
150~224	0	255	255	黄
225~299	255	0	0	蓝
300~374	255	0	255	品红
375~449	255	255	0	青
450~524	255	255	255	白
525~699	0	0	0	黑
其它	0	0	0	黑

orient=0(置 OFF 位)：垂直条幅				
hcnt	**blue**	**green**	**red**	**颜色**
0~99	0	0	255	红
100~199	0	255	0	绿
200~299	0	255	255	黄
300~399	255	0	0	蓝
400~499	255	0	255	品红
500~599	255	255	0	青
600~699	255	255	255	白
700~799	0	0	0	黑
其它	0	0	0	黑

- **部分参考源代码（Verilog 语言）：**

主要包括实现以下两个部分的功能：

1) 得到 800*600@75Hz 的行同步信号和帧同步信号；
2) 得到相应的颜色数据。

```
//Horizontal timing constants
//根据表 9-23 可知行像素为 800，行消隐前肩时间为 16，
//行同步脉冲时间为 80，行消隐后肩时间为 160。
parameter    H_PIXELS        = 'd800, //行像素
             H_FRONTPORCH = 'd16, //行前沿
             H_SYNCTIME      = 'd80, //行同步
             H_BACKPORCH  = 'd160, //行后沿
             H_SYNCSTART    = 'd816, //H_PIXELS + H_FRONTPORCH, 行同步前
             H_SYNCEND       = 'd896, //H_SYNCSTART + H_SYNCTIME, 行同步后
             H_PERIOD        = 'd1056, //H_SYNCEND + H_BACKPORCH, 行总像素
//Vertical timing constants
//根据表 9-23 可知帧像素为 600，帧消隐前肩时间为 1，
//帧同步脉冲时间为 3，帧消隐后肩时间为 21。
             V_LINES         = 'd600, //帧像素
             V_FRONTPORCH  = 'd1, //帧前沿
             V_SYNCTIME      = 'd3, //帧同步
             V_BACKPORCH    = 'd21, //帧后沿
             V_SYNCSTART     = 'd601, //V_LINES + V_FRONTPORCH, 帧同步前
             V_SYNCEND       = 'd604, // V_SYNCSTART + V_SYNCTIME, 帧同步后
             V_PERIOD        = 'd625; //V_SYNCEND + V_BACKPORCH, 帧总像素

//行、帧同步电路 vga_ctrl 为时序电路，可用计数器方法设计，计数器的模为一个扫描周期。
//行同步信号由同步(Hsynch)、消隐前肩(Hfp)、消隐后肩(Hbp)和有效显示区(Hactive)
//共 4 个阶段组成，因此行的总像素数为 1056(H_PERIOD)，
//即整个行扫描周期为 1056，计数器的模也为 1056。
//Horizontal counter of pixels 水平像素计数器
always @ (posedge clk_50m or negedge reset_n) //clk_50m 为系统时钟信号 50MHz
if(!reset_n)
    hcnt <= 0; //hcnt 为水平像素计数器
else if(hcnt < H_PERIOD)
    hcnt <= hcnt + 1;
else
    hcnt <= 0;

//行同步信号仅在同步脉冲(Hsynch)即 H_SYNCSTART 到 H_SYNCEND 阶段为有效低电平。
//Internal horizontal synchronization pulse generation (negative polarity)
//内部水平同步脉冲的产生（负极性）
always @ (posedge clk_50m or negedge reset_n)
if(!reset_n)
    hsyncint <= 1; //hsyncint 为行同步计数信号
else if(hcnt >= H_SYNCSTART && hcnt < H_SYNCEND) //产生行同步信号
    hsyncint <= 0;
else
    hsyncint <= 1;

//Horizontal synchronization output 水平同步输出
```

```
assign vga_hs = hsyncint; //vga_hs 为行同步信号（低电平有效）

//帧同步电路设计与行同步电路设计类似。
//帧同步信号由同步脉冲(Vsynch)、消隐前肩(Vfp)、消隐后肩(Vbp)和有效显示区(Vactive)
//共 4 个阶段组成，因此帧的总像素数为 625(V_PERIOD)，
//即整个帧扫描周期为 625，计数器的模也为 625。
//Vertical counter of lines 垂直行计数器
always @ (posedge hsyncint or negedge reset_n)
if(!reset_n)
    vcnt <= 0; //vcnt 为垂直行计数器
else if(vcnt < V_PERIOD)
    vcnt <= vcnt + 1;
else
    vcnt <= 0;

//帧同步信号仅在同步脉冲(Vsynch)即 V_SYNCSTART 到 V_SYNCEND 阶段为有效低电平。
//Vertical synchronization pulse generation (negative polarity)
//垂直同步脉冲的产生（负极性）
always @ (posedge hsyncint or negedge reset_n)
if(!reset_n)
    vga_vs <=1; //vga_vs 为帧同步信号（低电平有效）
else if(vcnt >= V_SYNCSTART && vcnt < V_SYNCEND) //产生帧同步信号
    vga_vs <=0;
else
    vga_vs <=1;

//只有在行 800 像素以及帧 600 像素内才是有效显示区域，其余区域均需要消隐。
//Enabling of color outputs  使能与消隐颜色输出
always @ (posedge clk_50m or negedge reset_n)
if(!reset_n)
    enable<=0; //enable 为消隐使能信号
else if(hcnt >= H_PIXELS || vcnt >= V_LINES) //不在有效区域中的行、帧都进行消隐
    enable<=0; //消隐
else
    enable<=1;

assign blank = enable; //输出消隐信号（低电平有效）

//VGA 信号电路 vga_data 为组合电路。
//Output image generation (horizontal or vertical color stripes)
//输出图像生成（水平或垂直的彩色条纹）
always @ (enable or orient or hcnt or vcnt)
    //orient 为拨码开关 CS1 控制信号（高电平时置为 OFF）
    if(enable==0) //消隐
        vga_d = 0; //vga_d 为三基色输出信号，从高位到低位分别表示蓝、绿、红
    else if(orient) //拨码开关  CS1 控制显示水平条幅、还是垂直条幅
```

```
    //水平彩条信号只与帧像素有关，8 种颜色平分 600 帧像素。
      begin                          //水平条幅，8 个颜色
          if(vcnt<75)
              vga_d='h1;             //红
          else if(vcnt<150)
              vga_d='h2;             //绿
          else if(vcnt<225)
              vga_d='h3;             //黄
          else if(vcnt<300)
              vga_d='h4;             //蓝
          else if(vcnt<375)
              vga_d='h5;             //品红
          else if(vcnt<450)
              vga_d='h6;             //青
          else if(vcnt<525)
              vga_d='h7;             //白
          else
              vga_d='h0;             //黑；其它均为黑，相当于消隐效果
      end
  else
  //垂直彩条信号只与行像素有关，8 种颜色平分 800 行像素。
      begin                          //垂直条幅，8 个颜色
          if(hcnt<100)
              vga_d='h1;             //红
          else if(hcnt<200)
              vga_d='h2;             //绿
          else if(hcnt<300)
              vga_d='h3;             //黄
          else if(hcnt<400)
              vga_d='h4;             //蓝
          else if(hcnt<500)
              vga_d='h5;             //品红
          else if(hcnt<600)
              vga_d='h6;             //青
          else if(hcnt<700)
              vga_d='h7;             //白
          else
              vga_d='h0;             //黑；其它均为黑，相当于消隐效果
      end
```

附录：用控制器方法实现行、帧同步电路

行、帧同步电路除了可以用上述的计数器方法实现之外，还可以用控制器方法来设计。图 9-35 所示为行、帧同步电路的结构框图。

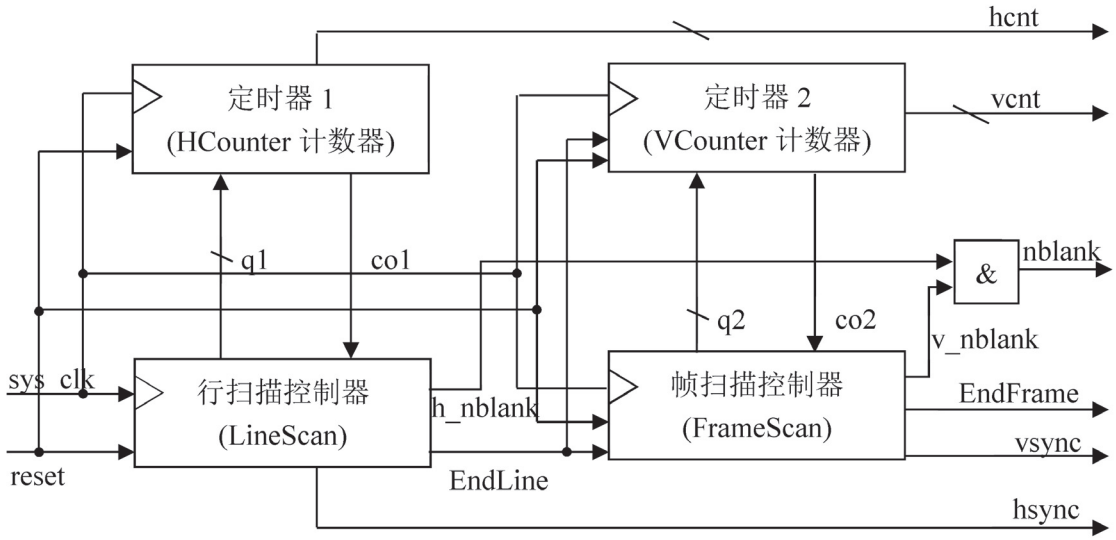


图 9-35 行、帧同步电路的结构框图

行同步信号由同步（Hsynch）、消隐前肩（Hfp）、消隐后肩（Hbp）和有效显示区（Hactive）4 个阶段组成，因此行扫描控制器有相应的 4 个状态。因为每个阶段持续时间不同，即在不同阶段定时器的定时时间不同，所以定时器应由行扫描控制器的状态来决定定时时间。信号 co1 为定时结束标志，宽度为 1 个像素时钟周期，同时 co1 还使定时器复位。

行扫描控制器的输出 EndLine 为一行的结束信号，宽度也为 1 个像素时钟周期，该信号可以作为帧扫描控制器的时钟使能信号；帧结束信号 EndFrame 仅用于仿真，不必输出。

行扫描控制器的算法状态机（ASM）图如图 9-36 所示。

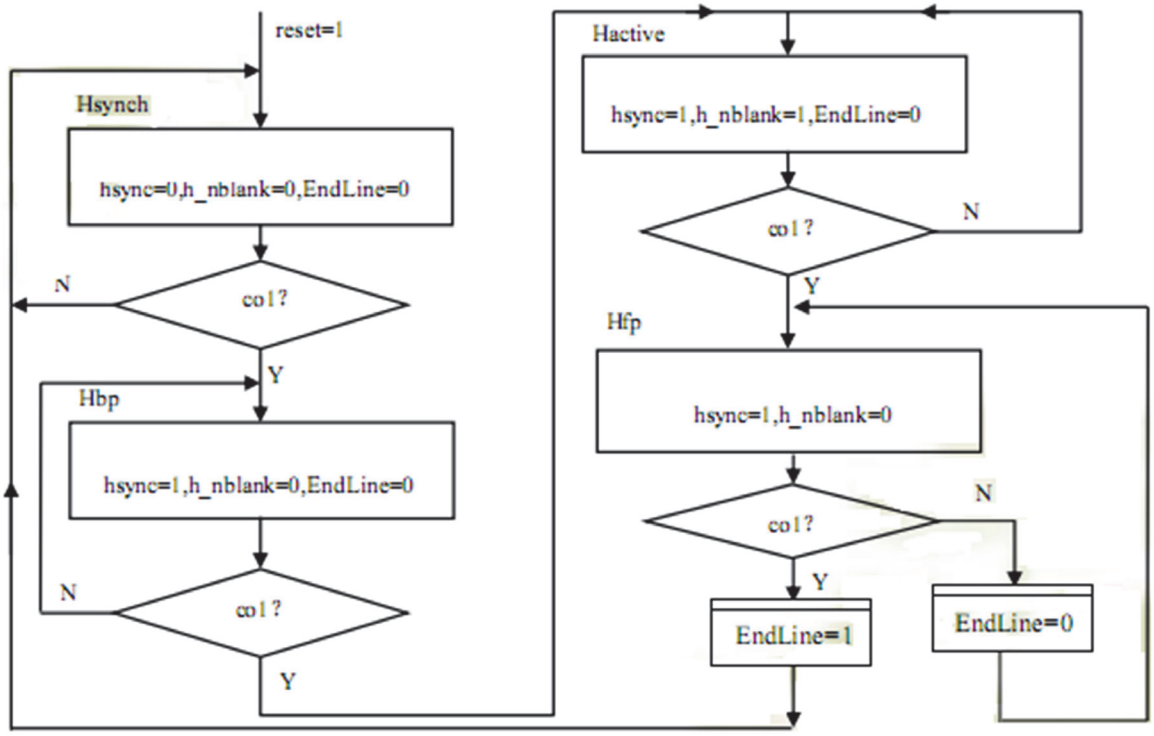


图 9-36 行扫描控制器的 ASM 图

行扫描控制器的定时器是可控多模定时器，定时器在行同步、行消隐后肩、有效显示区和行消隐前肩 4 个阶段的模分别为 80、160、800 和 16。定时结束标志 co1 即为计数器的进位，最后一个阶段（行消隐前肩）的进位即为行结束标志 EndLine。另外，只有在有效显示区（Hactive）显示坐标才有意义，因此定时器和显示坐标计数器可以共用。

帧扫描控制器与行扫描控制器设计类似，只需要注意两点：一是只有在 EndLine=1 时，状态才能转换；二是在消隐前肩阶段，不需要定时器。帧扫描控制器的 ASM 图如图 9-37 所示。

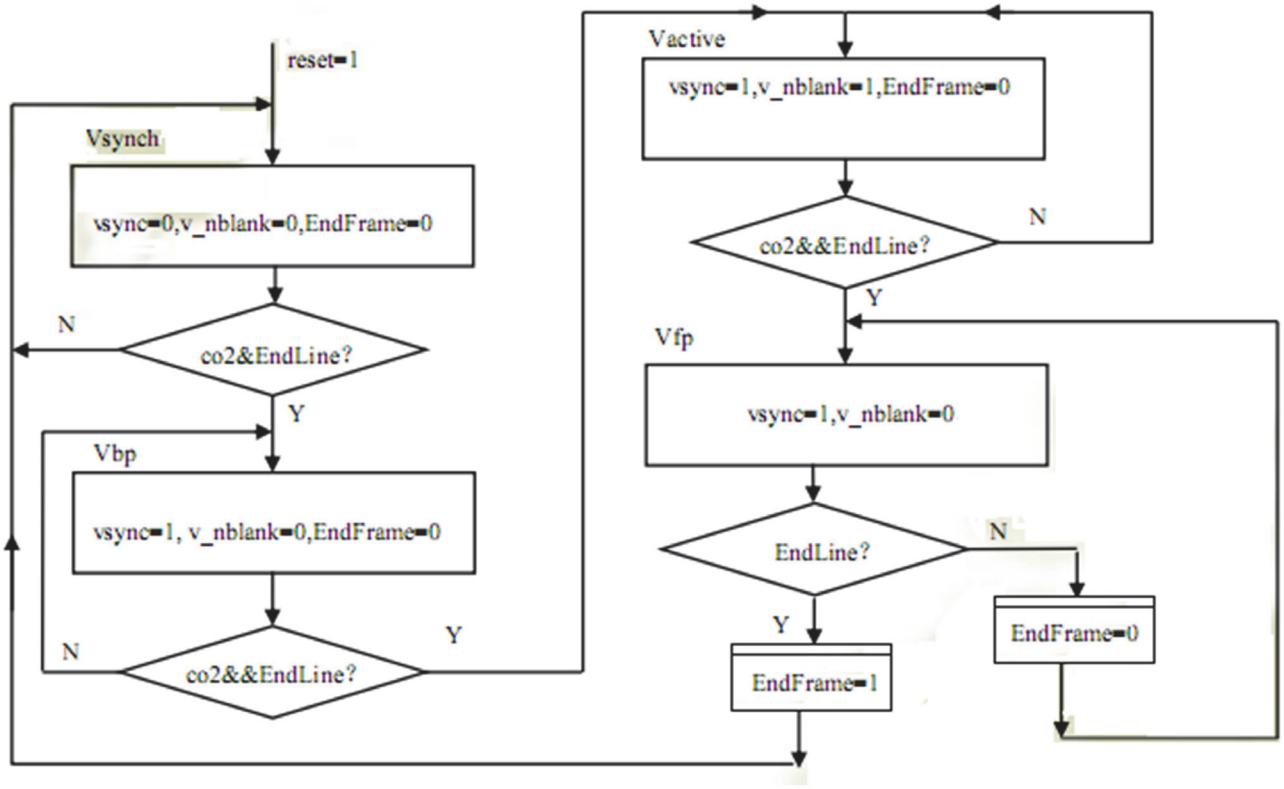


图 9-37 帧扫描控制器的 ASM 图

- 控制器设计的部分参考源代码（**Verilog** 语言）：

vga_top.v 的部分源代码：

```
vga_ctrl vga_ctrl(
        .reset(~reset_n),
        .sys_clk(sys_clk), //系统时钟信号
        .hsync(hsync), //行同步信号，低电平有效
        .vsync(vsync), //帧同步信号，低电平有效
        .EndFrame(EndFrame), //一帧扫描结束标志
        .nblank(nblank), //消隐信号，低电平有效
        .hcnt(hcnt),
        .vcnt(vcnt));
vga_data vga_data(
        .orient(orient), //拨码开关 CS1 控制信号（高电平为置 OFF）
        .nblank(nblank),
        .hcnt(hcnt),
        .vcnt(vcnt),
        .vga_d(vga_d)); //三基色输出信号，从高位到低位分别表示蓝、绿、红
```

vga_ctrl.v 的部分源代码：

```
//行、帧同步电路顶层模块
HCounter HCounter(
        .reset(reset),
        .sys_clk(sys_clk),
        .q1(q1),
        .co1(co1),
```

```
        .hcnt(hcnt));
VCounter VCounter(
        .reset(reset),
        .sys_clk(sys_clk),
        .EndLine(EndLine),
        .q2(q2),
        .co2(co2),
        .vcnt(vcnt));
LineScan LineScan(
        .reset(reset),
        .sys_clk(sys_clk),
        .co1(co1),
        .hsync(hsync),
        .h_nblank(h_nblank),
        .EndLine(EndLine),
        .q1(q1));
FrameScan FrameScan(
        .reset(reset),
        .sys_clk(sys_clk),
        .co2(co2),
        .EndLine(EndLine),
        .vsync(vsync),
        .v_nblank(v_nblank),
        .EndFrame(EndFrame),
        .q2(q2));

//消隐信号包括行的消隐与帧的消隐
assign nblank=h_nblank&&v_nblank;
```

vga_data.v 的部分源代码：

```
//VGA 信号电路，输出图像生成（水平或垂直的彩色条纹）
always @(*)
begin
    //消隐信号低电平有效时，三基色输出信号为 0，即黑色
    if(nblank==0)
    vga_d=0;

    //拨码开关 CS1 控制显示水平条幅、还是垂直条幅
    else if(orient)
        //水平彩条信号只与帧像素有关，8 种颜色平分 600 帧像素
        if(vcnt<75) vga_d=1; //红
        else if(vcnt<150) vga_d=2; //绿
        else if(vcnt<225) vga_d=3; //黄
        else if(vcnt<300) vga_d=4; //蓝
        else if(vcnt<375) vga_d=5; //品红
        else if(vcnt<450) vga_d=6; //青
```

```
            else if(vcnt<525) vga_d=7; //白
            else vga_d=0; //黑

        else
            //垂直彩条信号只与行像素有关，8 种颜色平分 800 行像素
            if(hcnt<100) vga_d=1; //红
            else if(hcnt<200) vga_d=2; //绿
            else if(hcnt<300) vga_d=3; //黄
            else if(hcnt<400) vga_d=4; //蓝
            lse if(hcnt<500) vga_d=5; //品红
            else if(hcnt<600) vga_d=6; //青
            else if(hcnt<700) vga_d=7; //白
            else vga_d=0; //黑
end
```

HCounter.v 的部分源代码：

```
//行扫描控制器的计数器
assign co1=(hcnt==cnt-1); //co1 为定时结束标志，即计数器进位信号；cnt 为计数器的模

always @(posedge sys_clk)
    begin
        case(q1) //q1 为行扫描控制器输出的控制器当前状态
            //行扫描控制器的 4 个状态分别对应行同步信号的 4 个阶段
            //根据行扫描控制器 ASM 图可以确定控制器每个状态计数器的模
            //Hsynch:80 Hbp:160 Hactive:800 Hfp:16
            2'b00: cnt=80; //行同步
            2'b01: cnt=160; //行后沿
            2'b10: cnt=800; //行像素
            2'b11: cnt=16; //行前沿
            default:cnt=80;
        endcase

        //系统重置或每一阶段计数完成时，计数器清零
        if(reset||co1) hcnt=0;
        else hcnt=hcnt+1;
    end
```

VCounter.v 的部分源代码：

```
//帧扫描控制器的计数器
assign co2=EndLine&&(vcnt==cnt-1); //co2 为定时结束标志，即计数器进位信号
//EndLine 为行扫描器扫描完一行结束的标志；cnt 为计数器的模

always @(posedge sys_clk)
    begin
        case(q2) //q2 为帧扫描控制器输出的控制器当前状态
        //帧扫描控制器的 4 个状态分别对应帧同步信号的 4 个阶段
```

```
        //根据帧扫描控制器 ASM 图可以确定控制器每个状态计数器的模
        //Vsynch:3 Vbp:21 Vactive:600 Vfp:1
        2'b00: cnt=3; //帧同步
        2'b01: cnt=21; //帧后沿
        2'b10: cnt=600; //帧像素
        2'b11: cnt=1; //帧前沿
        default:cnt=3;
        endcase

        //系统重置或每一阶段计数完成时，计数器清零。
        if(reset||co2) vcnt=0;
        //每扫描完一行时，帧扫描计数器加 1
        else if(EndLine) vcnt=vcnt+1;
    end
```

LineScan.v 的部分源代码：

```
//行扫描控制器
always @(posedge sys_clk)
    if(reset) q1<=Hsynch; //q1 为行扫描控制器当前状态
    else q1<=nextstate;

    //根据行扫描控制器 ASM 图设计
    always @(*)
    begin
        hsync<=0; h_nblank<=0; EndLine<=0;
        //hsync 为行同步信号；h_nblank 为行扫描消隐时间；EndLine 为一行扫描结束标志
        case(q1)
            Hsynch: if(co1) nextstate<=Hbp; else nextstate<=Hsynch;
                    //co1 为行扫描控制器的计数器定时结束标志
            Hbp: begin
                    hsync<=1;
                    if(co1) nextstate<=Hactive;
                    else nextstate<=Hbp;
                end
            //只有在行的有效显示区进行显示，其他区域均进行消隐
            Hactive: begin
                        hsync<=1; h_nblank<=1;
                        if(co1) nextstate<=Hfp;
                        else nextstate<=Hactive;
                    end
            //消隐后肩的下一状态进入同步阶段时，
            //说明一行扫描完成，输出一行扫描结束标志
            Hfp: begin
                    hsync<=1;
                    if(co1) begin nextstate<=Hsynch; EndLine<=1; end
                    else begin nextstate<=Hfp; EndLine<=0; end
```

```
                end
            default: nextstate<=Hsynch;
        endcase
    end
```

FrameScan.v 的部分源代码：

```
//帧扫描控制器
always @(posedge sys_clk)
    if(reset) q2<=Vsynch; //q2 为帧扫描控制器当前状态
    else q2<=nextstate;

    //根据帧扫描控制器 ASM 图设计
    always @(*)
    begin
        vsync<=0; v_nblank<=0; EndFrame<=0;
        //vsync 为帧同步信号；v_nblank 为帧扫描消隐时间；EndFrame 为一帧扫描结束标志
        case(q2)
            Vsynch: if(co2&&EndLine) nextstate<=Vbp; else nextstate<=Vsynch;
                    //只有在帧计数器一个状态计数完成，并且到最后一行扫描结束，
                    //即 co2=1 且 EndLine=1 时，才进行状态转换
                    //co2 为帧扫描控制器的计数器定时结束标志
            Vbp: begin
                        vsync<=1;
                        if(co2&&EndLine) nextstate<=Vactive;
                        else nextstate<=Vbp;
                 end
            //只有在帧的有效显示区进行显示，其他区域均进行消隐
            Vactive: begin
                        vsync<=1; v_nblank<=1;
                        if(co2&&EndLine) nextstate<=Vfp;
                        else nextstate<=Vactive;
                     end
            //消隐后肩的下一状态进入同步阶段时，
            //说明一帧扫描完成，输出一帧扫描结束标志
            Vfp: begin
                        vsync<=1;
                        if(EndLine) begin nextstate<=Vsynch; EndFrame<=1; end
                        else begin nextstate<=Vfp; EndFrame<=0; end
                 end
            default: nextstate<=Vsynch;
        endcase
    end
```